

GitHub Capstone - PROJECT STEP 2

Detailed click-by-click edition

From creating a Step 2 branch and documenting a data source to cloning in VS Code, running an executed Jupyter notebook, staging, committing, syncing, and saving evidence.

Who should use this guide?

Every allocated capstone team. SC11 Water Quality Potability is used only as the example. Replace its repository name, dataset file, fields, and project-specific logic with your own assigned project details.

Step 2 goal

Prepare a cited, clean, validated and reproducible data or scenario pipeline. Step 2 does not mean building the final ANN, fuzzy system, GA, or user interface yet.

How to use this guide

Follow each action in order. Do not treat screenshots as decoration: they are visual checkpoints. Stop if your screen does not match the expected result.

WHERE TO CLICK

The screenshot and action text identify the button, menu, folder, or screen area.

DO THIS

Perform the stated click or entry. Use your own project name, branch, source and dataset where the example uses SC11.

EXPECTED RESULT

Verify the named folder, output, branch, or Git state before moving on.

Important beginner words

Clone	Create a local working copy of the shared GitHub repository on your computer.
Branch	A separate safe line of work. Step 2 changes can be developed without immediately altering main.
Stage	Select the exact saved files that should enter the next commit.
Commit	Create a named Git checkpoint that records a meaningful unit of work.
Sync	Upload local commits to GitHub and download teammates' changes.
Notebook	An .ipynb file that contains explanation, code, and visible executed output.

Action 1 - Confirm the correct repository

Open your own team repository. Do not work inside the central catalogue repository or a classmate's repository.

The screenshot shows a GitHub repository page for 'vipulshahi-ai / SC11-Water-Quality-Potability-Example'. The repository is public and has 13 commits. The main branch is 'main'. The repository contains the following files and folders:

File/Folder	Description	Commit Date
data	Fix missing pH value in WTR-005 to pass validation	2 weeks ago
docs	Add files via upload	2 weeks ago
src	Add validation script and fix missing pH value in sample data	2 weeks ago
README.md	Update README.md	2 weeks ago
requirements.txt	Create requirements.txt	2 weeks ago

The README file is selected, showing the title 'SC11-Water-Quality-Potability-Example' and the section 'Data Dictionary'. The right sidebar shows the repository's metadata, including the number of stars (0), forks (0), and watchers (0). The repository is owned by 'vipulshahi-ai'.

DO THIS

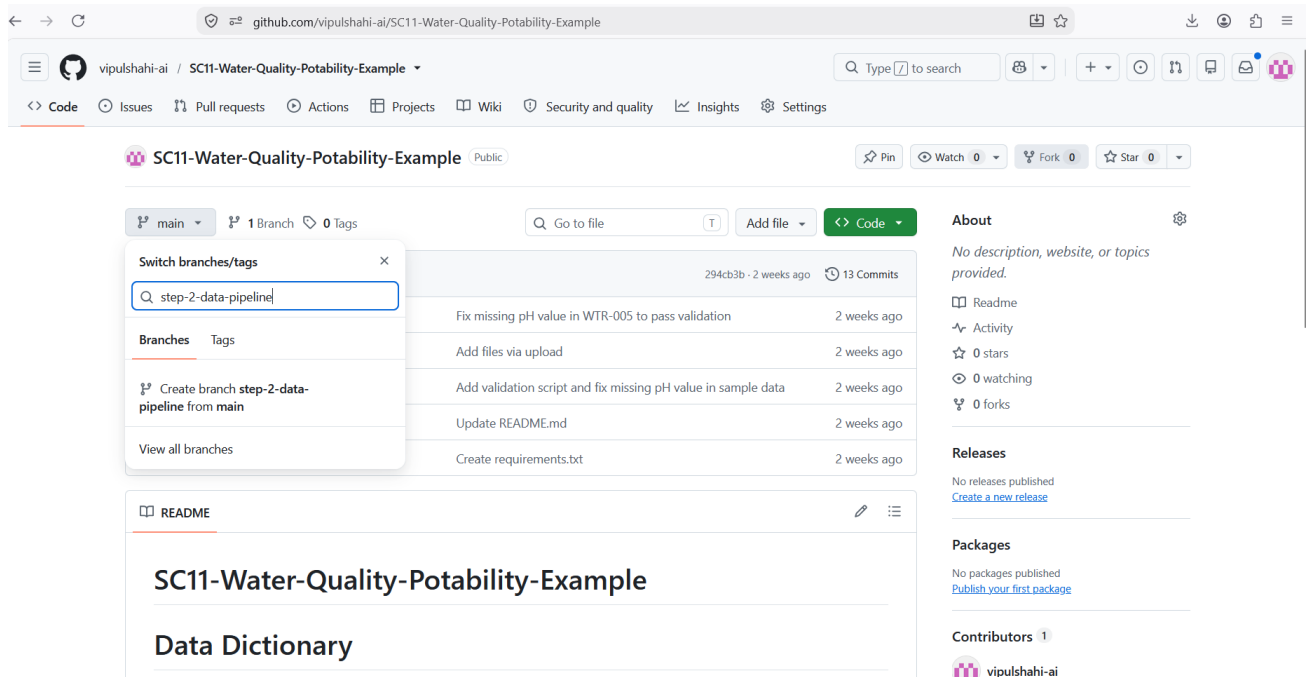
The repository name appears at the top. Check that it belongs to your team and that you are signed into the intended GitHub account.

EXPECTED RESULT

You should see your project files and a green Code button.

Action 2 - Open the branch menu

A branch is a safe working version of the project. It lets the team add Step 2 work without changing the stable main version immediately.



DO THIS

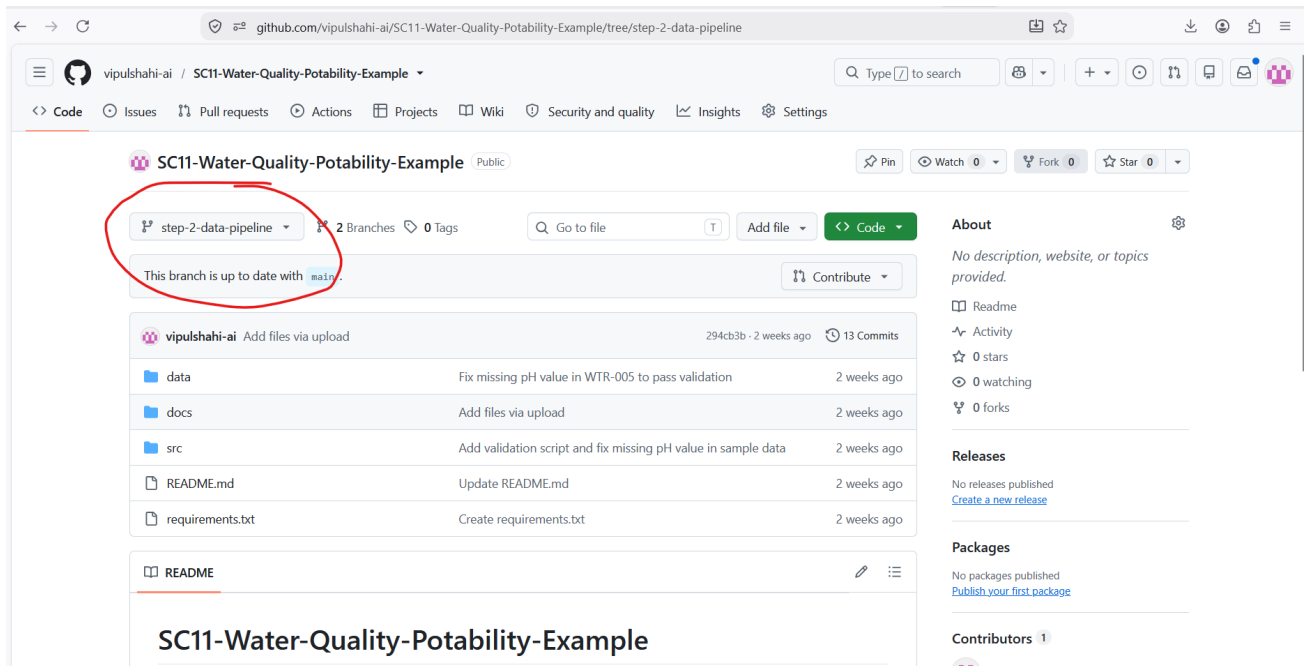
Above the file list, click the branch name (usually main). A menu opens where you can search or create branches.

EXPECTED RESULT

You should see the branch search box and the existing main branch.

Action 3 - Create the Step 2 branch

Create a branch named step-2-data-pipeline. Use the same branch name across the team so everyone is working in one shared Step 2 version.



DO THIS

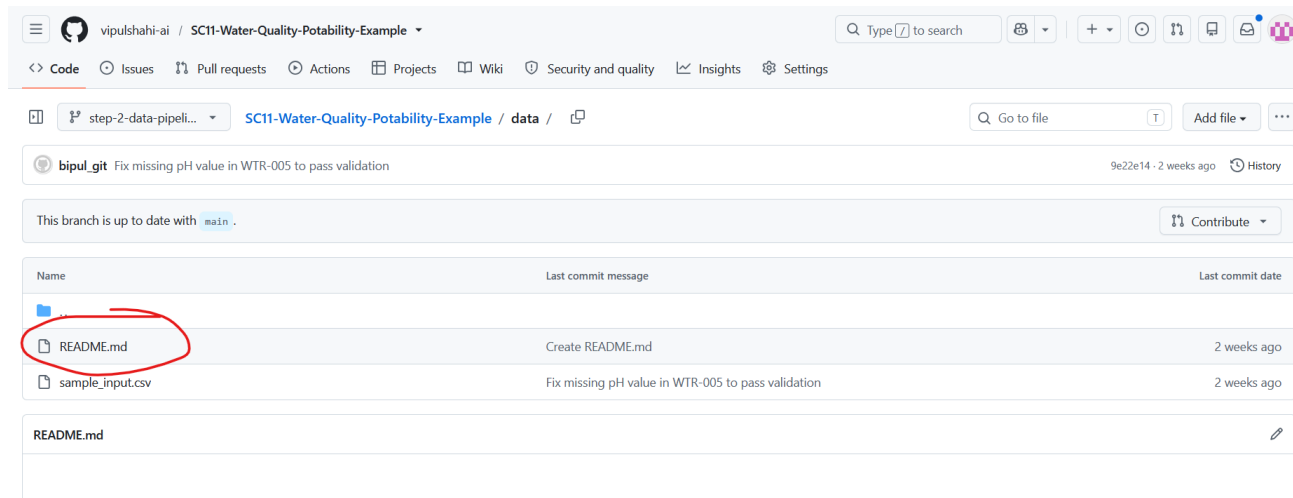
Type step-2-data-pipeline in the branch box, then select Create branch from main.

EXPECTED RESULT

The branch label above the file list must change to step-2-data-pipeline.

Action 4 - Create the data documentation file

Data documentation explains where data came from, what every column means, and how another person can repeat the work.



DO THIS

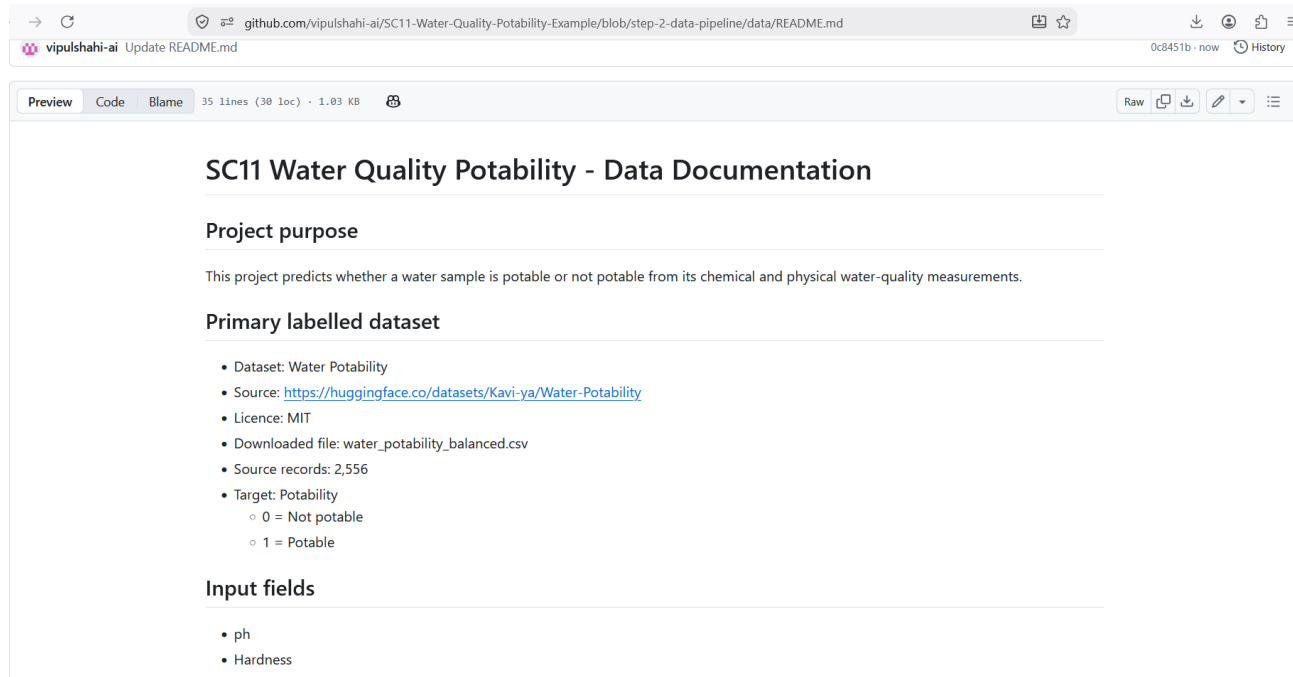
Use Add file > Create new file. Name it data/README.md. Typing a slash creates the data folder automatically.

EXPECTED RESULT

The filename field should show data/README.md before you write content.

Action 5 - Write the data source and plan

Every project must cite its dataset or the realistic ranges used to generate scenarios. This prevents anonymous or unexplained data from being used.



DO THIS

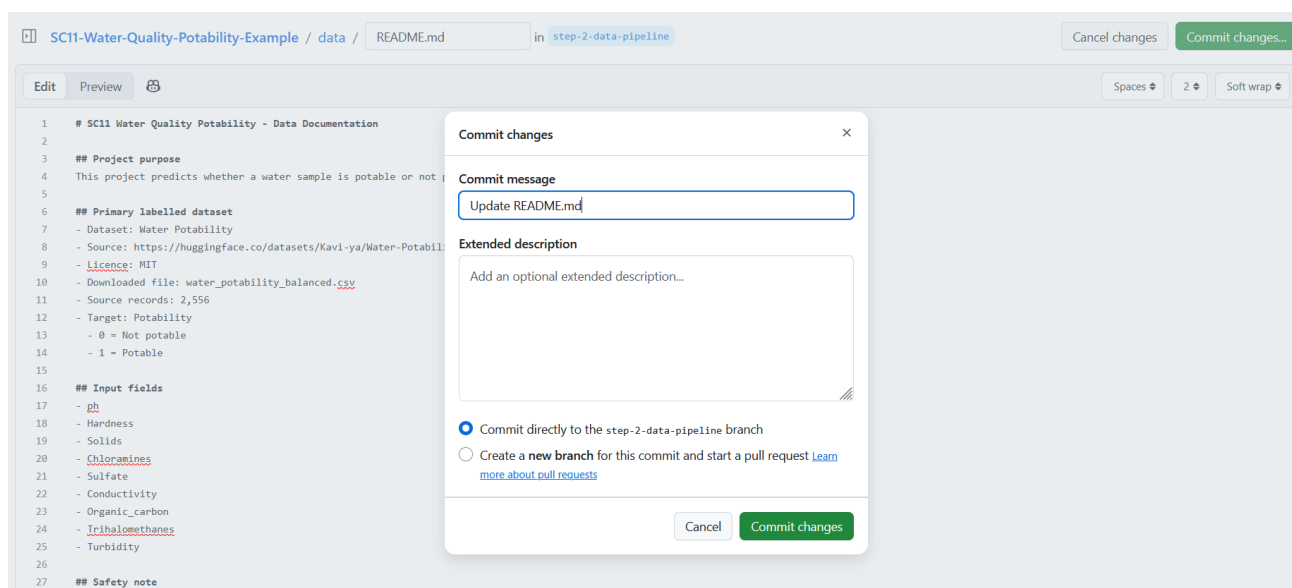
State the source URL, licence, original record count, target meaning, input fields, units, limitation, and the Step 2 plan.

EXPECTED RESULT

The README preview should clearly show headings such as Primary labelled dataset and Step 2 plan.

Action 6 - Commit the documentation

A commit is a named saved checkpoint in Git history. It records what changed and who made the change.



DO THIS

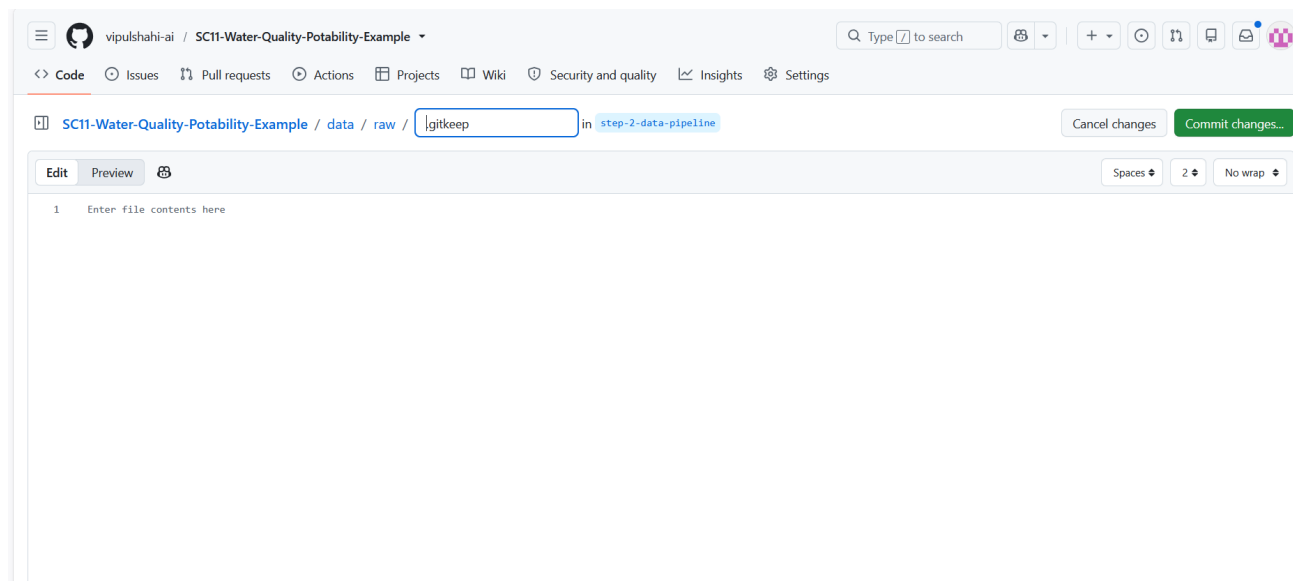
Enter a specific message such as Document SC11 data source and Step 2 plan, then click Commit changes.

EXPECTED RESULT

The commit must be made to step-2-data-pipeline, not main.

Action 7 - Create the raw-data folder

Raw data means the original downloaded file before cleaning. Never overwrite it; keeping it allows your work to be checked and repeated.



DO THIS

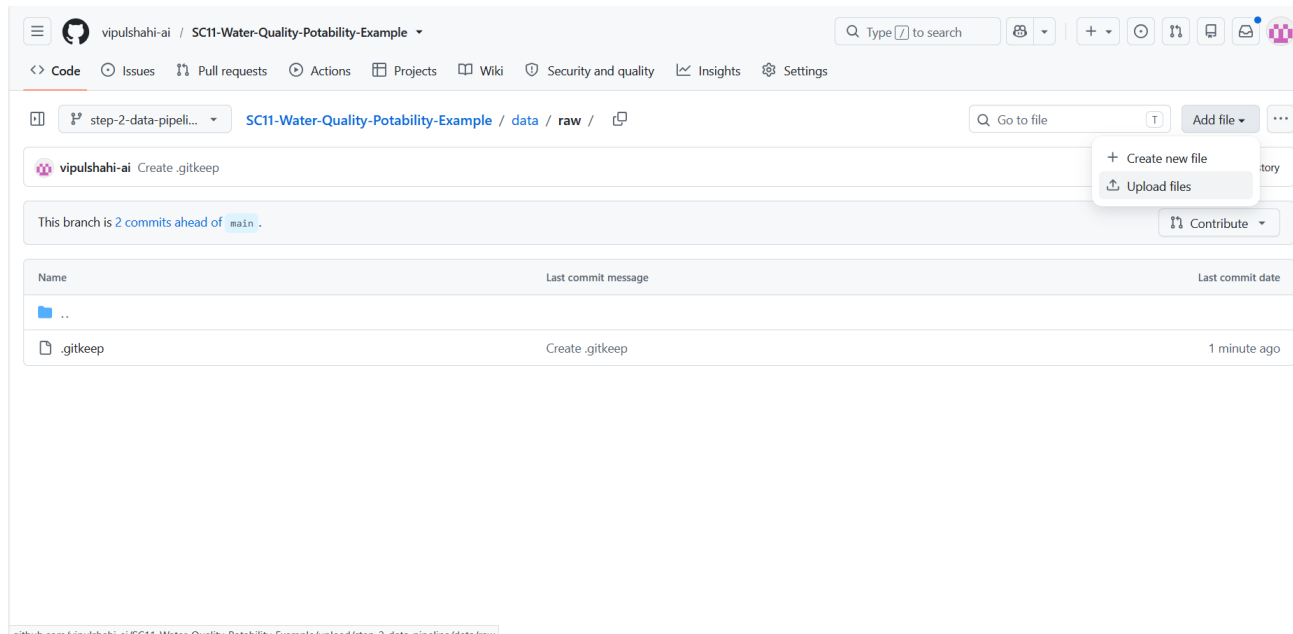
Create data/raw/.gitkeep. GitHub does not store completely empty folders, so .gitkeep makes the folder visible.

EXPECTED RESULT

You should see data/raw in the repository file path.

Action 8 - Upload the original dataset

Upload the original file exactly once into data/raw. Do not rename columns or edit the original before saving this copy.



DO THIS

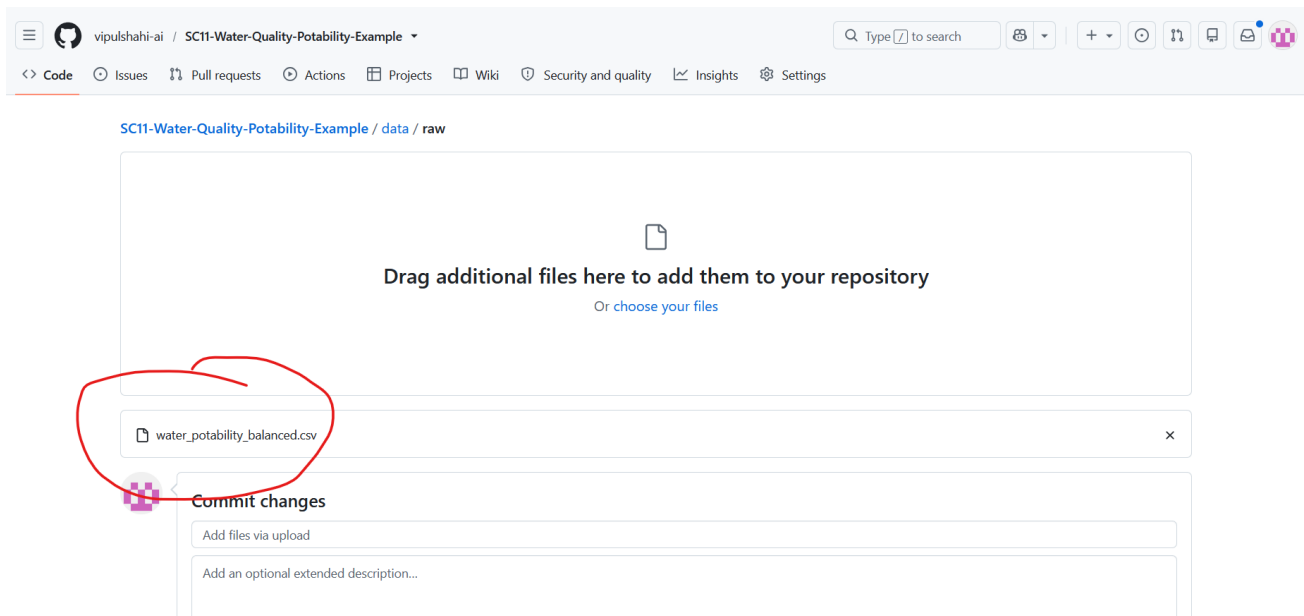
Open data/raw, select Add file > Upload files, then choose the downloaded CSV.

EXPECTED RESULT

The upload page should show a Drag files here area.

Action 9 - Select the dataset file

This action places the actual source dataset in the repository. It is evidence of the source used in the project.



DO THIS

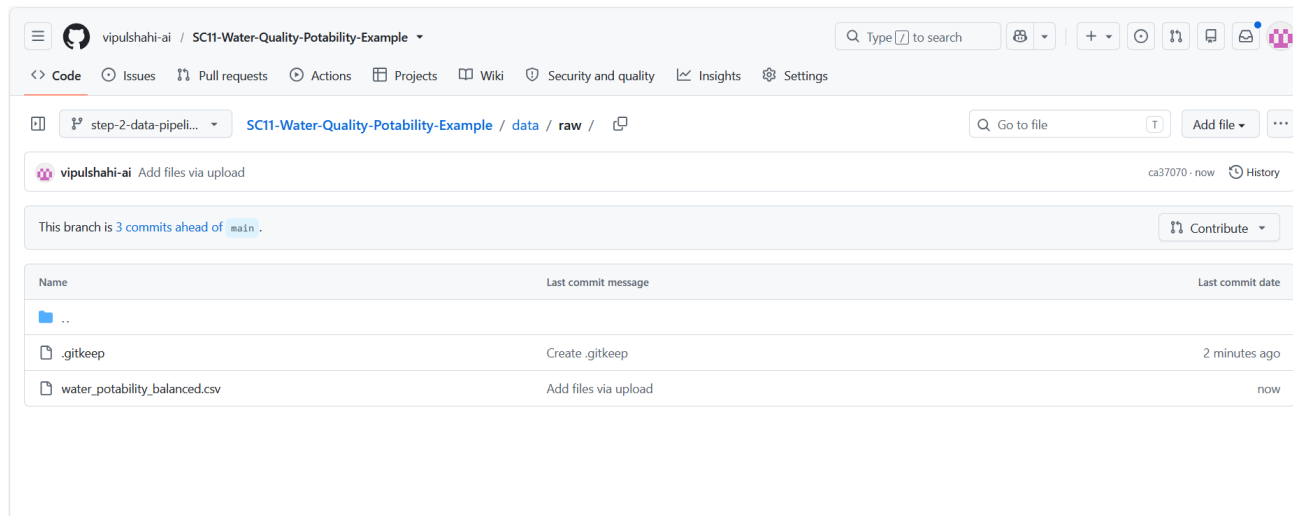
Choose the downloaded CSV, confirm its name appears in the upload list, then add a clear commit message.

EXPECTED RESULT

The selected filename must appear before committing.

Action 10 - Verify the raw file

Verification is important: a successful upload means the file is visible in the intended folder after the commit, not merely selected on your computer.



DO THIS

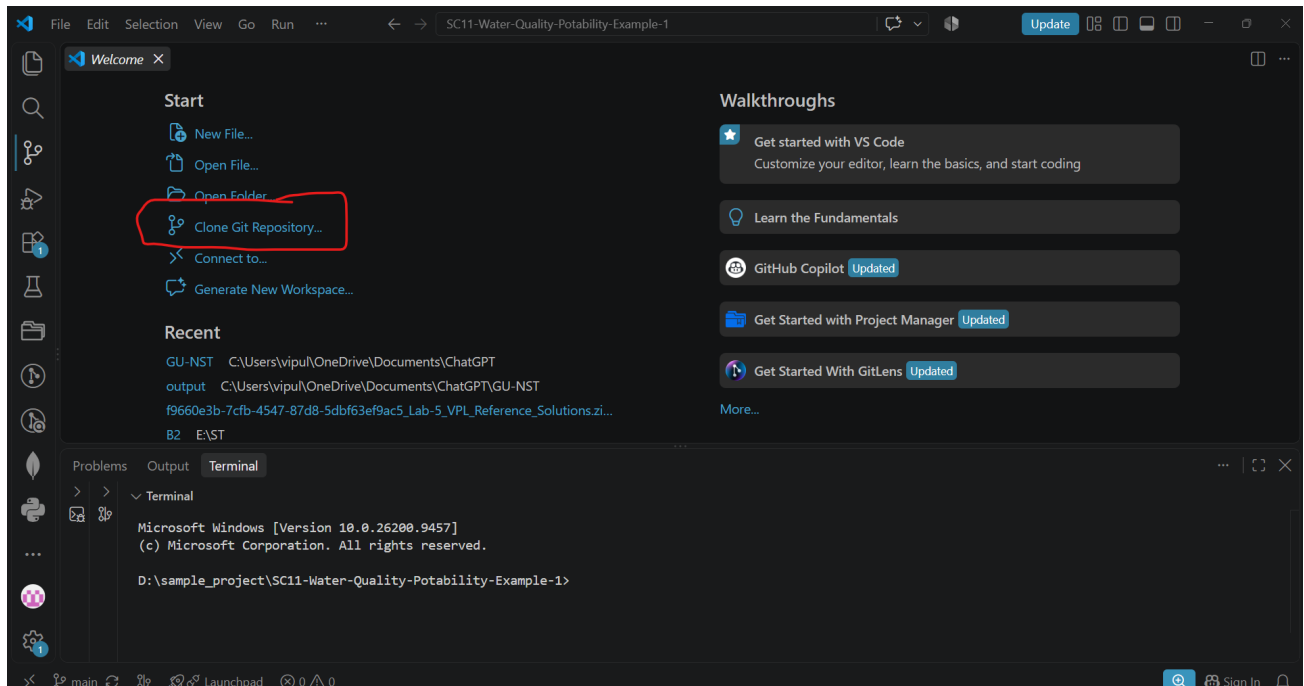
Open data/raw and verify that the CSV is listed there.

EXPECTED RESULT

The raw dataset file should be visible beside .gitkeep.

Action 11 - Start cloning in VS Code

Cloning means making a local working copy of the GitHub repository on your computer. It lets you run Python and notebooks while keeping GitHub as the shared backup.



DO THIS

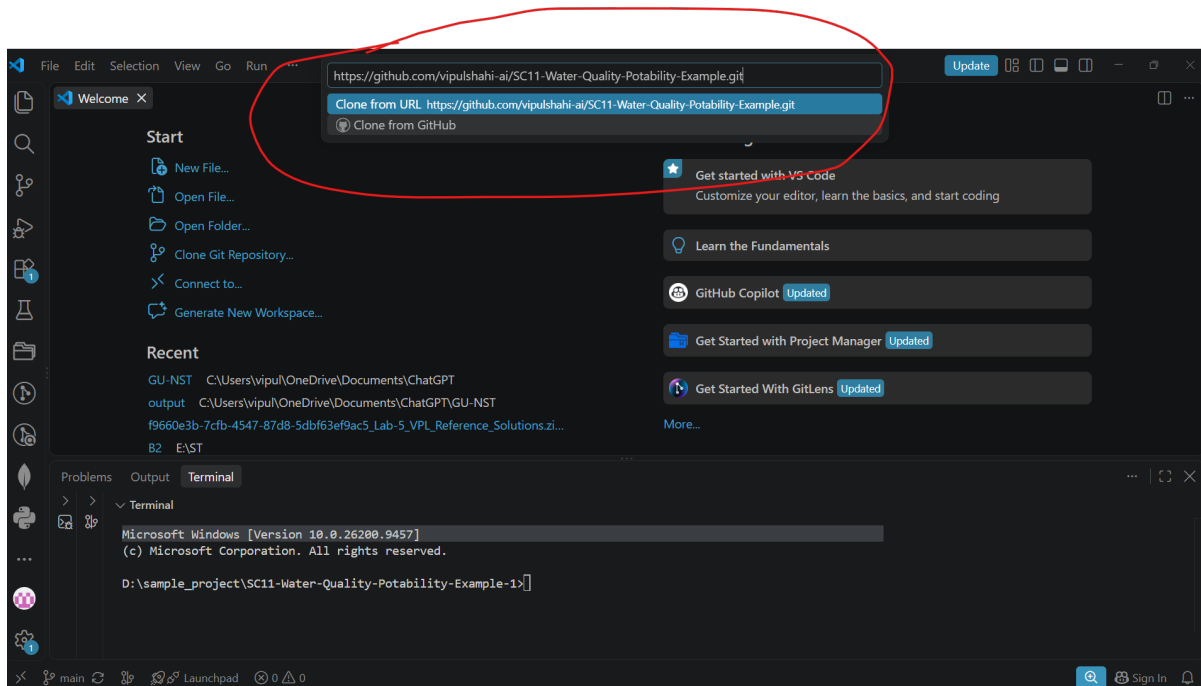
In VS Code, open Source Control and click Clone Repository.

EXPECTED RESULT

VS Code should ask for the repository URL.

Action 12 - Paste the GitHub repository URL

Use the HTTPS repository URL copied from GitHub's green Code button. Every team member clones the same team repository, not a downloaded ZIP folder.



DO THIS

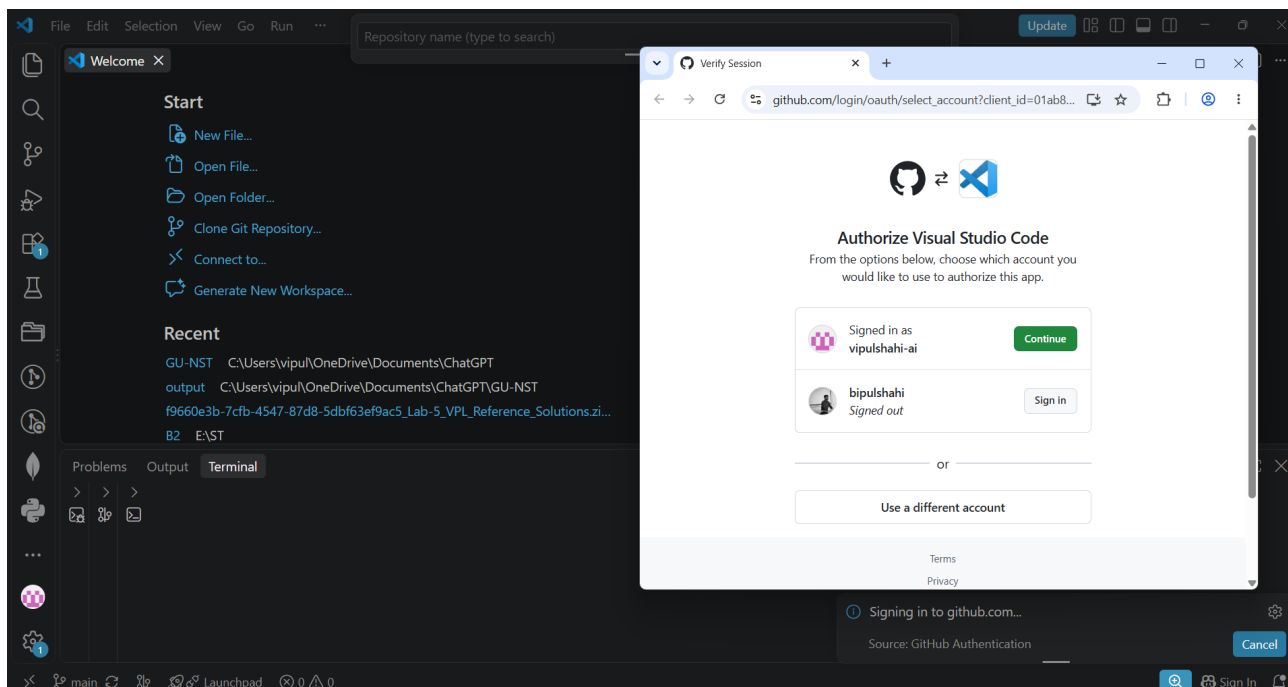
Paste the URL in the Clone Repository box and continue.

EXPECTED RESULT

The full GitHub URL should be visible in the input box.

Action 13 - Authorize GitHub when asked

VS Code needs GitHub permission to clone private repositories and to later publish your commits. This is an account connection, not a new repository.



DO THIS

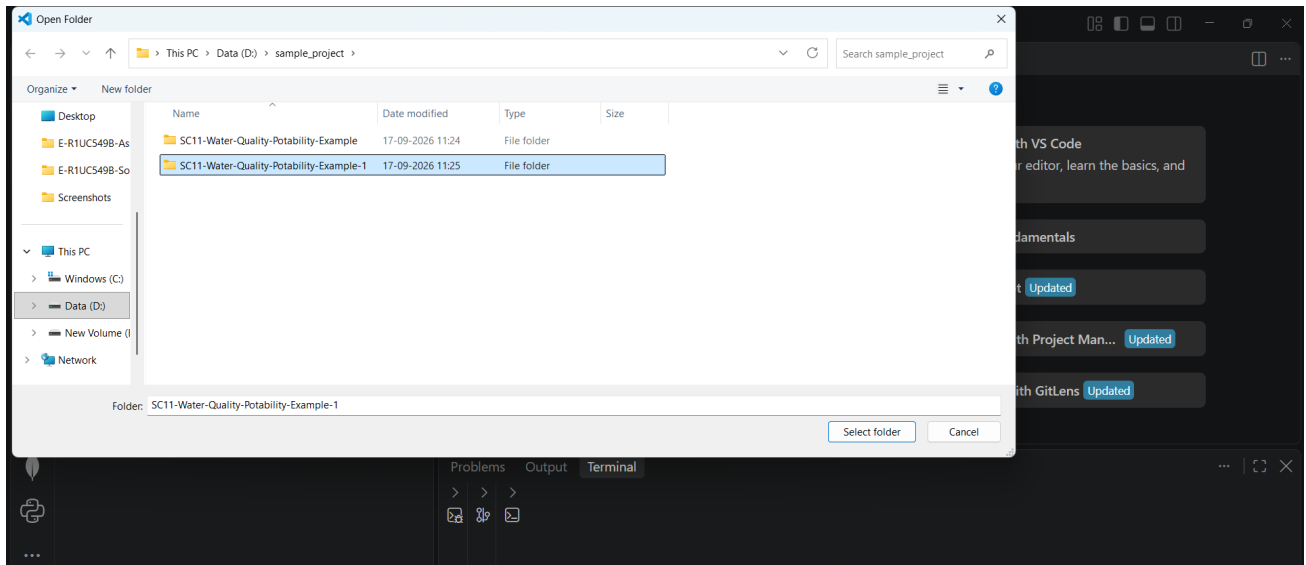
Complete sign-in only with your own GitHub account. Select the correct account if more than one is shown.

EXPECTED RESULT

GitHub should show a confirmation screen for Visual Studio Code access.

Action 14 - Choose the local project folder

Choose one easy-to-find folder, for example Documents or a Projects folder. Do not save the project inside Downloads where it may be deleted accidentally.



DO THIS

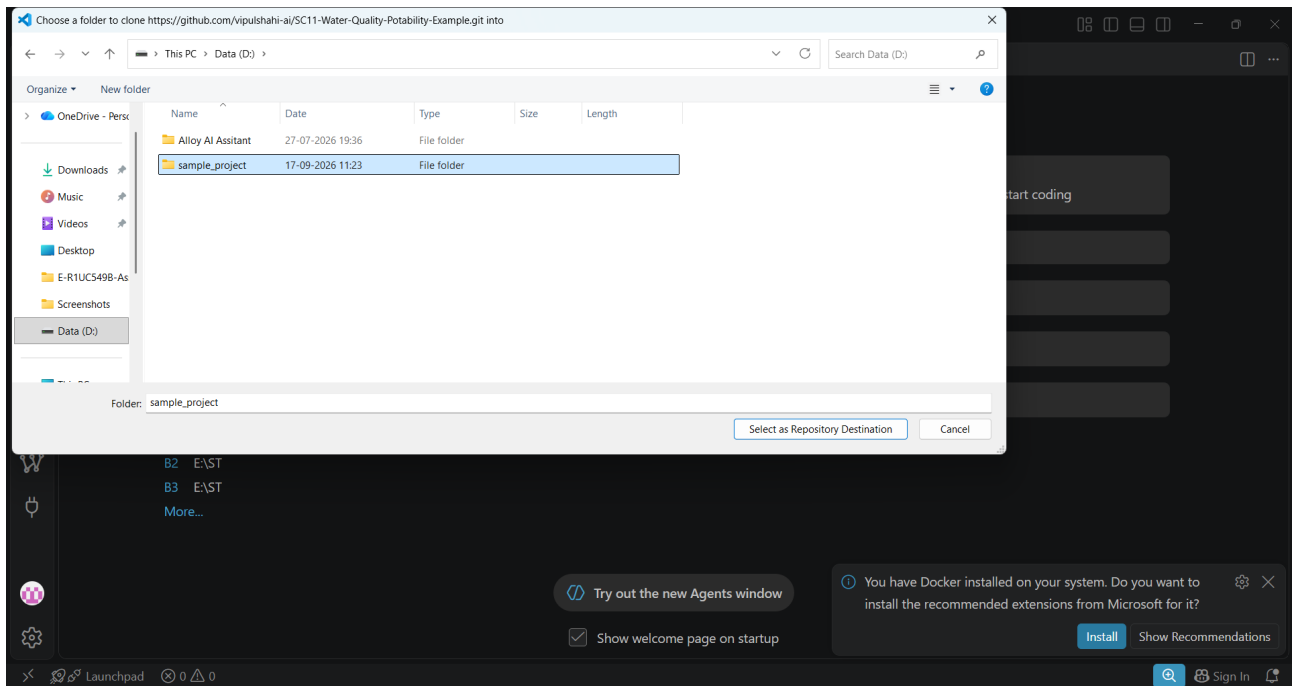
Use the folder picker and click Select Repository Location.

EXPECTED RESULT

VS Code will create one folder with your repository name inside the selected location.

Action 15 - Trust and open the folder

Opening the cloned folder gives VS Code access to the project's files, Python tools, and Git controls.



DO THIS

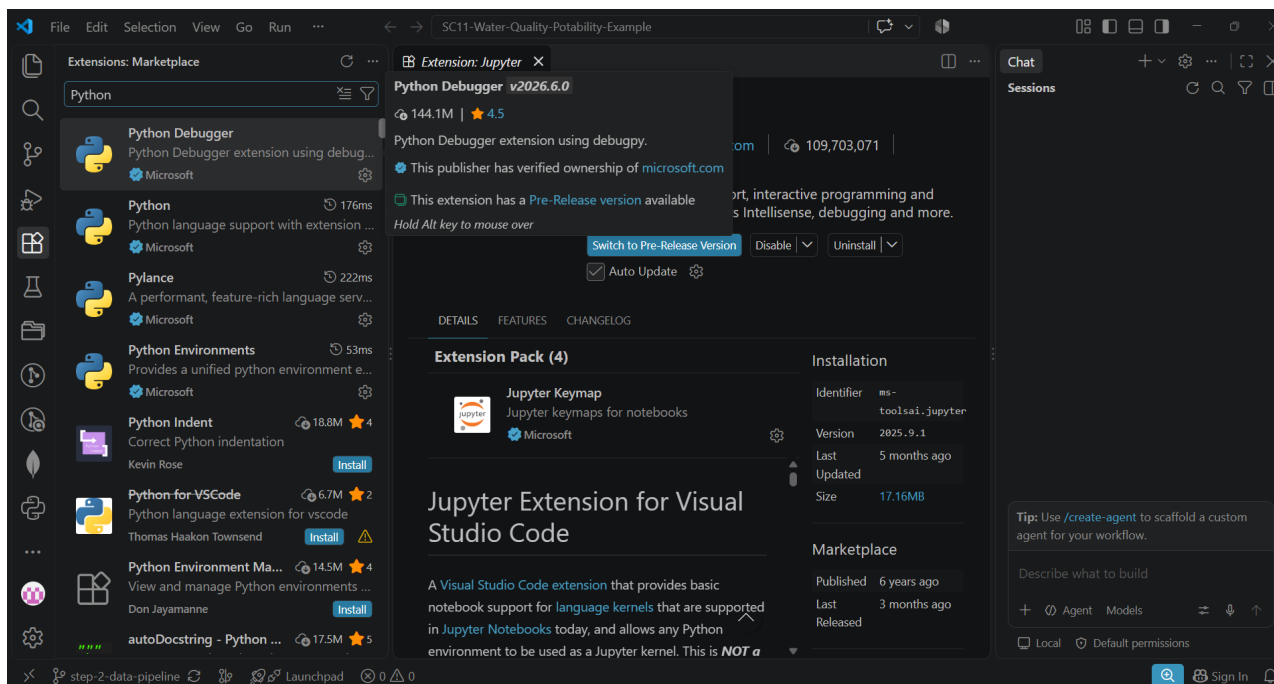
When VS Code asks whether you trust the folder, choose Yes only for your own cloned project.

EXPECTED RESULT

Explorer should show README, data, docs, src, and other project folders.

Action 16 - Install the Python and Jupyter extensions

Extensions add Python editing, notebook support, run buttons, and kernel selection to VS Code. They are required before running an .ipynb notebook.



DO THIS

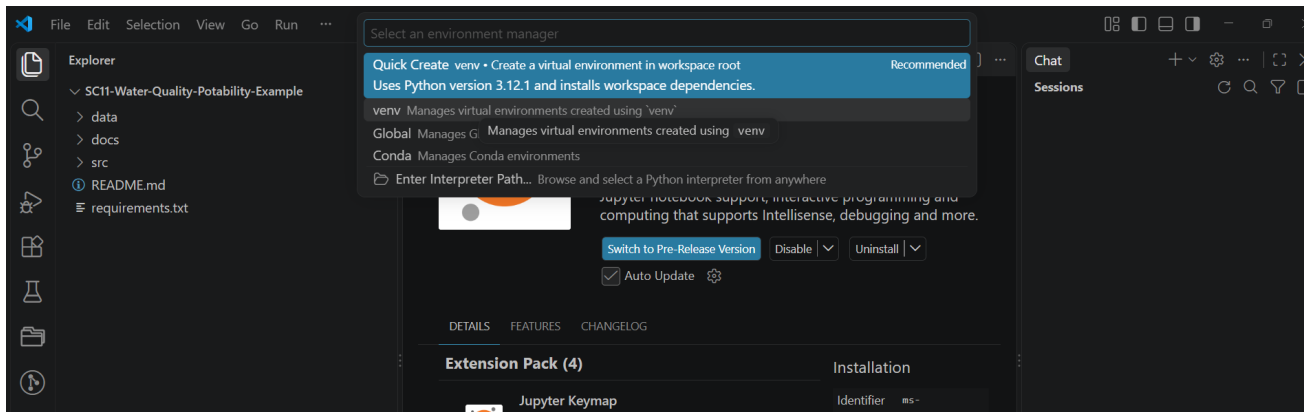
Open Extensions. Search Python and install the Microsoft extension. Then search Jupyter and install the Microsoft extension.

EXPECTED RESULT

Both extensions should display Installed or enabled status.

Action 17 - Create or select a Python environment

A Python environment keeps project libraries such as pandas separate from other projects. It helps every member run the same code without changing their whole computer.



DO THIS

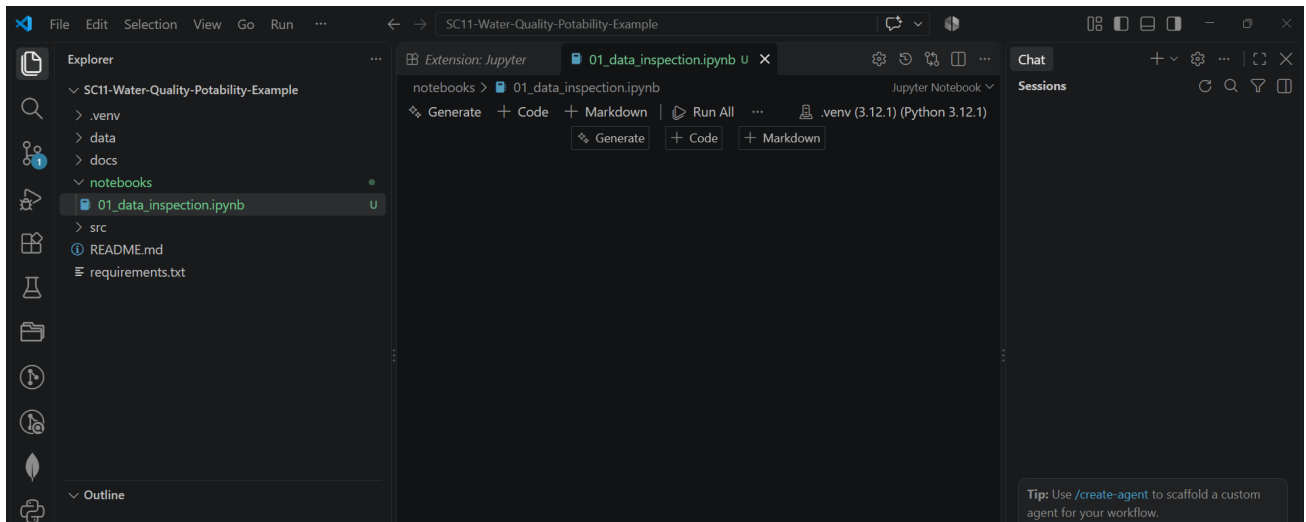
Press Ctrl+Shift+P, choose Python: Create Environment, select Venv, and choose the available Python installation.

EXPECTED RESULT

VS Code should show a .venv interpreter in the status bar or kernel list.

Action 18 - Create the inspection notebook

A notebook combines explanations, code, and visible output. It is evidence that the analysis was actually executed.



DO THIS

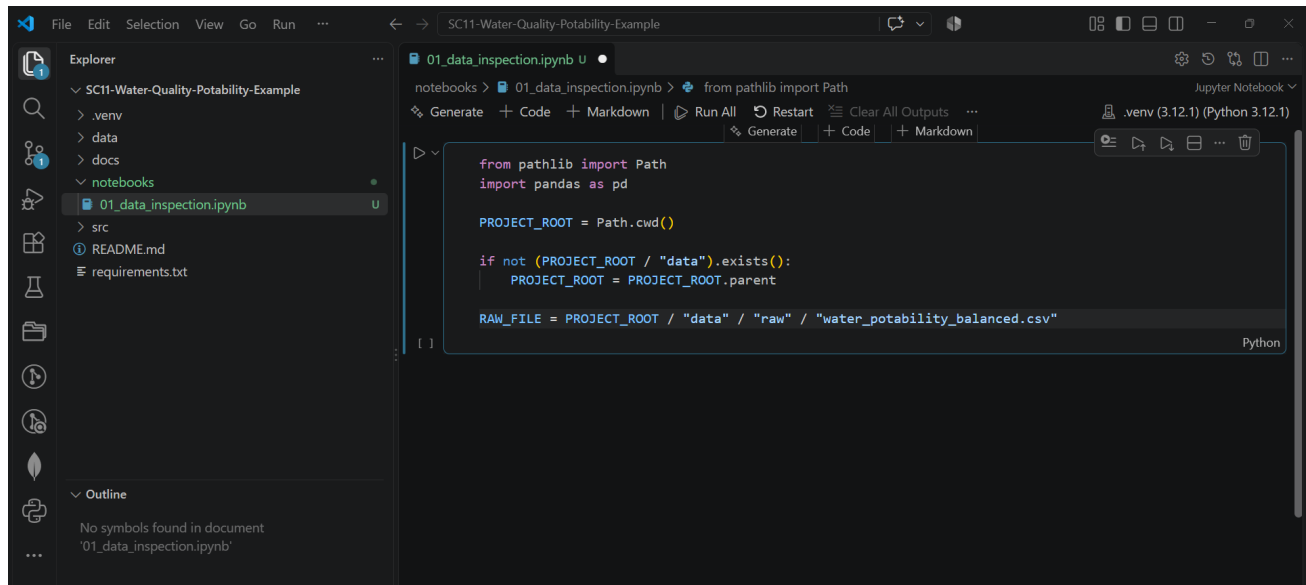
Create notebooks/01_data_inspection.ipynb. Select the .venv kernel at the upper-right of the notebook.

EXPECTED RESULT

The file tab must end in .ipynb and a selected Python kernel must be visible.

Action 19 - Add the data-inspection code

This code loads the raw CSV and checks shape, field names, missing values, duplicate rows, class balance, and a small preview. It does not clean or change data yet.



```
from pathlib import Path
import pandas as pd

PROJECT_ROOT = Path.cwd()

if not (PROJECT_ROOT / "data").exists():
    PROJECT_ROOT = PROJECT_ROOT.parent

RAW_FILE = PROJECT_ROOT / "data" / "raw" / "water_potability_balanced.csv"
```

DO THIS

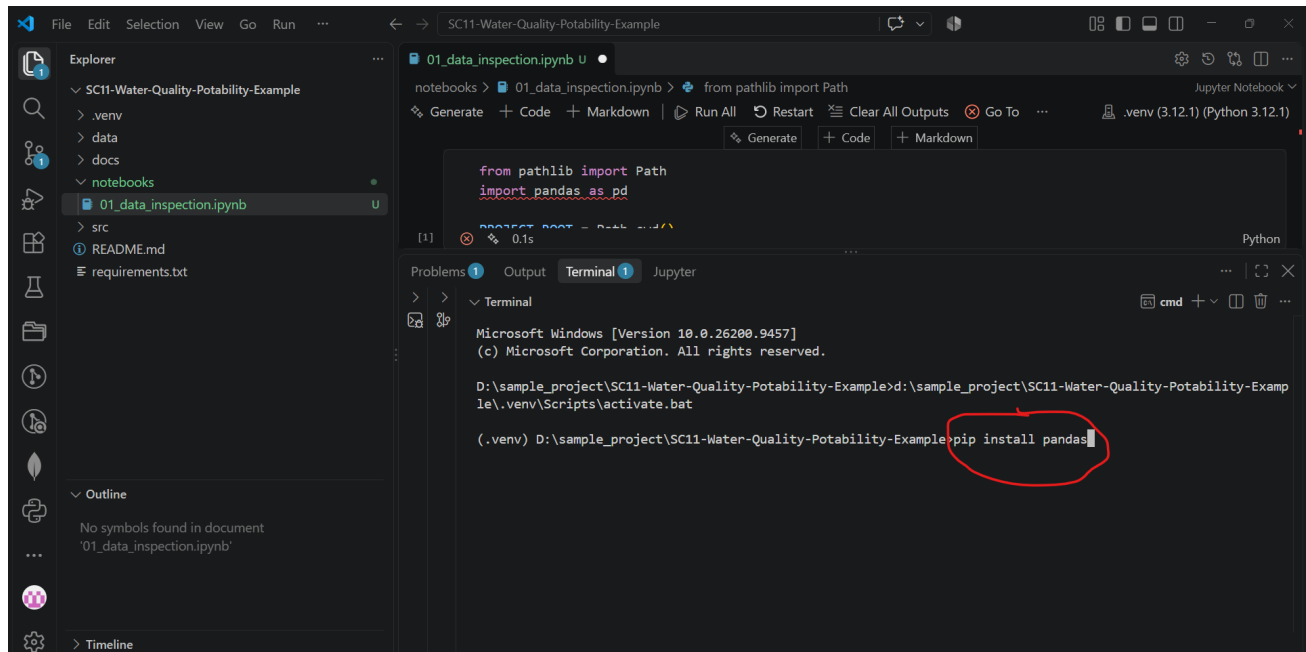
Paste the agreed inspection code into a notebook cell. Use the project-relative raw-data path, not a personal Downloads path.

EXPECTED RESULT

The code cell should contain the CSV path and inspection print statements.

Action 20 - Run the notebook cell

Running a cell means Python executes the code and shows its real result beneath the cell. A notebook without output is incomplete evidence.



DO THIS

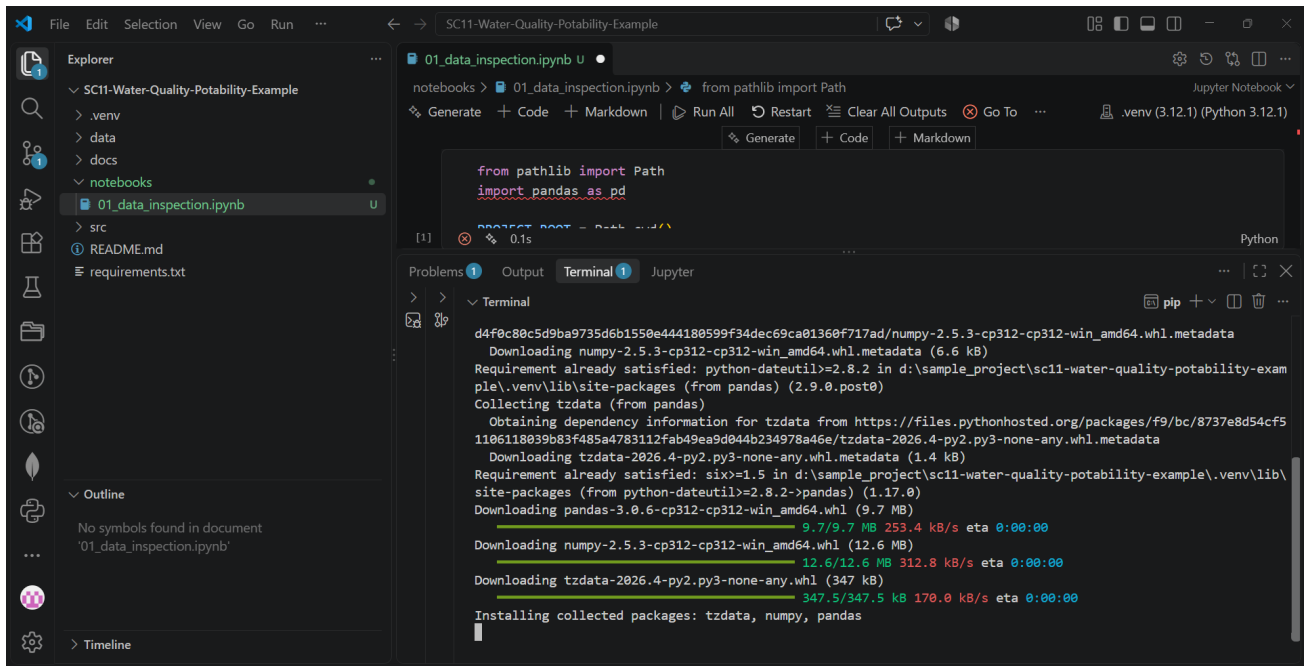
Click the play triangle beside the cell or use Run Cell. Wait until execution finishes.

EXPECTED RESULT

The output must show the dataset shape and no red error message.

Action 21 - Read the quality checks

Missing-value counts tell you which columns need cleaning. Duplicate count tells you whether repeated samples exist. Class balance shows the number of potable and non-potable records.



DO THIS

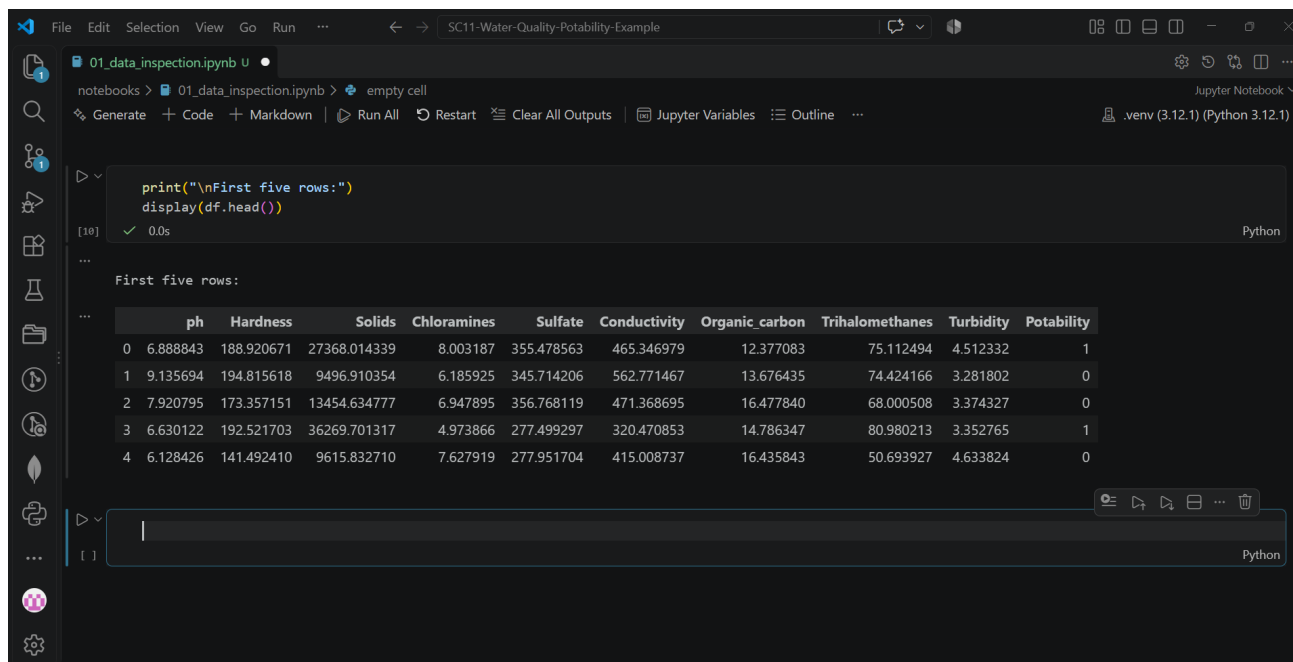
Read the output rather than only running it. Record these numbers later in results/data_quality_summary.md.

EXPECTED RESULT

The output should list every column, its missing count, duplicates, and Potability counts.

Action 22 - Confirm the data preview

The preview helps catch wrong separators, unexpected column names, or accidental loading of the wrong file before you begin cleaning.



The screenshot shows a Jupyter Notebook interface with a dark theme. The notebook is titled "01_data_inspection.ipynb". The code cell contains the following Python code:

```
print("\nFirst five rows:")
display(df.head())
```

The output of the code is displayed below the cell, showing the first five rows of a dataset. The output is a table with 10 columns: ph, Hardness, Solids, Chloramines, Sulfate, Conductivity, Organic_carbon, Trihalomethanes, Turbidity, and Potability. The rows are indexed from 0 to 4.

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity	Potability
0	6.888843	188.920671	27368.014339	8.003187	355.478563	465.346979	12.377083	75.112494	4.512332	1
1	9.135694	194.815618	9496.910354	6.185925	345.714206	562.771467	13.676435	74.424166	3.281802	0
2	7.920795	173.357151	13454.634777	6.947895	356.768119	471.368695	16.477840	68.000508	3.374327	0
3	6.630122	192.521703	36269.701317	4.973866	277.499297	320.470853	14.786347	80.980213	3.352765	1
4	6.128426	141.492410	9615.832710	7.627919	277.951704	415.008737	16.435843	50.693927	4.633824	0

DO THIS

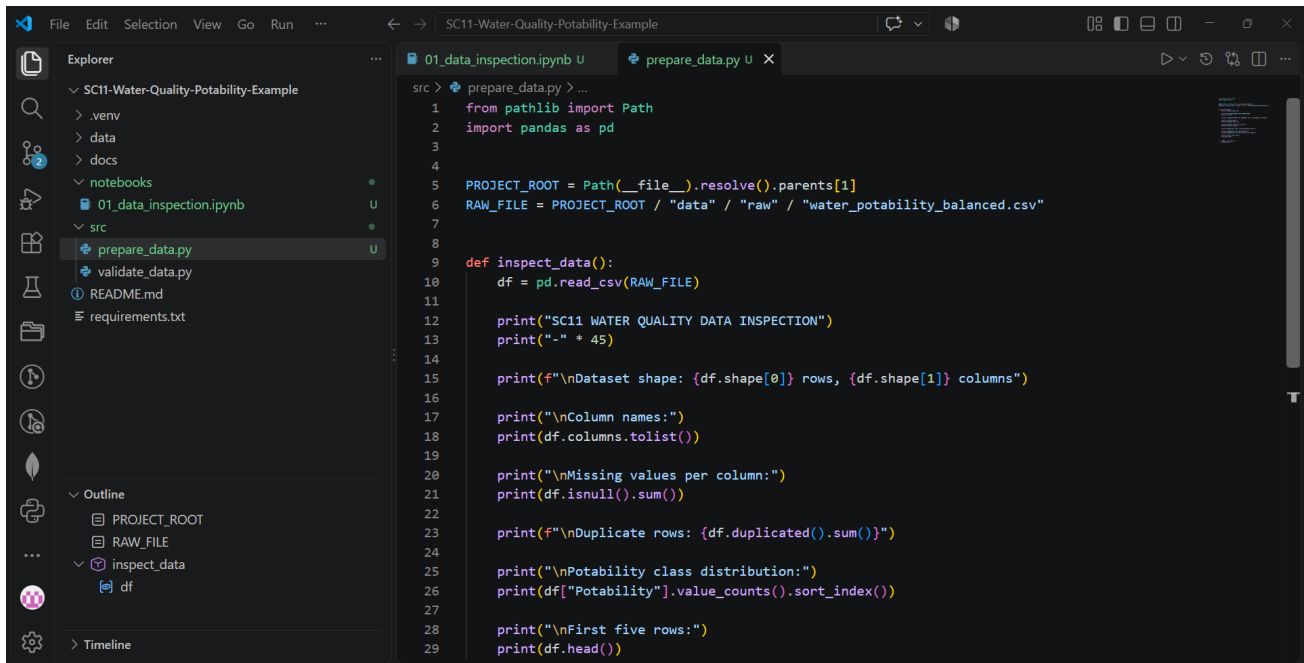
Use `display(df.head())` or `df.head()` as the final notebook output.

EXPECTED RESULT

You should see five rows with all project fields and Potability.

Action 23 - Create the reusable Python file

The .py file is the reusable program version of the notebook work. The notebook proves the output; the Python file makes the same workflow easy to rerun later.



```
src > prepare_data.py > ...
1  from pathlib import Path
2  import pandas as pd
3
4
5  PROJECT_ROOT = Path(__file__).resolve().parents[1]
6  RAW_FILE = PROJECT_ROOT / "data" / "raw" / "water_potability_balanced.csv"
7
8
9  def inspect_data():
10     df = pd.read_csv(RAW_FILE)
11
12     print("SC11 WATER QUALITY DATA INSPECTION")
13     print("-" * 45)
14
15     print(f"\nDataset shape: {df.shape[0]} rows, {df.shape[1]} columns")
16
17     print("\nColumn names:")
18     print(df.columns.tolist())
19
20     print("\nMissing values per column:")
21     print(df.isnull().sum())
22
23     print(f"\nDuplicate rows: {df.duplicated().sum()}")
24
25     print("\nPotability class distribution:")
26     print(df["Potability"].value_counts().sort_index())
27
28     print("\nFirst five rows:")
29     print(df.head())
```

DO THIS

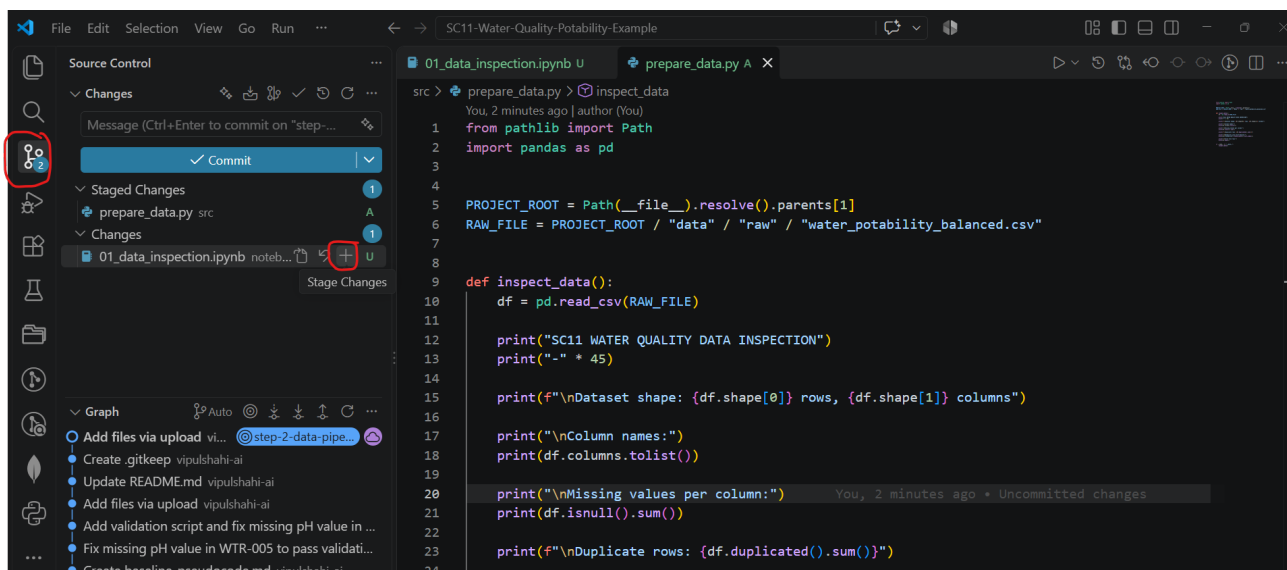
Create src/prepare_data.py and copy the inspection logic into a function. Keep the raw filename identical to the uploaded CSV name.

EXPECTED RESULT

The Python file should be visible under src and should not contain personal computer paths.

Action 24 - Save before Git actions

Saving writes the latest notebook output and code to the local project folder. Git can only stage saved files.



DO THIS

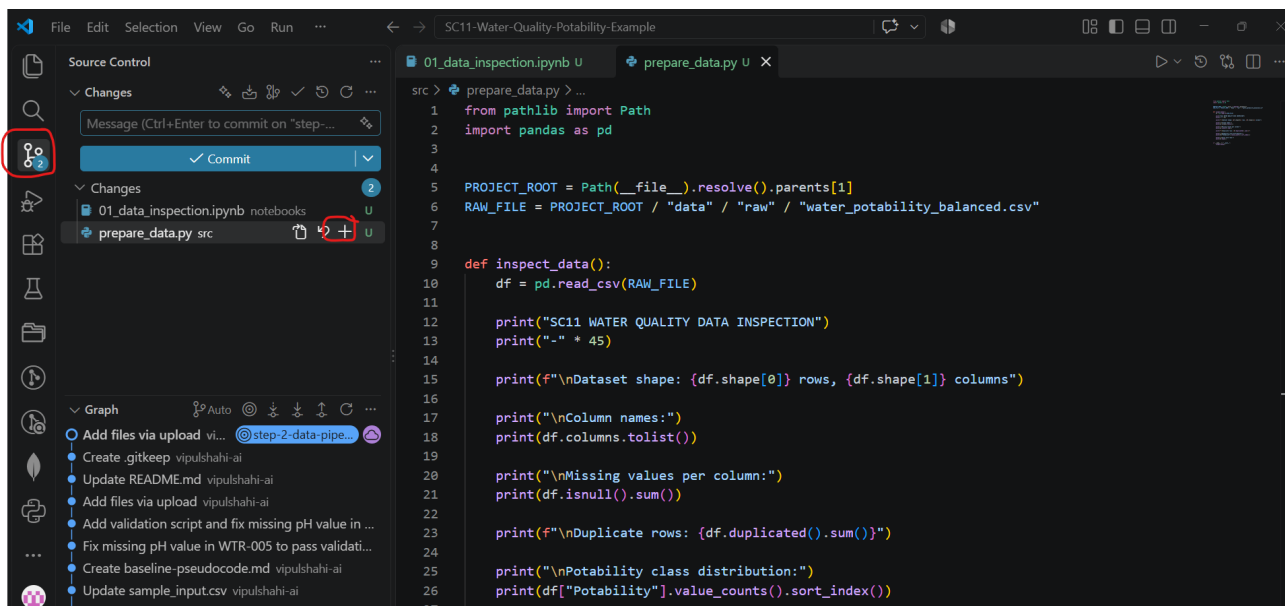
Use File > Save All, then check that unsaved-dot indicators disappear from notebook and Python tabs.

EXPECTED RESULT

No unsaved indicator should remain beside the files you intend to commit.

Action 25 - Open Source Control

Source Control shows the local differences between your working copy and the last saved Git checkpoint. It helps you review exactly what will be committed.



DO THIS

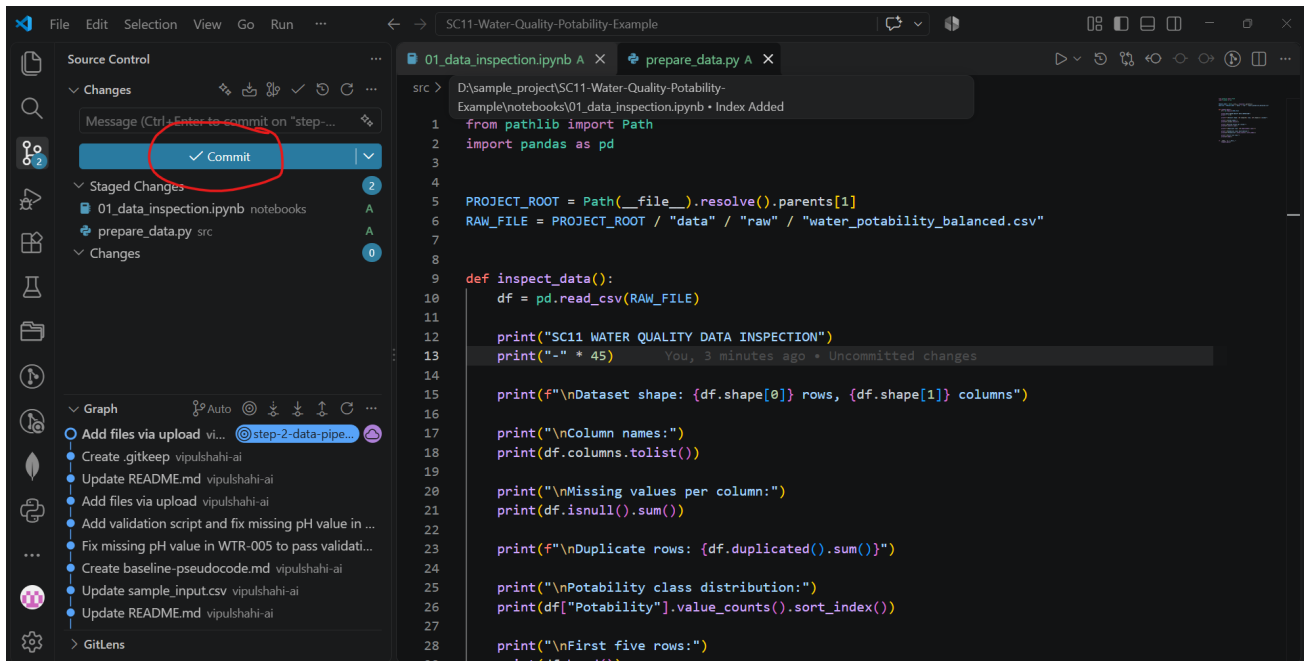
Click the Source Control icon on the left. Review the Changes list before staging anything.

EXPECTED RESULT

Only your intended Step 2 files should appear in Changes.

Action 26 - Stage the correct files

Staging means selecting exactly which saved changes will go into the next commit. It is like placing finished pages into a submission envelope before sealing it.



DO THIS

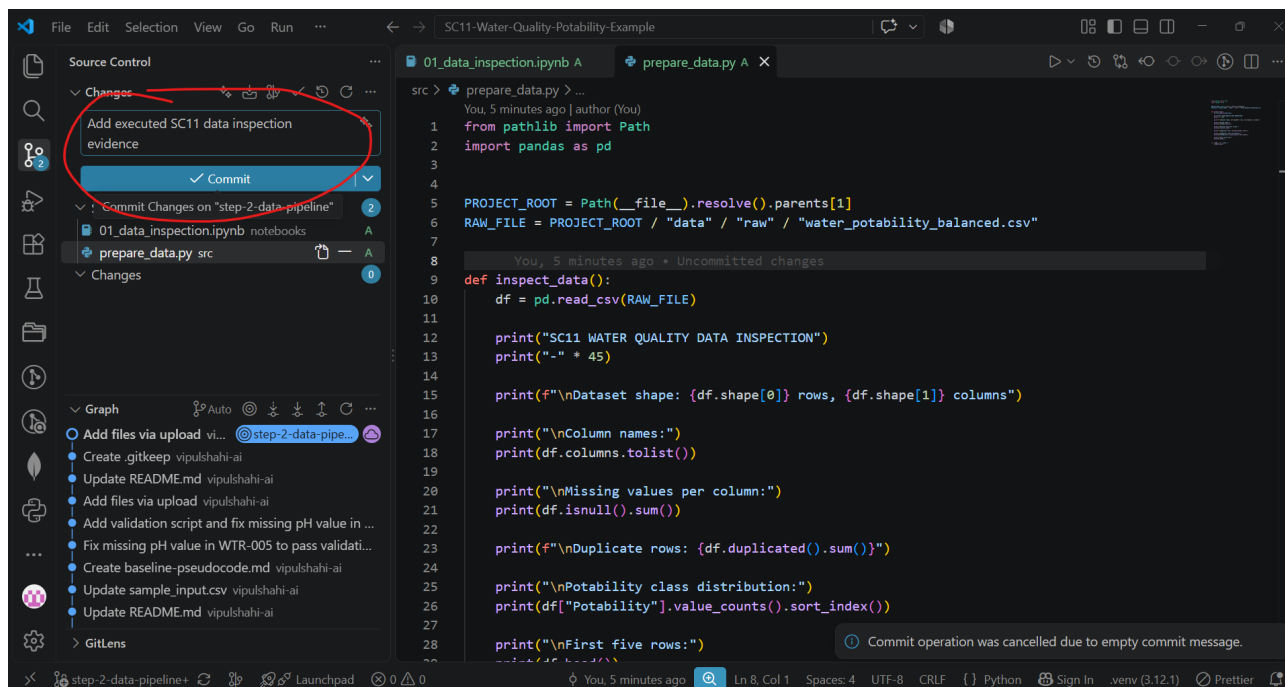
Click + beside the executed notebook, prepare_data.py, and evidence screenshot. Do not stage unrelated temporary files.

EXPECTED RESULT

The files move from Changes to Staged Changes.

Action 27 - Commit the evidence

Committing creates a permanent local Git checkpoint with a meaningful explanation. Weak messages such as update or final do not explain the work.



DO THIS

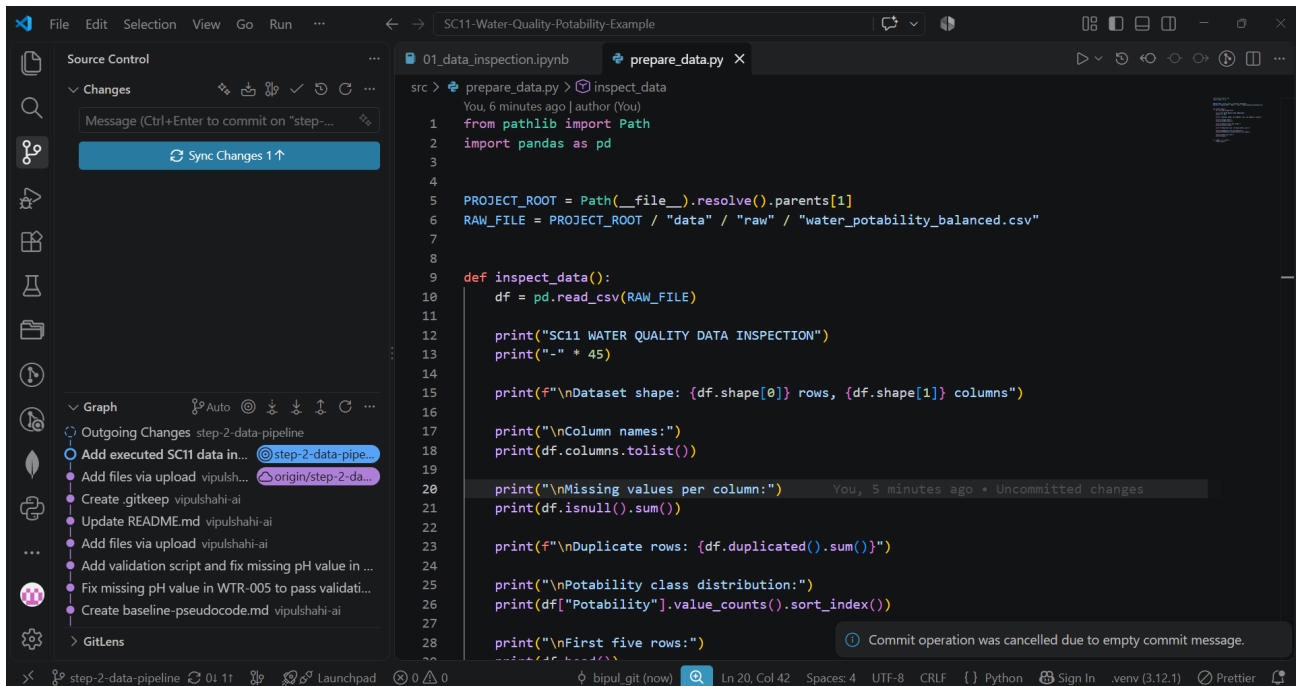
Write Add executed SC11 data inspection evidence, then click Commit.

EXPECTED RESULT

The staged list should clear after a successful commit.

Action 28 - Sync changes to GitHub

Sync uploads your local commit to GitHub and also receives teammates' new commits. Until sync is complete, classmates cannot see your local work in the shared repository.



DO THIS

Click Sync Changes. Wait for the progress indicator to finish, then refresh the repository page in GitHub.

EXPECTED RESULT

GitHub should show the latest commit and render notebook outputs beneath the cells.

Part B - Complete the Step 2 pipeline

Part A prepared the working environment and inspected the raw source. Part B completes the repeatable pipeline: clean the data, validate it, split and save it, write the reusable program and report, then commit and sync the complete evidence.

Important sequence

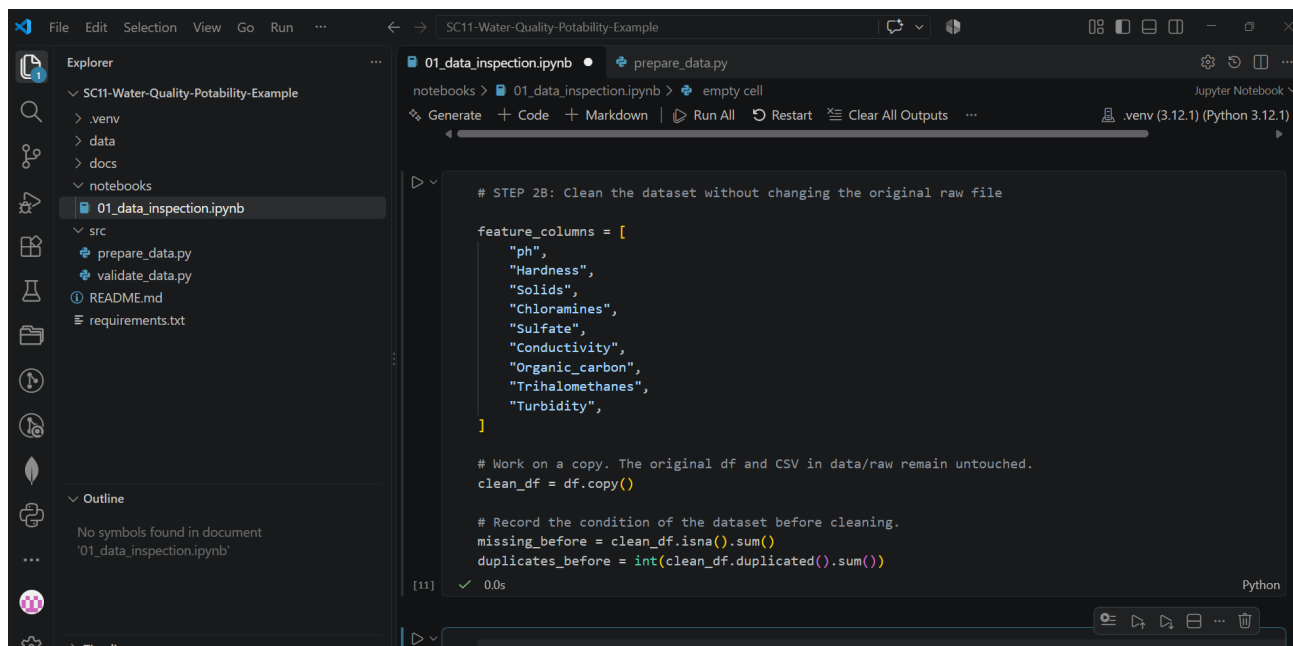
First make the process work visibly in the notebook. Then place the same logic in `src/prepare_data.py`. Finally document, stage, commit, and sync it. Do not skip the notebook outputs or the written report.

For all capstone teams

ANN teams clean and split cited records. Fuzzy and GA teams use the same evidence pattern for documented scenarios, validation rules, and reproducible generators. Use your own project fields and units.

Action 29 - Begin cleaning with a safe copy

Cleaning must never edit the raw CSV. A separate in-memory copy lets you improve data while preserving the original evidence in data/raw/.



The screenshot shows a Jupyter Notebook titled '01_data_inspection.ipynb' in a VS Code environment. The notebook is open to a cell with the following code:

```
# STEP 2B: Clean the dataset without changing the original raw file

feature_columns = [
    "ph",
    "Hardness",
    "Solids",
    "Chloramines",
    "Sulfate",
    "Conductivity",
    "Organic_carbon",
    "Trihalomethanes",
    "Turbidity",
]

# Work on a copy. The original df and CSV in data/raw remain untouched.
clean_df = df.copy()

# Record the condition of the dataset before cleaning.
missing_before = clean_df.isna().sum()
duplicates_before = int(clean_df.duplicated().sum())
```

The output of the cell is shown as [11]: 0.0s. The left sidebar shows the file explorer with the project structure, including a 'src' folder containing 'prepare_data.py' and 'validate_data.py'.

DO THIS

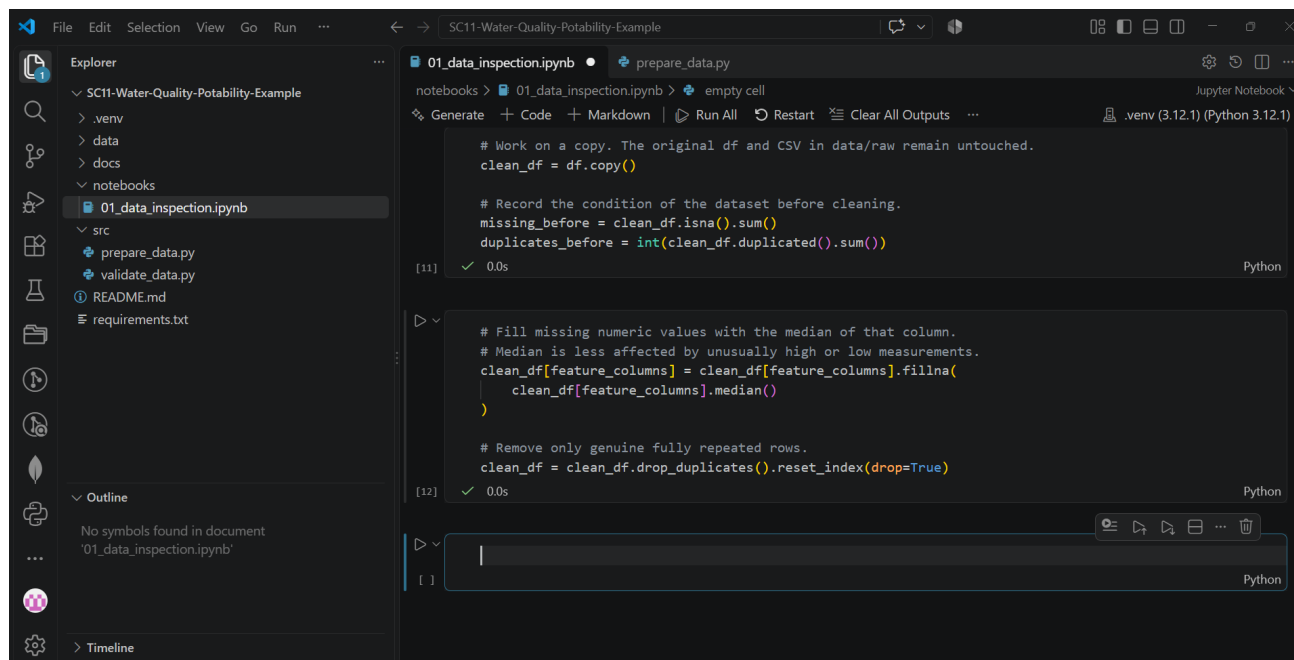
In the notebook, define the feature-column list and write `clean_df = df.copy()`. Make sure the notebook still uses your own project fields.

EXPECTED RESULT

The feature list and `clean_df` copy are visible; the original `df` has not been changed.

Action 30 - Record the condition before cleaning

Before changing data, count what is wrong. These before-values make your cleaning honest and make the final quality report meaningful.



The screenshot shows a Jupyter Notebook titled '01_data_inspection.ipynb' within a project named 'SC11-Water-Quality-Potability-Example'. The notebook is open in a dark-themed IDE. The left sidebar shows the file explorer with the project structure. The main area displays two code cells. The first cell, labeled [11], contains Python code to create a copy of the dataset and record the number of missing and duplicate values before cleaning. The second cell, labeled [12], contains code to fill missing numeric values with the median of each column and remove fully repeated rows. The output of the first cell is shown as a green checkmark and '0.0s'.

```
# Work on a copy. The original df and CSV in data/raw remain untouched.
clean_df = df.copy()

# Record the condition of the dataset before cleaning.
missing_before = clean_df.isna().sum()
duplicates_before = int(clean_df.duplicated().sum())

[11] ✓ 0.0s Python
```

```
# Fill missing numeric values with the median of that column.
# Median is less affected by unusually high or low measurements.
clean_df[feature_columns] = clean_df[feature_columns].fillna(
    clean_df[feature_columns].median()
)

# Remove only genuine fully repeated rows.
clean_df = clean_df.drop_duplicates().reset_index(drop=True)

[12] ✓ 0.0s Python
```

DO THIS

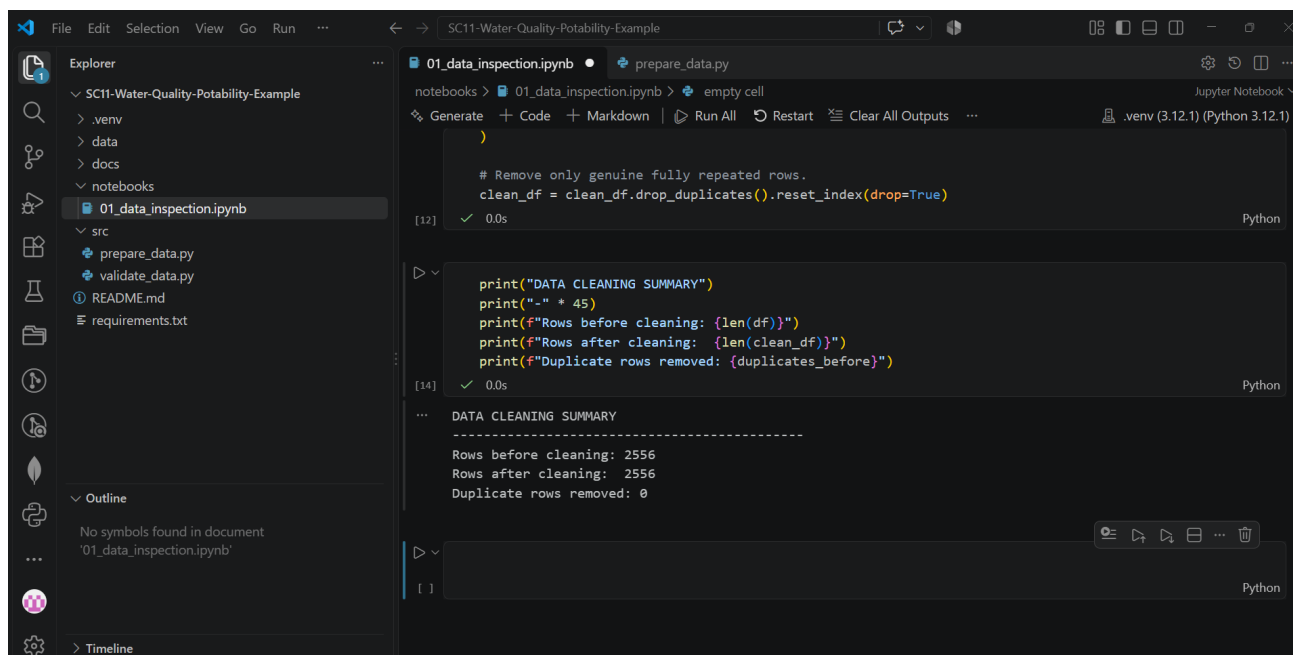
Count missing values and duplicate rows before any correction. Keep the variables `missing_before` and `duplicates_before` for the summary.

EXPECTED RESULT

The notebook has saved the original missing-value and duplicate counts.

Action 31 - Fill missing values and remove genuine duplicates

Median imputation fills a missing numeric value with the middle typical value of that column. It is less affected by extreme readings than an average. Duplicate removal deletes only fully identical rows.



The screenshot shows a Jupyter Notebook titled '01_data_inspection.ipynb' in a dark-themed IDE. The left sidebar shows a file explorer with a project named 'SC11-Water-Quality-Potability-Example' containing files like '.venv', 'data', 'docs', 'notebooks', 'src', 'prepare_data.py', 'validate_data.py', 'README.md', and 'requirements.txt'. The main area displays two code cells. The first cell, labeled [12], contains a Python code snippet:

```
)  
  
# Remove only genuine fully repeated rows.  
clean_df = clean_df.drop_duplicates().reset_index(drop=True)
```

 The second cell, labeled [14], contains a Python code snippet:

```
print("DATA CLEANING SUMMARY")  
print("-" * 45)  
print(f"Rows before cleaning: {len(df)}")  
print(f"Rows after cleaning: {len(clean_df)}")  
print(f"Duplicate rows removed: {duplicates_before}")
```

 The output of the second cell is displayed below the code:

```
DATA CLEANING SUMMARY  
-----  
Rows before cleaning: 2556  
Rows after cleaning: 2556  
Duplicate rows removed: 0
```

DO THIS

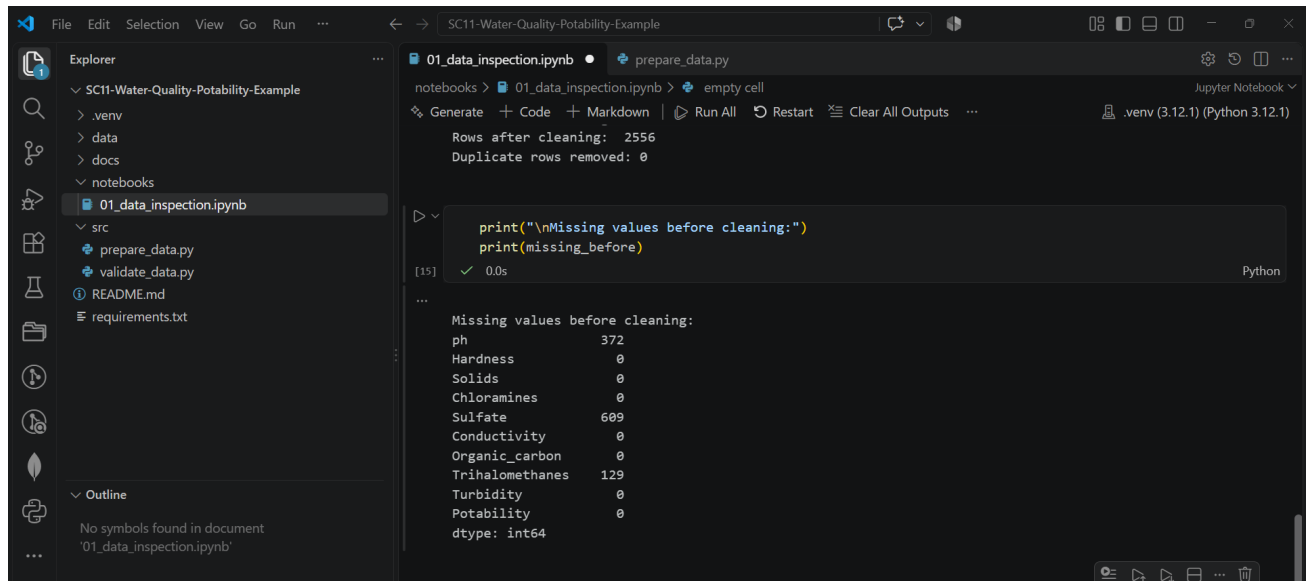
Fill missing feature values with each column's median, then run `drop_duplicates()` and reset the row index. Do not invent or overwrite the target label.

EXPECTED RESULT

`clean_df` contains repaired feature values and contains only unique rows.

Action 32 - Show the cleaning summary

A visible summary proves what was changed. It also lets a reviewer compare row counts before and after cleaning.



The screenshot shows a Jupyter Notebook interface with a dark theme. The left sidebar contains an Explorer view showing the project structure: 'SC11-Water-Quality-Potability-Example' with subfolders '.env', 'data', 'docs', 'notebooks', and 'src'. The 'notebooks' folder is expanded, showing '01_data_inspection.ipynb'. The main area displays the notebook content. The top cell shows the output of a previous action: 'Rows after cleaning: 2556' and 'Duplicate rows removed: 0'. The current cell contains a Python code snippet:

```
print("\nMissing values before cleaning:")
print(missing_before)
```

. The output of this cell is a text summary of missing values:

```
Missing values before cleaning:
ph                372
Hardness           0
Solids             0
Chloramines        0
Sulfate            609
Conductivity       0
Organic_carbon     0
Trihalomethanes    129
Turbidity          0
Potability         0
dtype: int64
```

DO THIS

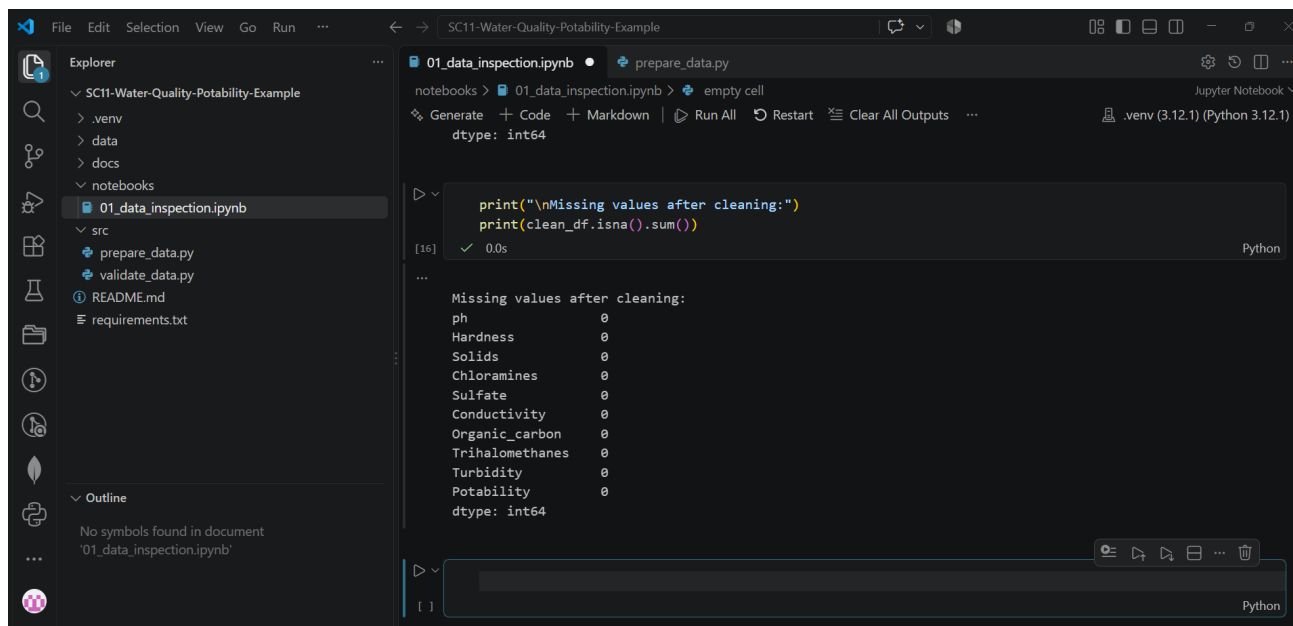
Print rows before cleaning, rows after cleaning, and duplicates removed. Run the cell and keep the output visible.

EXPECTED RESULT

A DATA CLEANING SUMMARY appears with real numbers and no error message.

Action 33 - Confirm missing values are now zero

Cleaning is not complete merely because code ran. Check every required column again to confirm that no missing values remain.



```
notebooks > 01_data_inspection.ipynb > empty cell
Generate + Code + Markdown | Run All Restart Clear All Outputs ... .venv (3.12.1) (Python 3.12.1)

dtype: int64

print("\nMissing values after cleaning:")
print(clean_df.isna().sum())

[16] ✓ 0.0s Python

...

Missing values after cleaning:
ph                0
Hardness          0
Solids            0
Chloramines       0
Sulfate           0
Conductivity      0
Organic_carbon    0
Trihalomethanes   0
Turbidity         0
Potability        0
dtype: int64
```

DO THIS

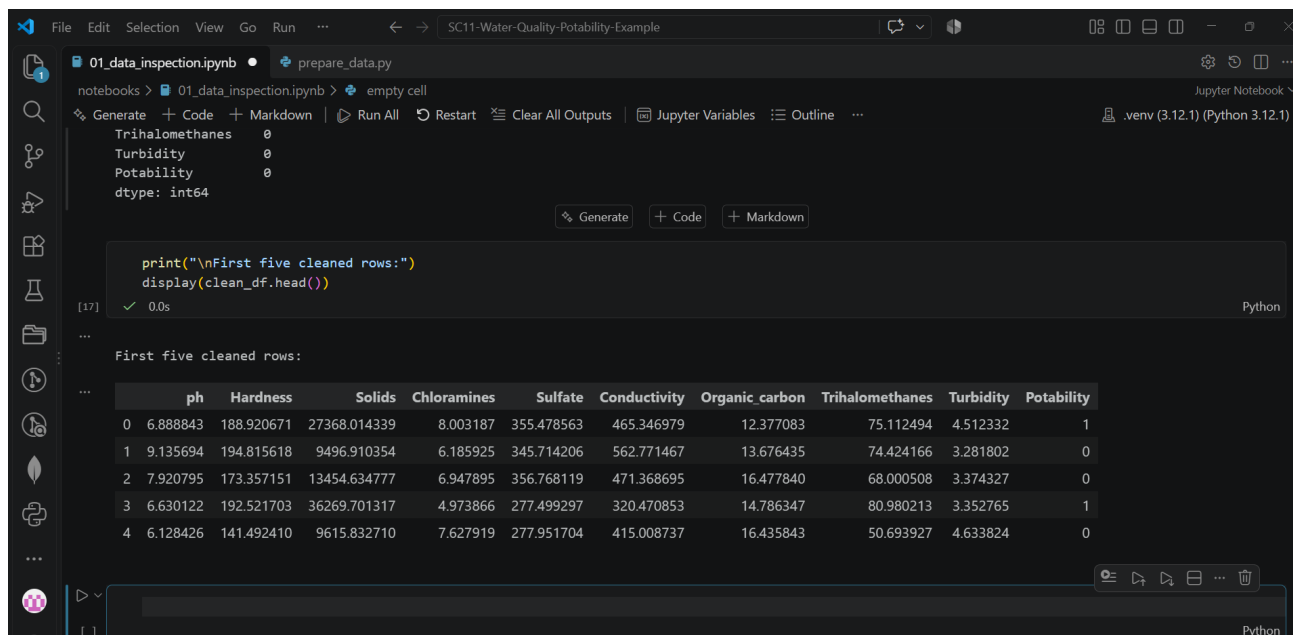
Print `clean_df.isna().sum()` after the median-imputation step and read each number.

EXPECTED RESULT

Every required project field shows 0 missing values.

Action 34 - Preview the cleaned records

A cleaned preview catches accidental column changes, wrong separators, and implausible output before the data is split for later work.



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The notebook is titled '01_data_inspection.ipynb'. The code cell contains the following Python code:

```
Trihalomethanes    0
Turbidity           0
Potability          0
dtype: int64
```

Below the code cell, the output is displayed as a table titled 'First five cleaned rows:'. The table has 11 columns: ph, Hardness, Solids, Chloramines, Sulfate, Conductivity, Organic_carbon, Trihalomethanes, Turbidity, and Potability. The data is as follows:

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity	Potability
0	6.888843	188.920671	27368.014339	8.003187	355.478563	465.346979	12.377083	75.112494	4.512332	1
1	9.135694	194.815618	9496.910354	6.185925	345.714206	562.771467	13.676435	74.424166	3.281802	0
2	7.920795	173.357151	13454.634777	6.947895	356.768119	471.368695	16.477840	68.000508	3.374327	0
3	6.630122	192.521703	36269.701317	4.973866	277.499297	320.470853	14.786347	80.980213	3.352765	1
4	6.128426	141.492410	9615.832710	7.627919	277.951704	415.008737	16.435843	50.693927	4.633824	0

DO THIS

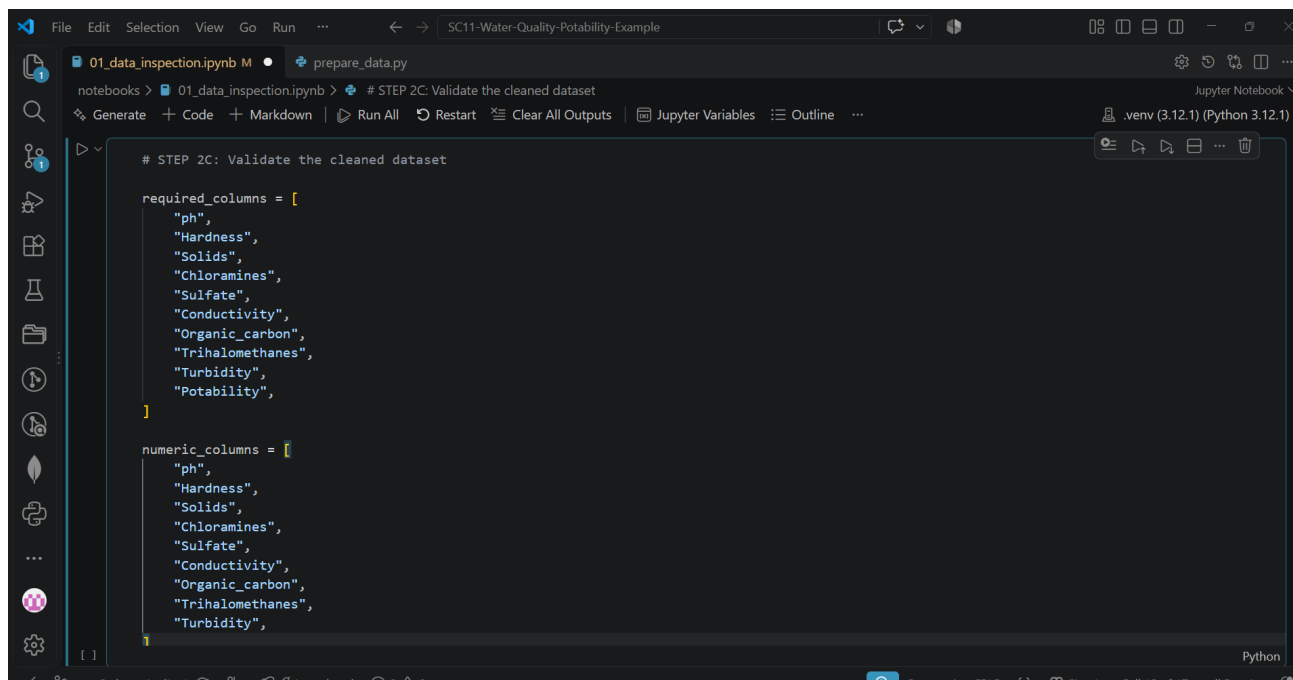
Run `display(clean_df.head())` and check the field names, numeric values, and target column.

EXPECTED RESULT

Five cleaned rows are visible with the expected fields.

Action 35 - Define the validation contract

Validation is a list of rules that data must obey. It prevents a program from quietly training or deciding on incomplete, invalid, or incorrectly labelled input.



The screenshot shows a Jupyter Notebook interface with a dark theme. The notebook is titled '01_data_inspection.ipynb' and is currently on a cell labeled '# STEP 2C: Validate the cleaned dataset'. The code in the cell defines two lists: 'required_columns' and 'numeric_columns'. Both lists contain the same set of column names: 'ph', 'Hardness', 'Solids', 'Chloramines', 'Sulfate', 'Conductivity', 'Organic_carbon', 'Trihalomethanes', 'Turbidity', and 'Potability'. The notebook interface includes a sidebar with icons for file explorer, search, and other tools. The top bar shows the file name and various action buttons like 'Run All', 'Restart', and 'Clear All Outputs'. The bottom status bar indicates the environment is '.venv (3.12.1) (Python 3.12.1)'.

```
# STEP 2C: Validate the cleaned dataset

required_columns = [
    "ph",
    "Hardness",
    "Solids",
    "Chloramines",
    "Sulfate",
    "Conductivity",
    "Organic_carbon",
    "Trihalomethanes",
    "Turbidity",
    "Potability",
]

numeric_columns = [
    "ph",
    "Hardness",
    "Solids",
    "Chloramines",
    "Sulfate",
    "Conductivity",
    "Organic_carbon",
    "Trihalomethanes",
    "Turbidity",
]
```

DO THIS

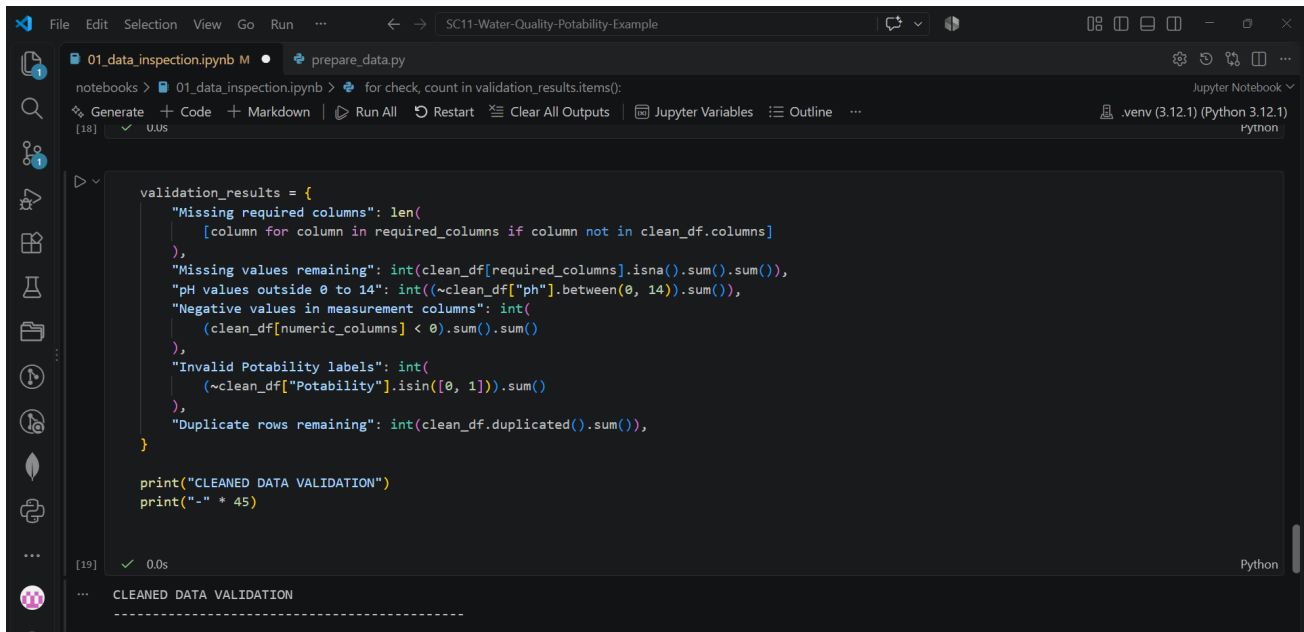
List the required columns and numeric columns for your own project. For SC11, these are water measurements plus Potability.

EXPECTED RESULT

The notebook clearly identifies the fields that must exist and be numeric.

Action 36 - Calculate validation checks

A validation-result dictionary converts rules into clear counts. Zero means the rule passed; a non-zero count means stop and correct the data.



```
validation_results = {
    "Missing required columns": len(
        [column for column in required_columns if column not in clean_df.columns]
    ),
    "Missing values remaining": int(clean_df[required_columns].isna().sum().sum()),
    "pH values outside 0 to 14": int((~clean_df["ph"].between(0, 14)).sum()),
    "Negative values in measurement columns": int(
        (clean_df[numeric_columns] < 0).sum().sum()
    ),
    "Invalid Potability labels": int(
        (~clean_df["Potability"].isin([0, 1])).sum()
    ),
    "Duplicate rows remaining": int(clean_df.duplicated().sum()),
}

print("CLEANED DATA VALIDATION")
print("-" * 45)
```

CLEANED DATA VALIDATION

DO THIS

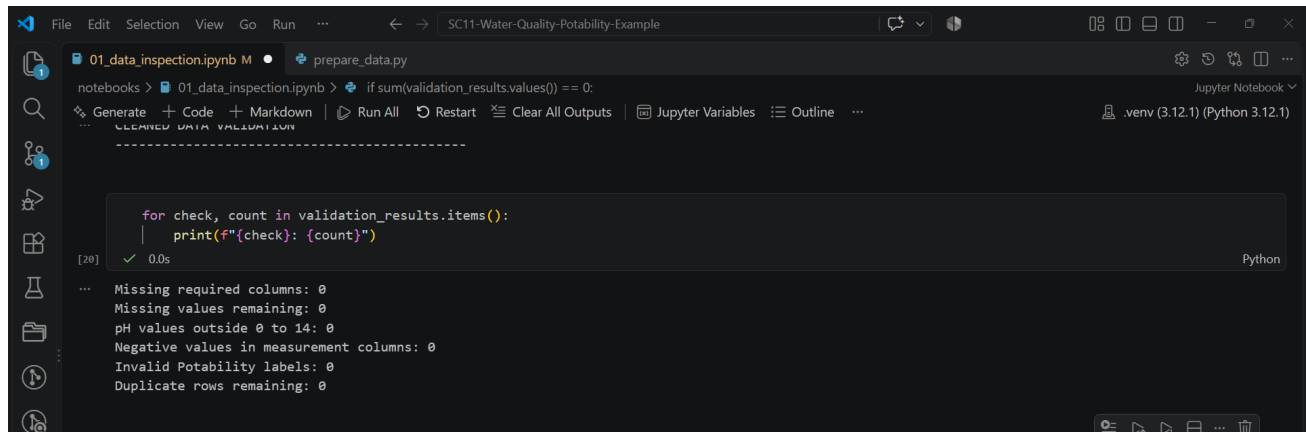
Check for missing required columns, remaining missing values, pH outside 0-14, negative measurements, invalid Potability labels, and duplicate rows.

EXPECTED RESULT

The CLEANED DATA VALIDATION heading and its checks appear below the cell.

Action 37 - Read every validation count

Students must read results, not only execute code. The counts explain whether the prepared dataset can safely move forward.

A screenshot of a Jupyter Notebook interface. The notebook is titled '01_data_inspection.ipynb' and is part of a project named 'SC11-Water-Quality-Potability-Example'. The code cell shows a loop that iterates over 'validation_results.items()' and prints each check and its count. The output shows all counts are 0, indicating no issues. The checks listed are: Missing required columns, Missing values remaining, pH values outside 0 to 14, Negative values in measurement columns, Invalid Potability labels, and Duplicate rows remaining.

```
for check, count in validation_results.items():
    print(f"{check}: {count}")
```

```
Missing required columns: 0
Missing values remaining: 0
pH values outside 0 to 14: 0
Negative values in measurement columns: 0
Invalid Potability labels: 0
Duplicate rows remaining: 0
```

DO THIS

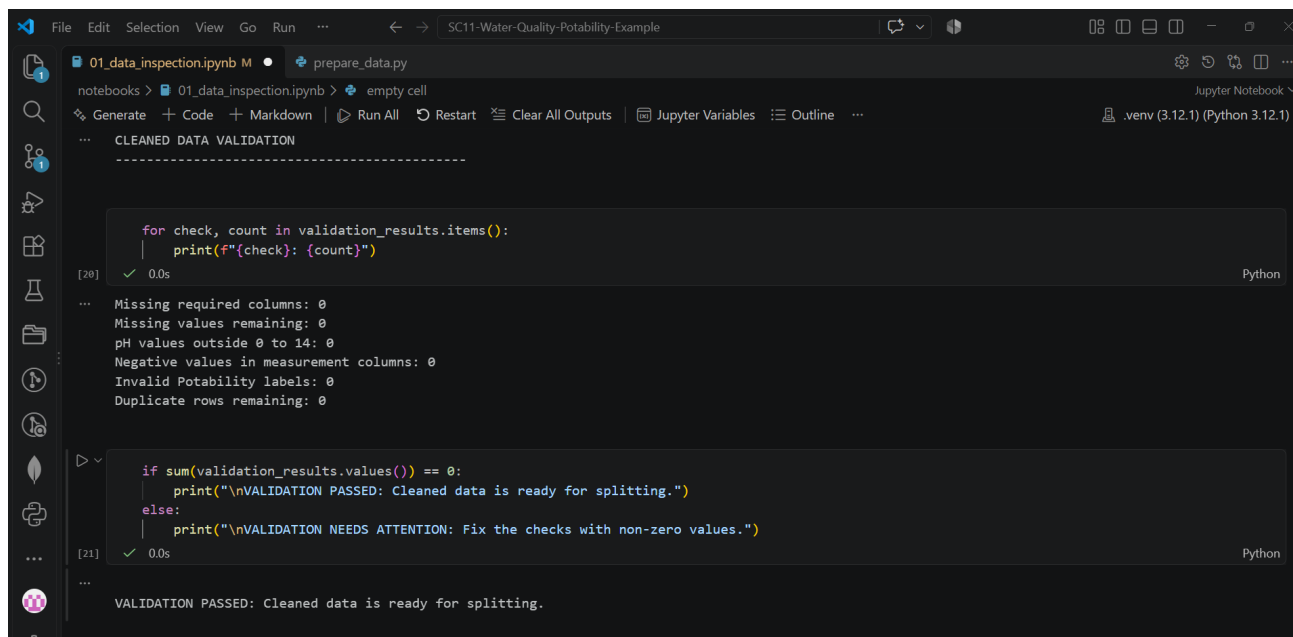
Print every check and examine every number. If any number is above zero, fix the source or cleaning rule before continuing.

EXPECTED RESULT

All applicable checks display 0 for this SC11 example.

Action 38 - Confirm validation passed

A simple pass/fail message makes the notebook easy to assess. It means all the validation counts together are zero.



The screenshot shows a Jupyter Notebook interface with two code cells. The first cell, labeled [20], contains a loop that iterates over validation results and prints each check and its count. The output shows all counts are zero. The second cell, labeled [21], contains an if/else statement that prints a message based on whether the sum of validation counts is zero. The output shows the 'VALIDATION PASSED' message.

```
for check, count in validation_results.items():
    print(f"{check}: {count}")
```

```
Missing required columns: 0
Missing values remaining: 0
pH values outside 0 to 14: 0
Negative values in measurement columns: 0
Invalid Potability labels: 0
Duplicate rows remaining: 0
```

```
if sum(validation_results.values()) == 0:
    print("\nVALIDATION PASSED: Cleaned data is ready for splitting.")
else:
    print("\nVALIDATION NEEDS ATTENTION: Fix the checks with non-zero values.")
```

```
VALIDATION PASSED: Cleaned data is ready for splitting.
```

DO THIS

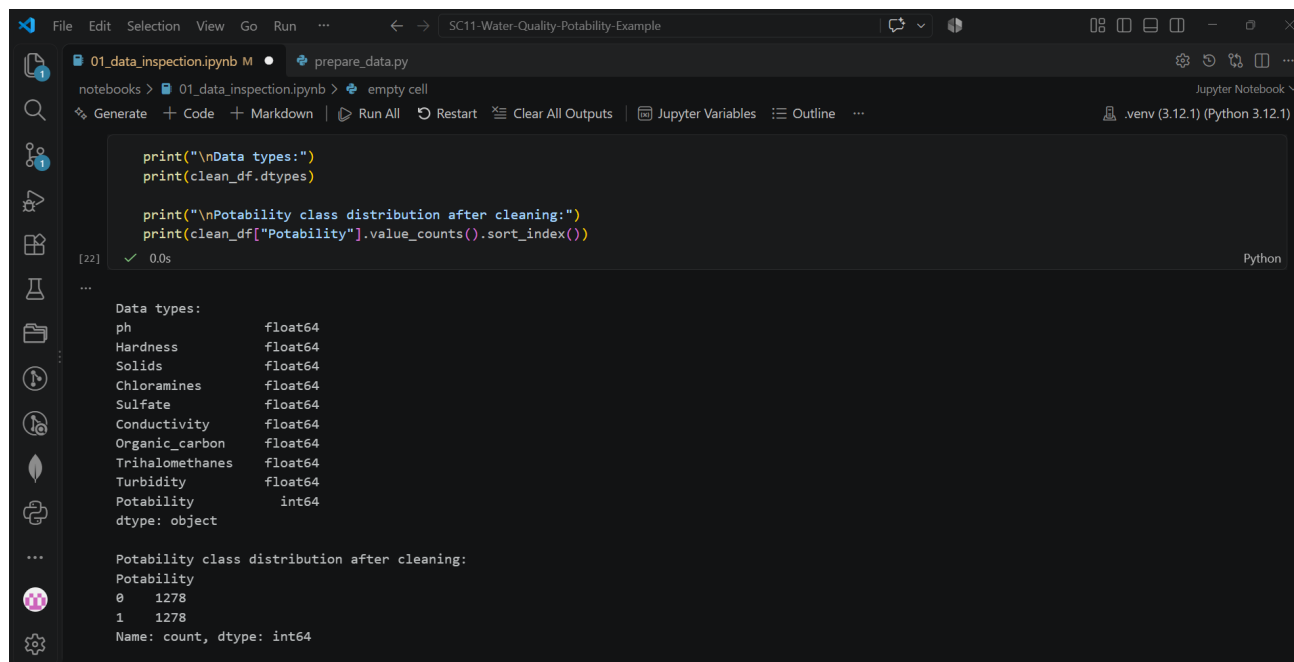
Run the if/else cell that prints VALIDATION PASSED only when all validation counts equal zero.

EXPECTED RESULT

The output says that cleaned data is ready for splitting.

Action 39 - Check data types and class balance

Data types confirm that numeric measurements are numbers, not text. Class balance shows whether each target category is represented fairly before splitting.



The screenshot shows a Jupyter Notebook interface with a dark theme. The notebook is titled '01_data_inspection.ipynb' and is in a 'prepare_data.py' cell. The code in the cell is as follows:

```
print("\nData types:")
print(clean_df.dtypes)

print("\nPotability class distribution after cleaning:")
print(clean_df["Potability"].value_counts().sort_index())
```

The output of the code is displayed below the cell. It shows the data types for various features and the distribution of the 'Potability' class.

```
[22] ✓ 0.0s Python

...
Data types:
ph                float64
Hardness          float64
Solids            float64
Chloramines       float64
Sulfate           float64
Conductivity      float64
Organic_carbon    float64
Trihalomethanes   float64
Turbidity         float64
Potability        int64
dtype: object

...
Potability class distribution after cleaning:
Potability
0    1278
1    1278
Name: count, dtype: int64
```

DO THIS

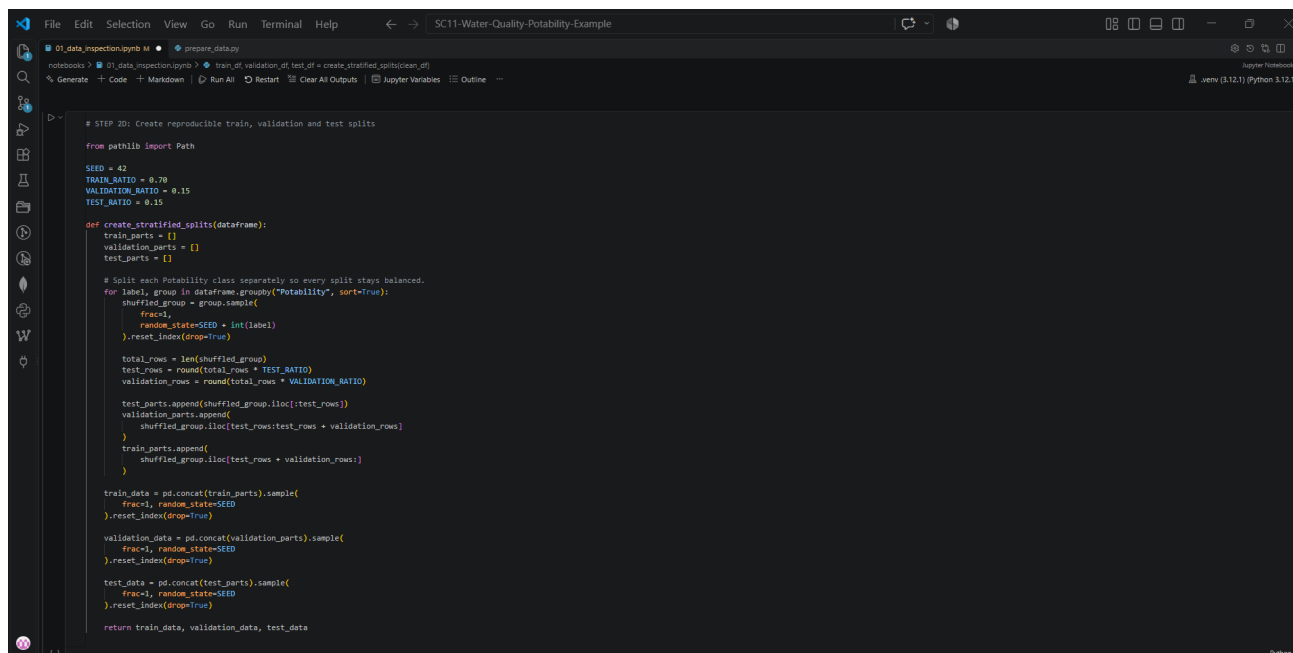
Print `clean_df.dtypes` and `Potability value_counts()`. Use equivalent target or scenario summaries for your own project.

EXPECTED RESULT

Numeric project fields have numeric types and the class/scenario counts are visible.

Action 40 - Define a reproducible stratified split

A train split teaches the model, a validation split helps select settings, and a test split is held back for final evaluation. Stratified means each class keeps a similar proportion in every split.



```
# STEP 20: Create reproducible train, validation and test splits
from pathlib import Path

SEED = 42
TRAIN_RATIO = 0.70
VALIDATION_RATIO = 0.15
TEST_RATIO = 0.15

def create_stratified_splits(dataframe):
    train_parts = []
    validation_parts = []
    test_parts = []

    # Split each Potability class separately so every split stays balanced.
    for label, group in dataframe.groupby("Potability", sort=True):
        shuffled_group = group.sample(
            frac=1,
            random_state=SEED + int(label),
        ).reset_index(drop=True)

        total_rows = len(shuffled_group)
        test_rows = round(total_rows * TEST_RATIO)
        validation_rows = round(total_rows * VALIDATION_RATIO)

        test_parts.append(shuffled_group.iloc[:test_rows])
        validation_parts.append(
            shuffled_group.iloc[test_rows:test_rows + validation_rows]
        )
        train_parts.append(
            shuffled_group.iloc[test_rows + validation_rows:]
        )

    train_data = pd.concat(train_parts).sample(
        frac=1, random_state=SEED
    ).reset_index(drop=True)

    validation_data = pd.concat(validation_parts).sample(
        frac=1, random_state=SEED
    ).reset_index(drop=True)

    test_data = pd.concat(test_parts).sample(
        frac=1, random_state=SEED
    ).reset_index(drop=True)

    return train_data, validation_data, test_data
```

DO THIS

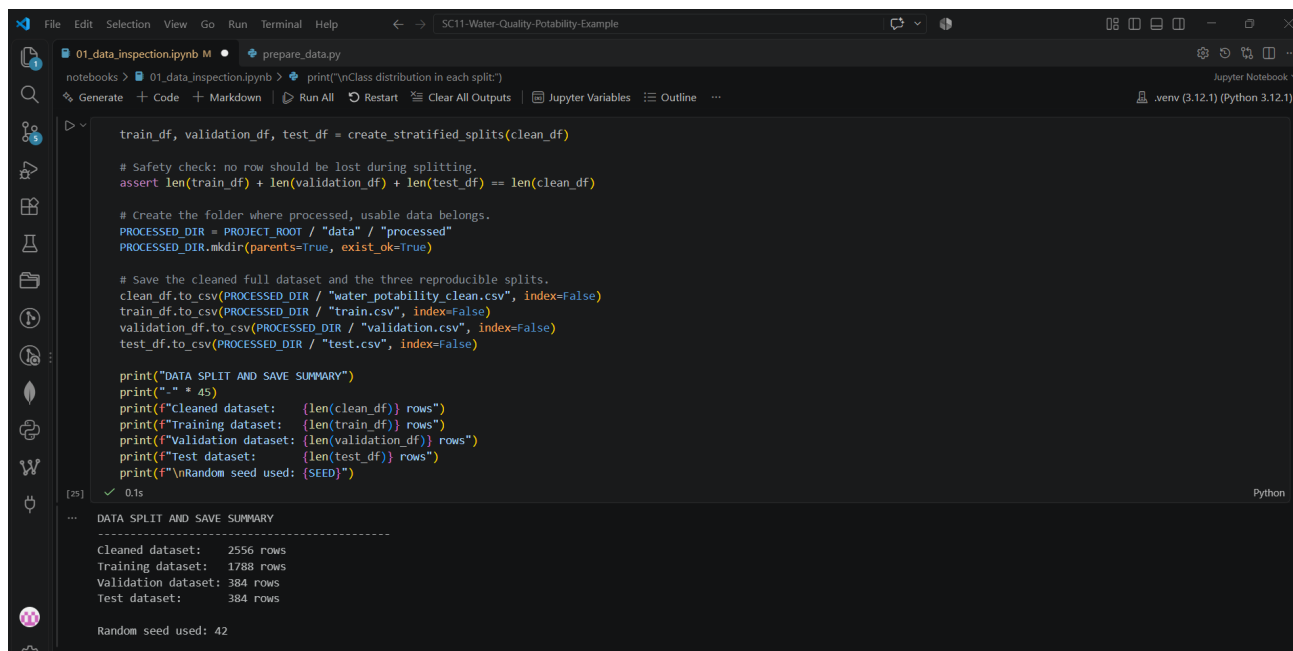
Set one fixed seed, then define the function that shuffles each target class separately and returns train, validation, and test data.

EXPECTED RESULT

The notebook contains a named split function and an explicit fixed random seed.

Action 41 - Save the cleaned dataset and splits

Saved CSV files let another team member or evaluator rerun later steps without manually repeating notebook cells. The raw file remains separate and unchanged.



The screenshot shows a Jupyter Notebook interface with a dark theme. The notebook is titled "01_data_inspection.ipynb" and is open to a cell containing Python code. The code performs the following actions:

- Imports necessary libraries and defines the project root.
- Creates a folder named "processed" within the "data" directory.
- Saves the cleaned full dataset as "water_potability_clean.csv".
- Saves the training, validation, and test splits as "train.csv", "validation.csv", and "test.csv" respectively.
- Prints a summary of the data split and save process, including row counts for each dataset and the random seed used.

The output of the code execution is displayed below the cell, showing the following summary:

```
DATA SPLIT AND SAVE SUMMARY
-----
Cleaned dataset: 2556 rows
Training dataset: 1788 rows
Validation dataset: 384 rows
Test dataset: 384 rows

Random seed used: 42
```

DO THIS

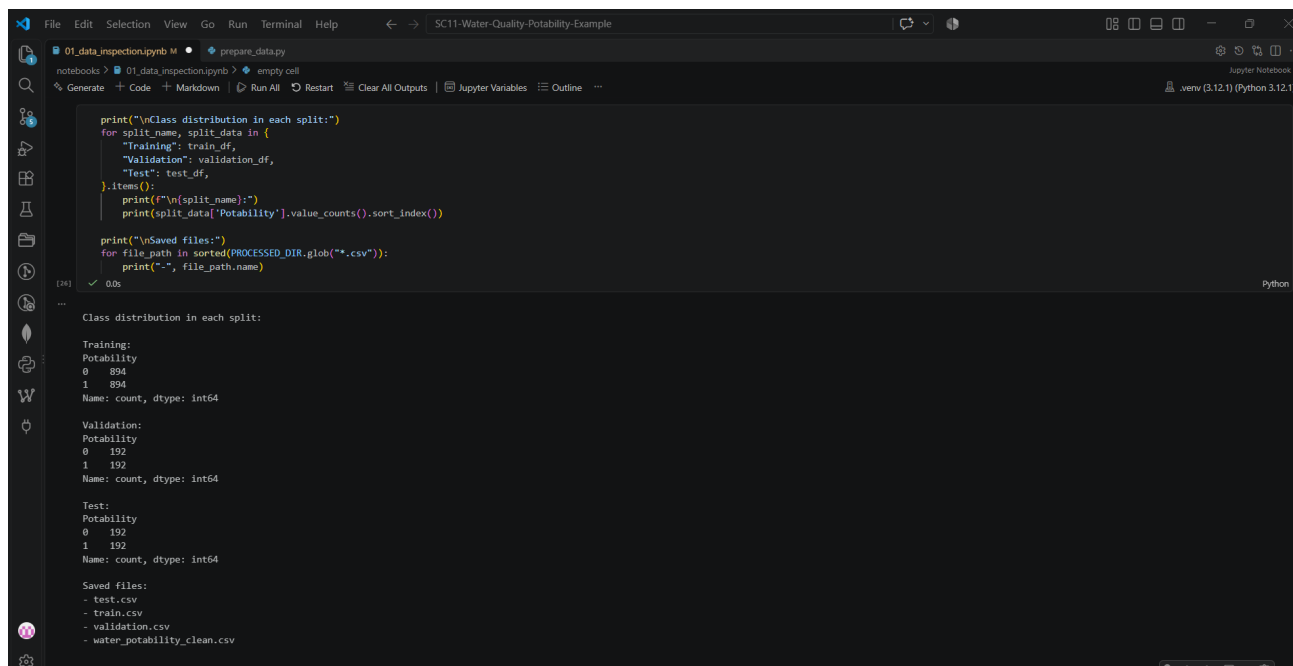
Create data/processed/ if needed. Save the cleaned full data plus train.csv, validation.csv, and test.csv. Print a summary of sizes and seed.

EXPECTED RESULT

The output reports cleaned, training, validation, and test row counts.

Action 42 - Verify split balance and saved files

A split is trustworthy only when its class counts are visible and its output files exist. This prevents accidentally losing records or creating an unbalanced evaluation set.



```
File Edit Selection View Go Run Terminal Help
SC11-Water-Quality-Potability-Example
01_data_inspection.ipynb M prepare_data.py
notebooks > 01_data_inspection.ipynb > empty cell
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline
Python 3.12.1 (Python 3.12.1)

print("\nClass distribution in each split:")
for split_name, split_data in {
    "Training": train_df,
    "Validation": validation_df,
    "Test": test_df,
}.items():
    print(f"\n{split_name}:")
    print(split_data["Potability"].value_counts().sort_index())

print("\nSaved files:")
for file_path in sorted(PROCESSED_DIR.glob("**.csv")):
    print("-", file_path.name)

[44] ✓ 0.0s Python

...

Class distribution in each split:

Training:
Potability
0    894
1    894
Name: count, dtype: int64

Validation:
Potability
0    192
1    192
Name: count, dtype: int64

Test:
Potability
0    192
1    192
Name: count, dtype: int64

Saved files:
- test.csv
- train.csv
- validation.csv
- water_potability_clean.csv
```

DO THIS

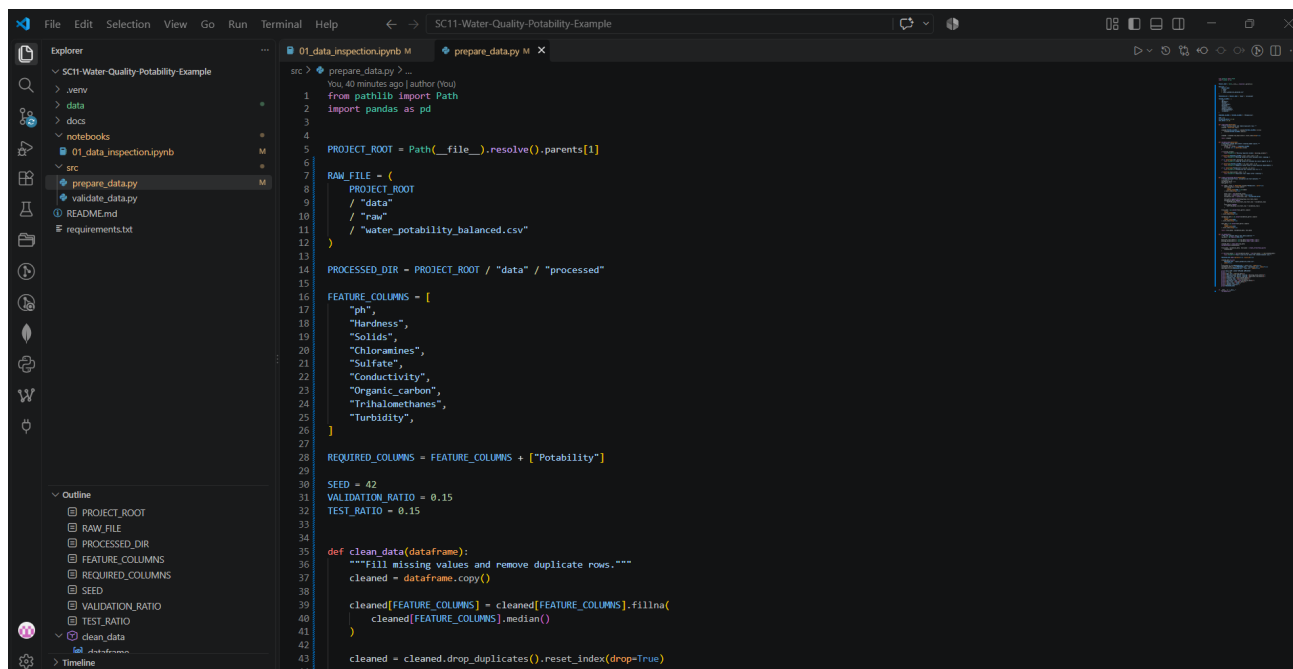
Print the target distribution for each split and list every CSV in data/processed/.

EXPECTED RESULT

Training, validation, and test distributions are shown, followed by the saved filenames.

Action 43 - Open the reusable Python program

The notebook is evidence; src/prepare_data.py is the reusable program. It lets the team recreate the same prepared data using one run button instead of rewriting notebook cells.



```
src > prepare_data.py > ...
1 from pathlib import Path
2 import pandas as pd
3
4
5 PROJECT_ROOT = Path(__file__).resolve().parents[1]
6
7 RAW_FILE = (
8     PROJECT_ROOT
9     / "data"
10    / "raw"
11    / "water_potability_balanced.csv"
12 )
13
14 PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"
15
16 FEATURE_COLUMNS = [
17     "ph",
18     "Hardness",
19     "Solids",
20     "Chloramines",
21     "Sulfate",
22     "Conductivity",
23     "Organic_carbon",
24     "Trihalomethanes",
25     "Turbidity",
26 ]
27
28 REQUIRED_COLUMNS = FEATURE_COLUMNS + ["Potability"]
29
30 SEED = 42
31 VALIDATION_RATIO = 0.15
32 TEST_RATIO = 0.15
33
34
35 def clean_data(dataframe):
36     """Fill missing values and remove duplicate rows."""
37     cleaned = dataframe.copy()
38
39     cleaned[FEATURE_COLUMNS] = cleaned[FEATURE_COLUMNS].fillna(
40         cleaned[FEATURE_COLUMNS].median()
41     )
42
43     cleaned = cleaned.drop_duplicates().reset_index(drop=True)
44
```

DO THIS

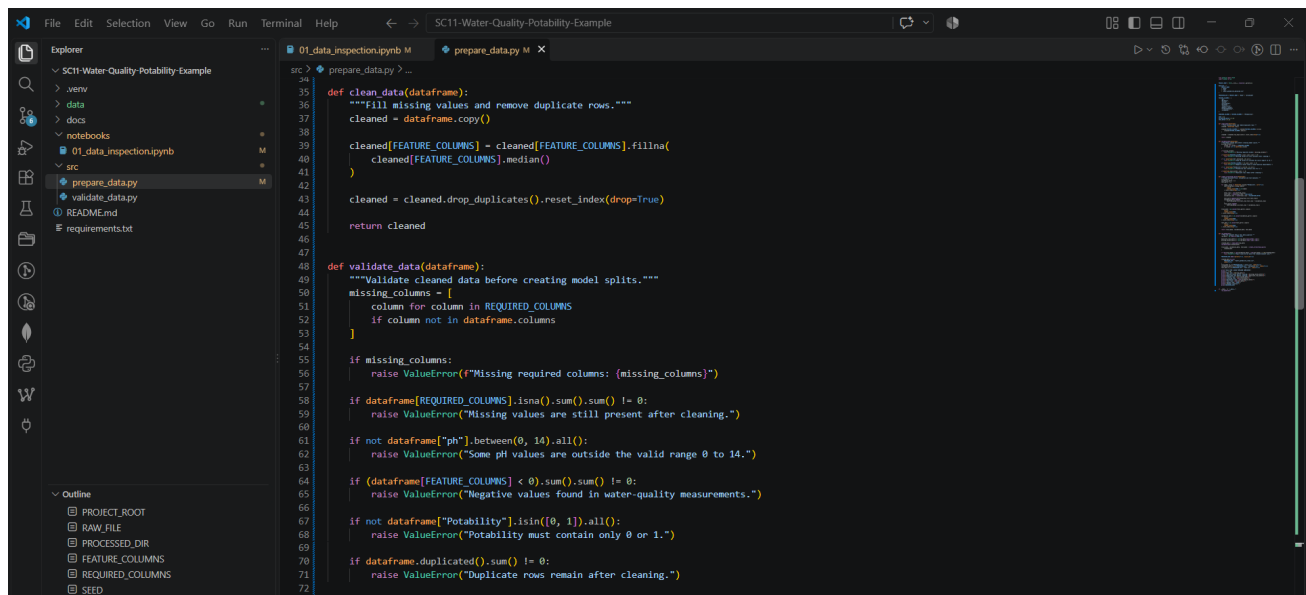
Open src/prepare_data.py. Use PROJECT_ROOT and relative paths, never a personal Downloads or desktop path.

EXPECTED RESULT

The program begins with imports, project paths, output folder, feature fields, and a random seed.

Action 44 - Write reusable cleaning and validation functions

A function gives one operation a clear name. `clean_data()` repeats the documented repair method, while `validate_data()` stops later steps when a required rule fails.



```
34 def clean_data(dataframe):
35     """Fill missing values and remove duplicate rows."""
36     cleaned = dataframe.copy()
37
38     cleaned[FEATURE_COLUMNS] = cleaned[FEATURE_COLUMNS].fillna(
39         cleaned[FEATURE_COLUMNS].median()
40     )
41
42     cleaned = cleaned.drop_duplicates().reset_index(drop=True)
43
44     return cleaned
45
46
47 def validate_data(dataframe):
48     """Validate cleaned data before creating model splits."""
49     missing_columns = [
50         column for column in REQUIRED_COLUMNS
51         if column not in dataframe.columns
52     ]
53
54     if missing_columns:
55         raise ValueError(f"Missing required columns: {missing_columns}")
56
57     if dataframe[REQUIRED_COLUMNS].isna().sum().sum() != 0:
58         raise ValueError("Missing values are still present after cleaning.")
59
60     if not dataframe["ph"].between(0, 14).all():
61         raise ValueError("Some pH values are outside the valid range 0 to 14.")
62
63     if (dataframe[FEATURE_COLUMNS] < 0).sum().sum() != 0:
64         raise ValueError("Negative values found in water-quality measurements.")
65
66     if not dataframe["Potability"].isin([0, 1]).all():
67         raise ValueError("Potability must contain only 0 or 1.")
68
69     if dataframe.duplicated().sum() != 0:
70         raise ValueError("Duplicate rows remain after cleaning.")
71
72
```

DO THIS

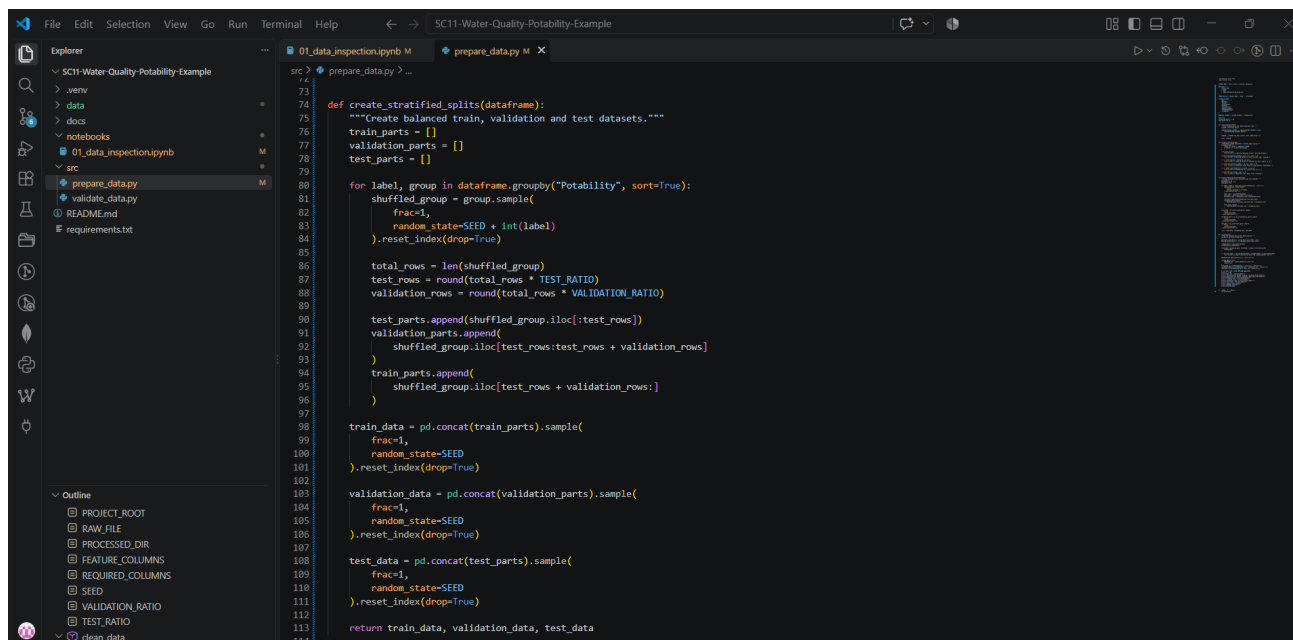
Add `clean_data(dataframe)` and `validate_data(dataframe)`. Clean a copy, then raise a clear `ValueError` if a required field, value range, label, or duplicate rule fails.

EXPECTED RESULT

Both reusable functions are visible; `clean_data` does not alter the raw input dataframe.

Action 45 - Write the reusable split function

A fixed seed and one split function make results repeatable. Another person should get the same split sizes and class balance when they rerun the program.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'SC11-Water-Quality-Potability-Example'. The code editor shows the implementation of a function named 'create_stratified_splits' in a file named 'prepare_data.py'. The function takes a 'dataframe' as input and returns three datasets: 'train_data', 'validation_data', and 'test_data'. The function uses 'groupby' to split the data by 'Potability' and 'sort=True'. It then uses 'sample' to create balanced train, validation, and test datasets. The function uses a fixed seed for reproducibility.

```
def create_stratified_splits(dataframe):  
    """Create balanced train, validation and test datasets."""  
    train_parts = []  
    validation_parts = []  
    test_parts = []  
  
    for label, group in dataframe.groupby("Potability", sort=True):  
        shuffled_group = group.sample(  
            frac=1,  
            random_state=SEED + int(label)  
        ).reset_index(drop=True)  
  
        total_rows = len(shuffled_group)  
        test_rows = round(total_rows * TEST_RATIO)  
        validation_rows = round(total_rows * VALIDATION_RATIO)  
  
        test_parts.append(shuffled_group.iloc[:test_rows])  
        validation_parts.append(  
            shuffled_group.iloc[test_rows:test_rows + validation_rows]  
        )  
        train_parts.append(  
            shuffled_group.iloc[test_rows + validation_rows:]  
        )  
  
    train_data = pd.concat(train_parts).sample(  
        frac=1,  
        random_state=SEED  
    ).reset_index(drop=True)  
  
    validation_data = pd.concat(validation_parts).sample(  
        frac=1,  
        random_state=SEED  
    ).reset_index(drop=True)  
  
    test_data = pd.concat(test_parts).sample(  
        frac=1,  
        random_state=SEED  
    ).reset_index(drop=True)  
  
    return train_data, validation_data, test_data
```

DO THIS

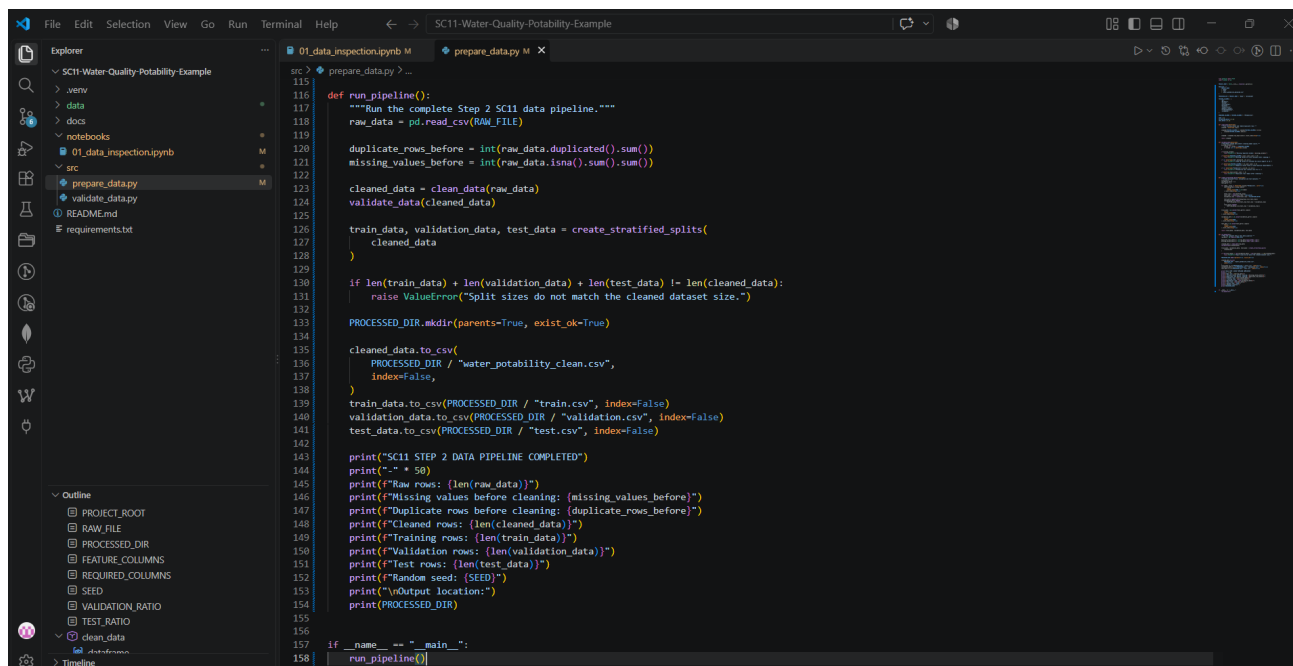
Add `create_stratified_splits(dataframe)`. Use the same seed, ratios, class-wise shuffle, and returned train/validation/test datasets as the notebook.

EXPECTED RESULT

The function visibly creates all three datasets and returns them.

Action 46 - Connect the full pipeline

`run_pipeline()` is the ordered recipe: read raw data, clean it, validate it, split it, save it, and print proof. One clear entry point makes the project maintainable.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'SC11-Water-Quality-Potability-Example' with files like .env, data, docs, notebooks, 01_data_inspection.ipynb, src, prepare_data.py, validate_data.py, README.md, and requirements.txt. The code editor shows the `run_pipeline()` function in `prepare_data.py`. The function reads raw data, cleans it, validates it, splits it into training, validation, and test sets, and saves them as CSV files. It also prints a summary of the data and the output location.

```
115 def run_pipeline():
116     """Run the complete Step 2 SC11 data pipeline."""
117     raw_data = pd.read_csv(RAW_FILE)
118
119     duplicate_rows_before = int(raw_data.duplicated().sum())
120     missing_values_before = int(raw_data.isna().sum().sum())
121
122     cleaned_data = clean_data(raw_data)
123     validate_data(cleaned_data)
124
125     train_data, validation_data, test_data = create_stratified_splits(
126         cleaned_data
127     )
128
129     if len(train_data) + len(validation_data) + len(test_data) != len(cleaned_data):
130         raise ValueError("Split sizes do not match the cleaned dataset size.")
131
132     PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
133
134     cleaned_data.to_csv(
135         PROCESSED_DIR / "water_potability_clean.csv",
136         index=False,
137     )
138     train_data.to_csv(PROCESSED_DIR / "train.csv", index=False)
139     validation_data.to_csv(PROCESSED_DIR / "validation.csv", index=False)
140     test_data.to_csv(PROCESSED_DIR / "test.csv", index=False)
141
142     print("SC11 STEP 2 DATA PIPELINE COMPLETED")
143     print("-" * 50)
144     print(f"Raw rows: {len(raw_data)}")
145     print(f"Missing values before cleaning: {missing_values_before}")
146     print(f"Duplicate rows before cleaning: {duplicate_rows_before}")
147     print(f"Cleaned rows: {len(cleaned_data)}")
148     print(f"Training rows: {len(train_data)}")
149     print(f"Validation rows: {len(validation_data)}")
150     print(f"Test rows: {len(test_data)}")
151     print(f"Random seed: {SEED}")
152     print(f"Output location:")
153     print(PROCESSED_DIR)
154
155 if __name__ == "__main__":
156     run_pipeline()
```

DO THIS

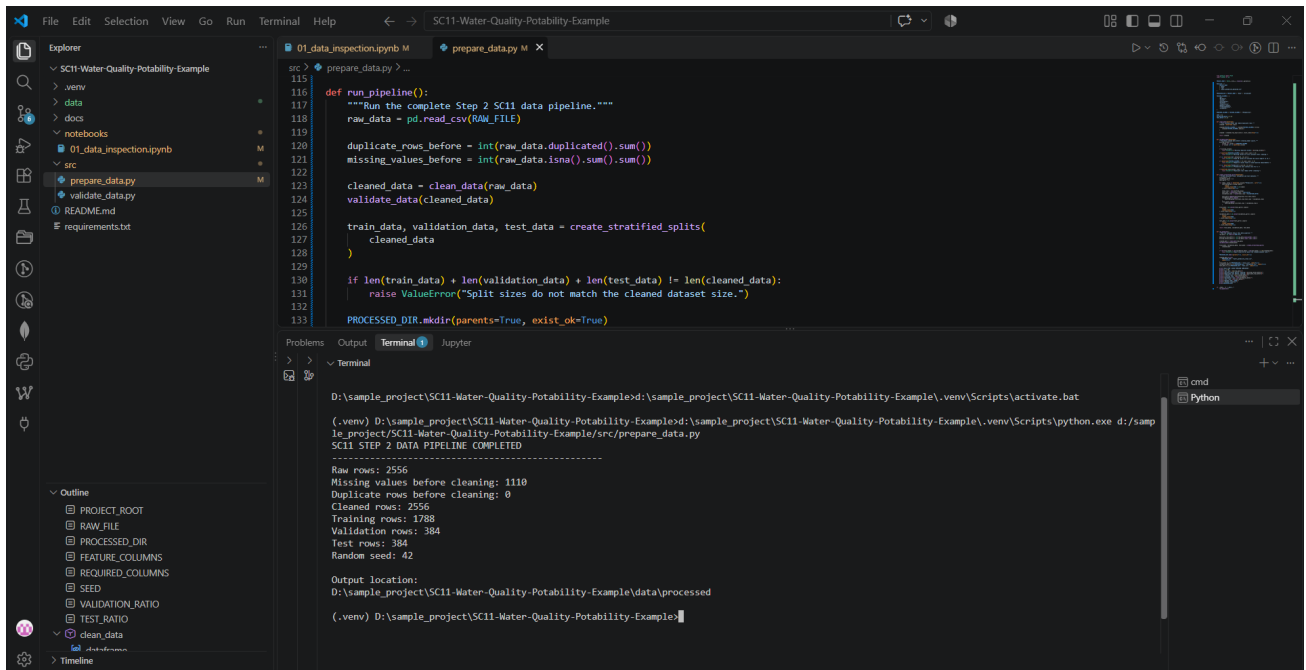
Create `run_pipeline()`, call the helper functions in order, save four processed CSV files, print the summary, and call it under `if __name__ == '__main__':`.

EXPECTED RESULT

The full path from raw input to processed output is visible in one program.

Action 47 - Run the Python pipeline from VS Code

The green Run Python File button runs the saved .py program. This is different from a notebook cell: it proves the reusable file works independently.



DO THIS

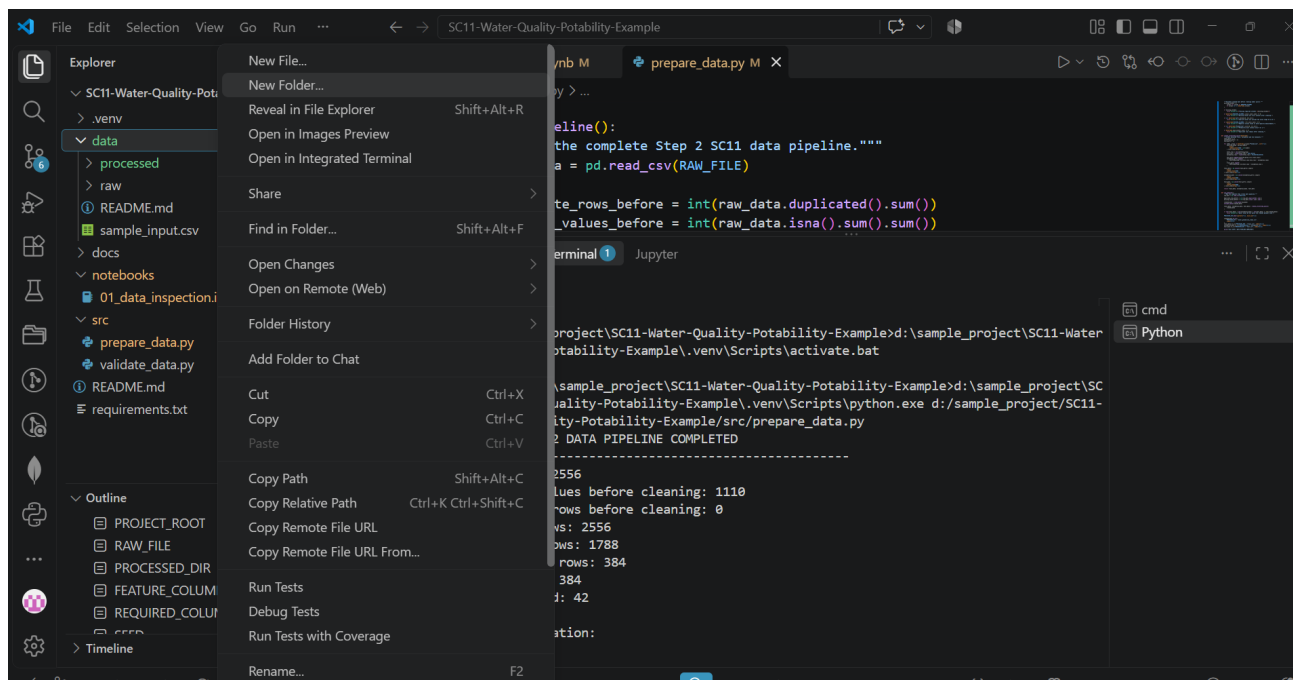
Save the file, then click Run Python File. Read the terminal summary; do not type personal file paths into the program.

EXPECTED RESULT

SC11 STEP 2 DATA PIPELINE COMPLETED appears with rows, seed, and output location.

Action 48 - Create the reports folder

A report folder keeps written evidence separate from source code and raw data. This makes a repository easier to assess and maintain.



DO THIS

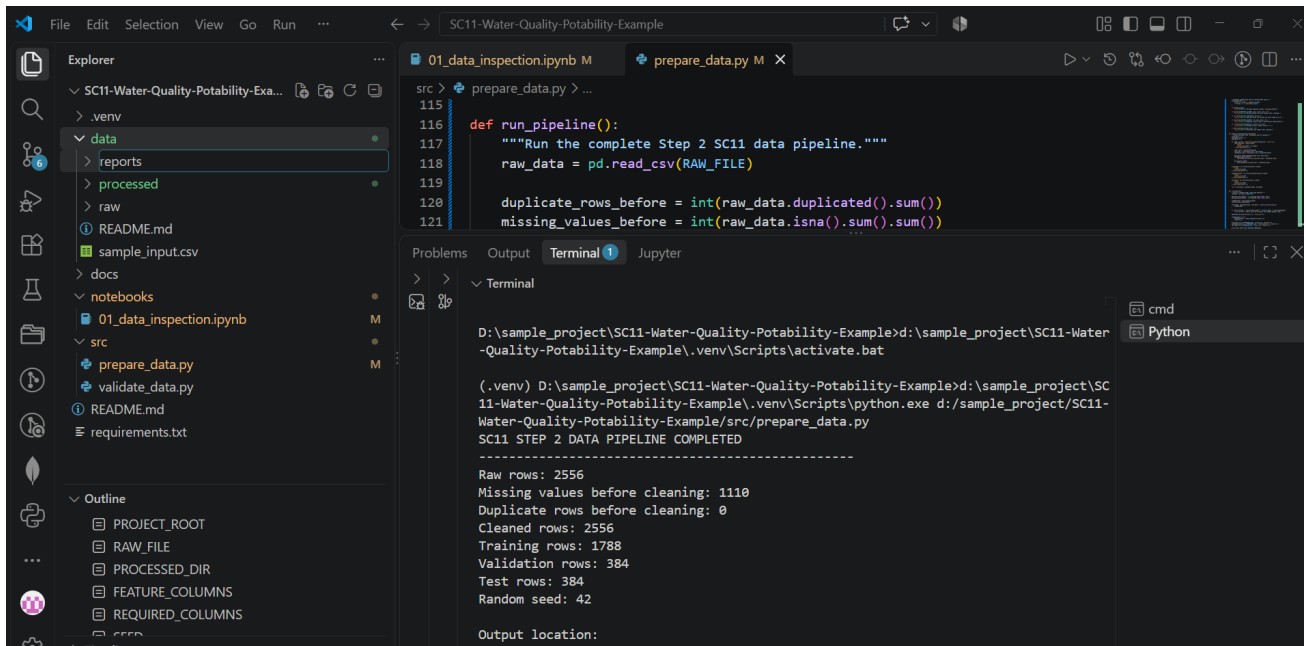
In Explorer, right-click data and choose New Folder.

EXPECTED RESULT

VS Code opens a name field for the new folder under data.

Action 49 - Name the reports folder

A meaningful folder name keeps repository evidence easy to locate. The reports folder will hold the written explanation of your data pipeline.



DO THIS

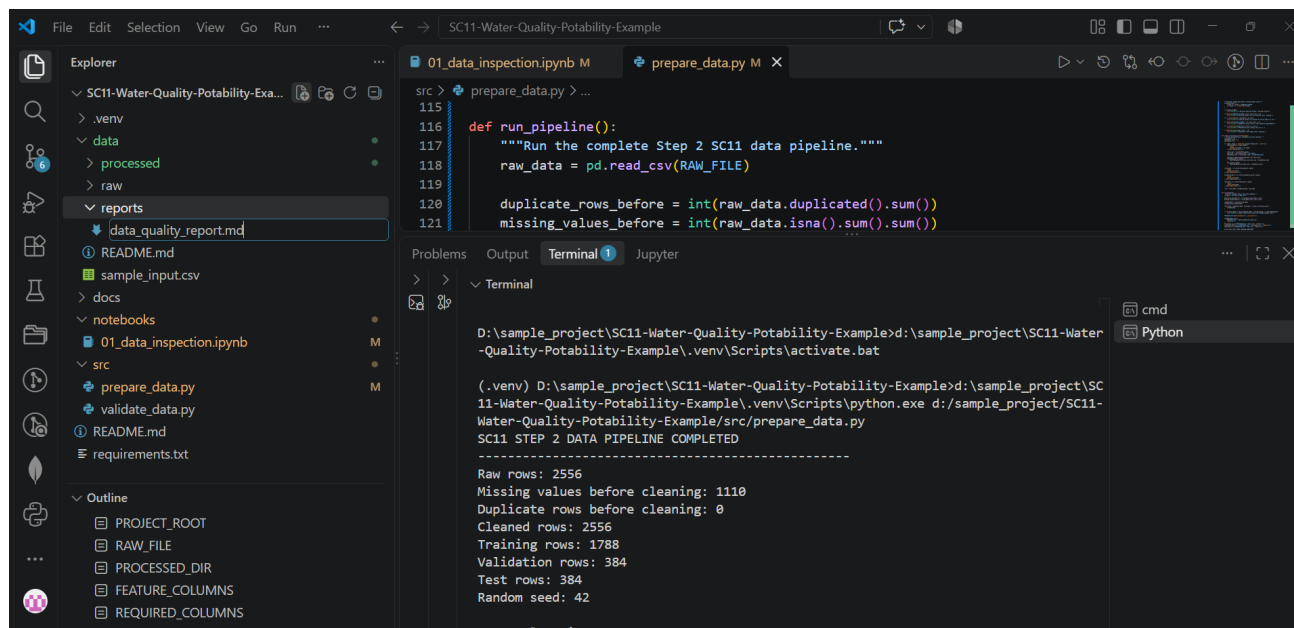
Type reports and press Enter. It must be inside data, beside raw and processed.

EXPECTED RESULT

data/reports appears beside raw and processed.

Action 50 - Create the data quality report

Markdown is a simple text document that GitHub displays neatly. The report explains results that code output alone may not make clear.



DO THIS

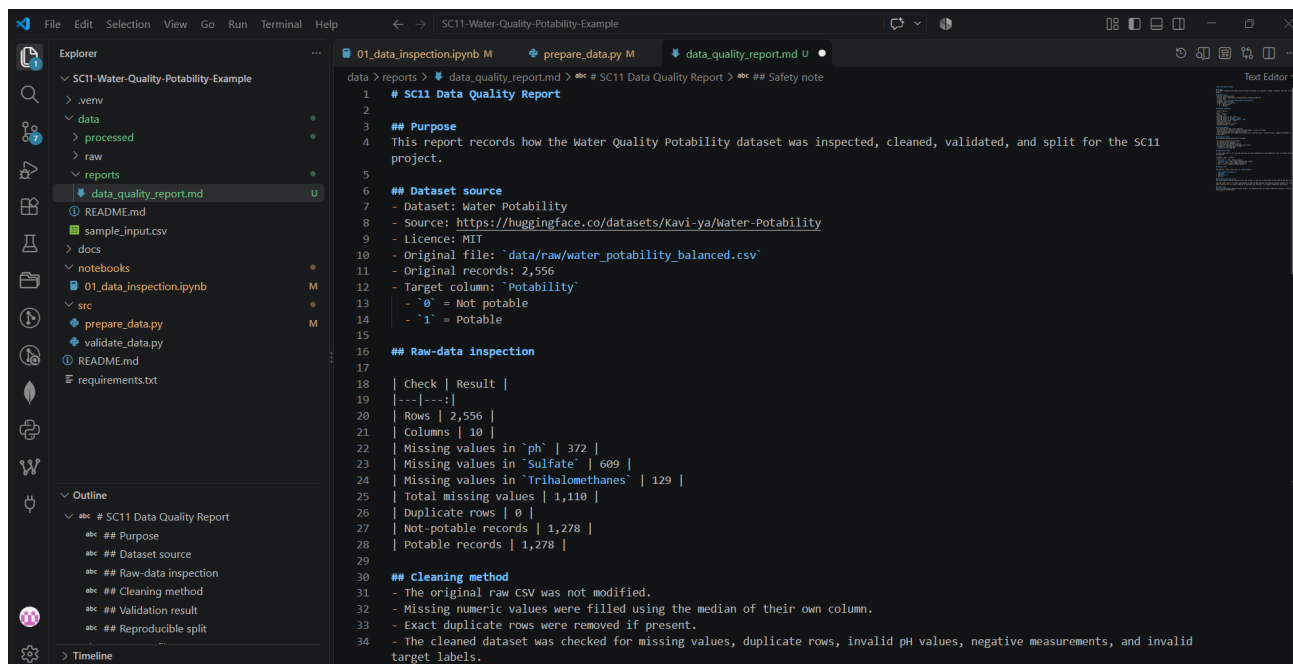
Inside data/reports, create data_quality_report.md. Use a clear filename and save it before writing more files.

EXPECTED RESULT

The new Markdown file appears under data/reports.

Action 51 - Document source and cleaning decisions

A useful report states the source, licence, raw record count, target meaning, missing-value counts, and cleaning method. This is evidence, not a copy-paste formality.



```
1 # SC11 Data Quality Report
2
3 ## Purpose
4 This report records how the Water Quality Potability dataset was inspected, cleaned, validated, and split for the SC11
5 project.
6
7 ## Dataset source
8 - Dataset: Water Potability
9 - Source: https://huggingface.co/datasets/Kavi-ya/Water-Potability
10 - Licence: MIT
11 - Original file: `data/raw/water_potability_balanced.csv`
12 - Original records: 2,556
13 - Target column: `Potability`
14   - `0` = Not potable
15   - `1` = Potable
16
17 ## Raw-data inspection
18 | Check | Result |
19 |---|---|
20 | Rows | 2,556 |
21 | Columns | 10 |
22 | Missing values in `ph` | 372 |
23 | Missing values in `Sulfate` | 609 |
24 | Missing values in `Trihalomethanes` | 129 |
25 | Total missing values | 1,110 |
26 | Duplicate rows | 0 |
27 | Not-potable records | 1,278 |
28 | Potable records | 1,278 |
29
30 ## Cleaning method
31 - The original raw CSV was not modified.
32 - Missing numeric values were filled using the median of their own column.
33 - Exact duplicate rows were removed if present.
34 - The cleaned dataset was checked for missing values, duplicate rows, invalid pH values, negative measurements, and invalid
35 target labels.
```

DO THIS

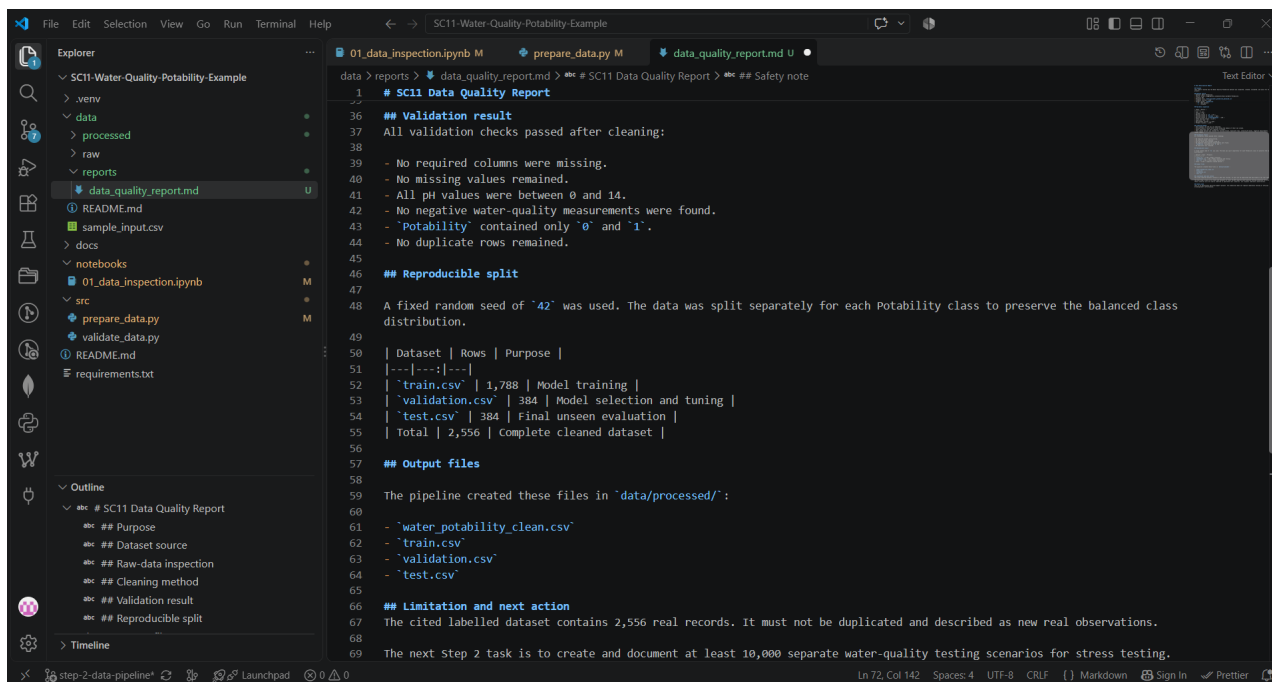
Write the project purpose, source URL, licence, original data facts, raw inspection table, and the exact cleaning rules used.

EXPECTED RESULT

The report shows headings and facts that match the executed notebook output.

Action 52 - Document validation and splits

Readers need to know why the prepared data is safe to use and how the three files will be used later. This makes model evaluation traceable.



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'SC11-Water-Quality-Potability-Example' with subfolders like '.venv', 'data', 'processed', 'raw', 'reports', and files like 'data_quality_report.md', 'README.md', 'sample_input.csv', 'docs', 'notebooks', '01_data_inspection.ipynb', 'src', 'prepare_data.py', 'validate_data.py', 'requirements.txt', and 'README.md'. The code editor displays the content of 'data_quality_report.md', which includes a section for 'SC11 Data Quality Report' and a 'Reproducible split' table.

```
data > reports > data_quality_report.md > SC11 Data Quality Report > Safety note
1  ## SC11 Data Quality Report
2
3  ## Validation result
36 All validation checks passed after cleaning:
37
38 - No required columns were missing.
39 - No missing values remained.
40 - All pH values were between 0 and 14.
41 - No negative water-quality measurements were found.
42 - 'Potability' contained only '0' and '1'.
43 - No duplicate rows remained.
44
45 ## Reproducible split
46
47 A fixed random seed of '42' was used. The data was split separately for each Potability class to preserve the balanced class
48 distribution.
49
50 | Dataset | Rows | Purpose |
51 |---|---|---|
52 | 'train.csv' | 1,788 | Model training |
53 | 'validation.csv' | 384 | Model selection and tuning |
54 | 'test.csv' | 384 | Final unseen evaluation |
55 | Total | 2,556 | complete cleaned dataset |
56
57 ## Output files
58
59 The pipeline created these files in 'data/processed/':
60
61 - 'water_potability_clean.csv'
62 - 'train.csv'
63 - 'validation.csv'
64 - 'test.csv'
65
66 ## Limitation and next action
67
68 The cited labelled dataset contains 2,556 real records. It must not be duplicated and described as new real observations.
69
70 The next Step 2 task is to create and document at least 10,000 separate water-quality testing scenarios for stress testing.
```

DO THIS

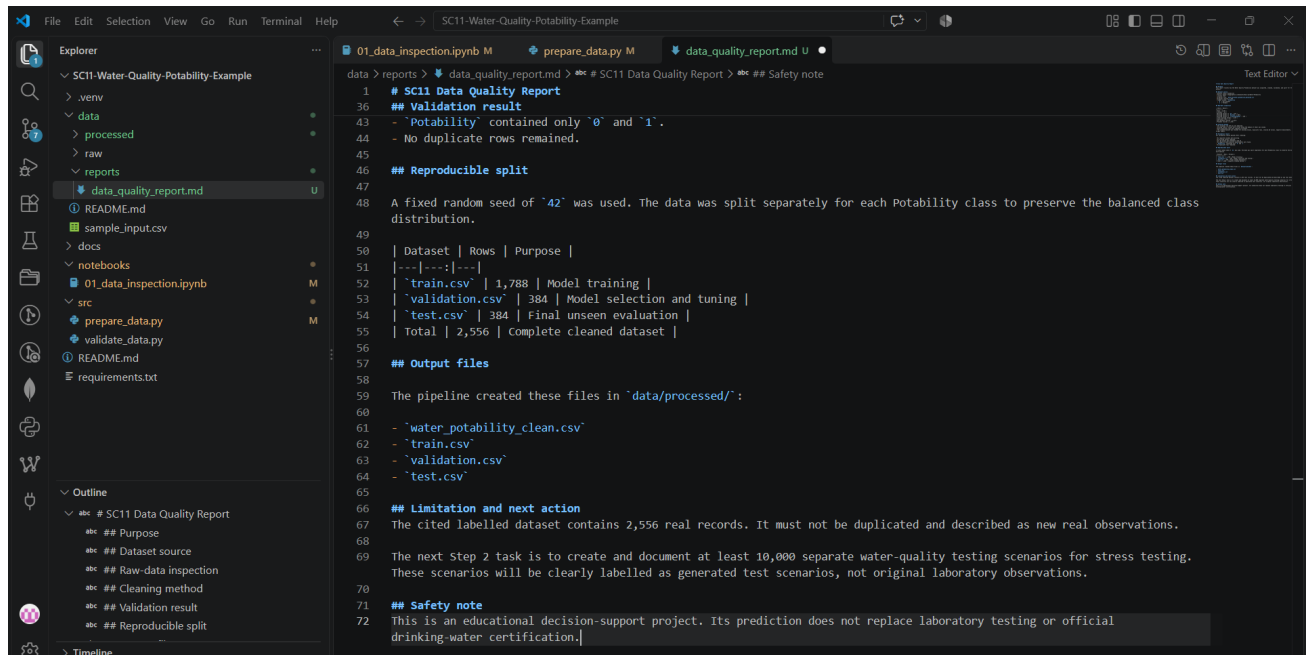
Record the passing validation rules, the fixed seed, the split sizes, intended purpose of each split, and the output filenames.

EXPECTED RESULT

The report includes the validation result and a reproducible split table.

Action 53 - State the limitation honestly

Academic quality includes stating what the data cannot prove. Do not turn 2,556 real observations into a false claim of 10,000 real laboratory records.



```
1 # SC11 Data Quality Report
36 ## Validation result
43 - 'Potability' contained only '0' and '1'.
44 - No duplicate rows remained.
45
46 ## Reproducible split
47
48 A fixed random seed of '42' was used. The data was split separately for each Potability class to preserve the balanced class
distribution.
49
50 | Dataset | Rows | Purpose |
51 |---|---|---|
52 | 'train.csv' | 1,788 | Model training |
53 | 'validation.csv' | 384 | Model selection and tuning |
54 | 'test.csv' | 384 | Final unseen evaluation |
55 | Total | 2,556 | complete cleaned dataset |
56
57 ## Output files
58
59 The pipeline created these files in `data/processed/`:
60
61 - `water_potability_clean.csv`
62 - `train.csv`
63 - `validation.csv`
64 - `test.csv`
65
66 ## Limitation and next action
67 The cited labelled dataset contains 2,556 real records. It must not be duplicated and described as new real observations.
68
69 The next Step 2 task is to create and document at least 10,000 separate water-quality testing scenarios for stress testing.
These scenarios will be clearly labelled as generated test scenarios, not original laboratory observations.
70
71 ## Safety note
72 This is an educational decision-support project. Its prediction does not replace laboratory testing or official
drinking water certification.
```

DO THIS

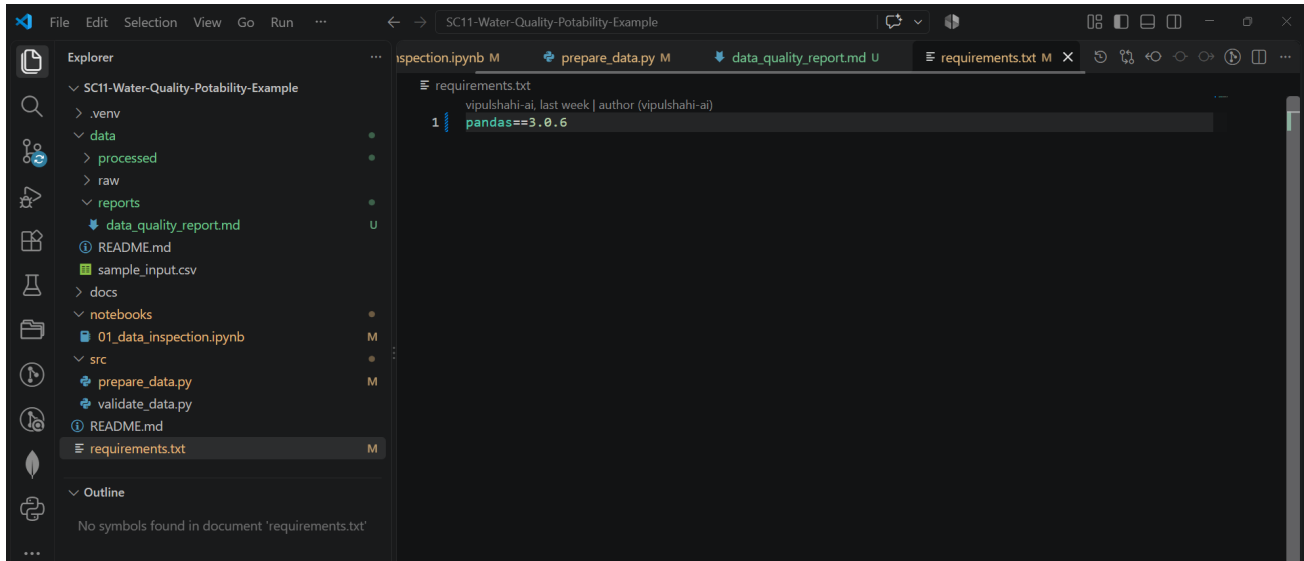
Explain that separately generated stress-test scenarios must be labelled as generated, not real observations. Add the project safety note.

EXPECTED RESULT

The report clearly separates real cited data from later generated testing scenarios.

Action 54 - Record required Python packages

requirements.txt tells a new team member what to install before running the same program. Pinning the working package version improves repeatability.



DO THIS

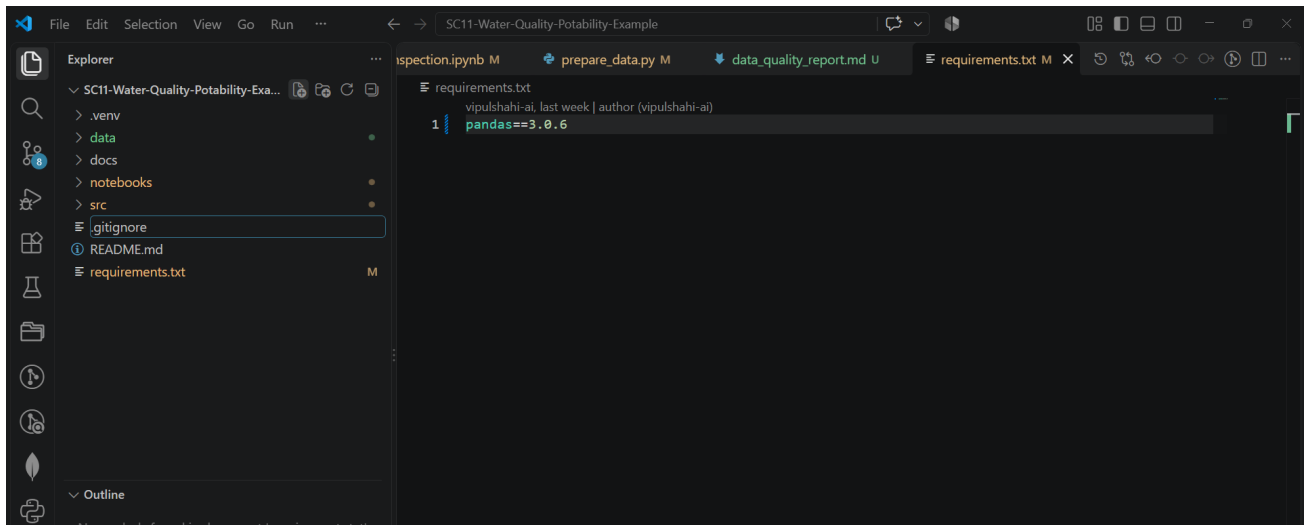
Open requirements.txt and add the package used by the pipeline, for example pandas==3.0.6 for SC11.

EXPECTED RESULT

The file lists the required package and version on its own line.

Action 55 - Create a Git ignore file

A .gitignore file tells Git which local-only files must stay out of the shared repository. It protects the repository from bulky environments and temporary cache files.



DO THIS

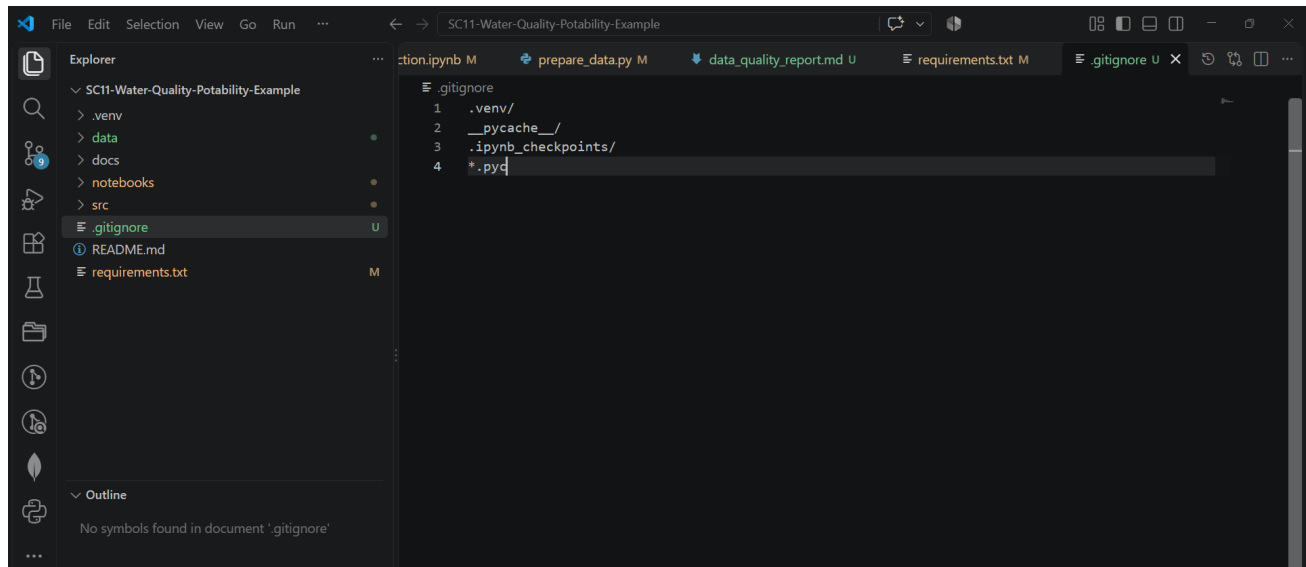
At the repository root, create a file named exactly .gitignore, including the leading dot.

EXPECTED RESULT

The Explorer displays .gitignore in the project root.

Action 56 - Ignore local environment and cache files

A virtual environment is installed separately on each computer. It is not project source code and should not be uploaded to GitHub.



DO THIS

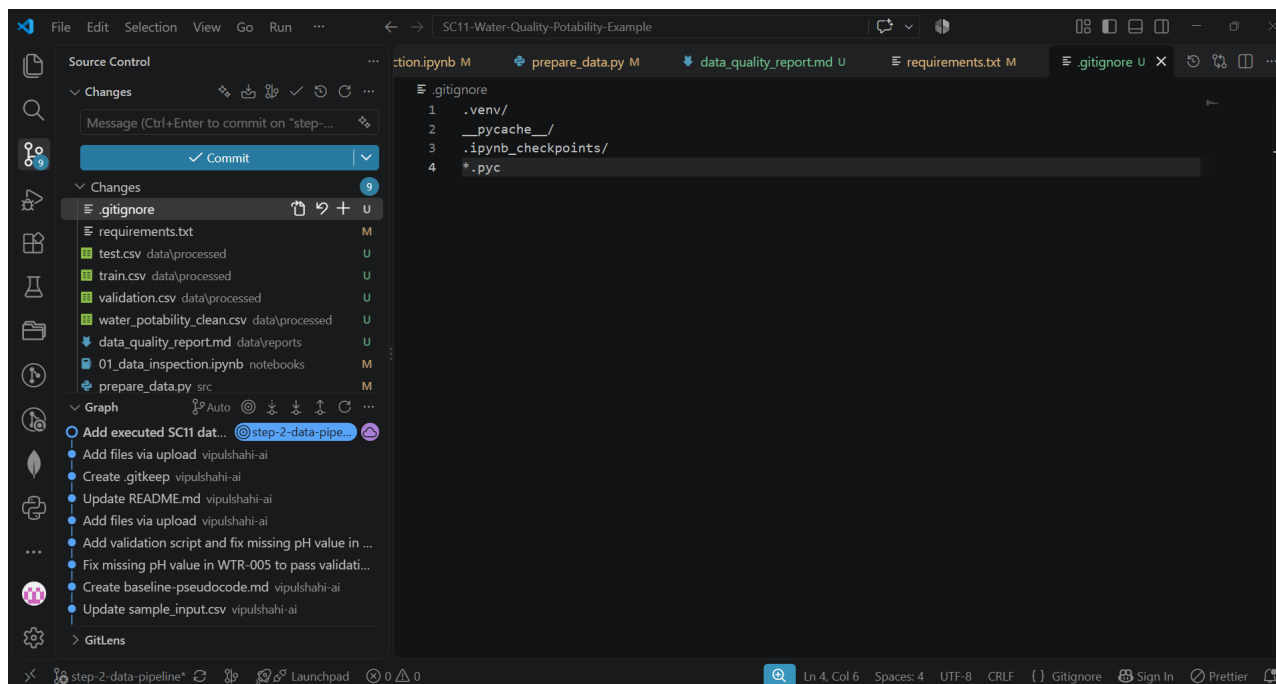
Add `.venv/`, `__pycache__/`, `.ipynb_checkpoints/`, and `*.pyc` to `.gitignore`, then save.

EXPECTED RESULT

Those local files no longer appear as files to stage in Source Control.

Action 57 - Review every changed file

Source Control is the final review before saving a Git checkpoint. Its Changes list shows exactly what will be included and helps prevent accidental uploads.



DO THIS

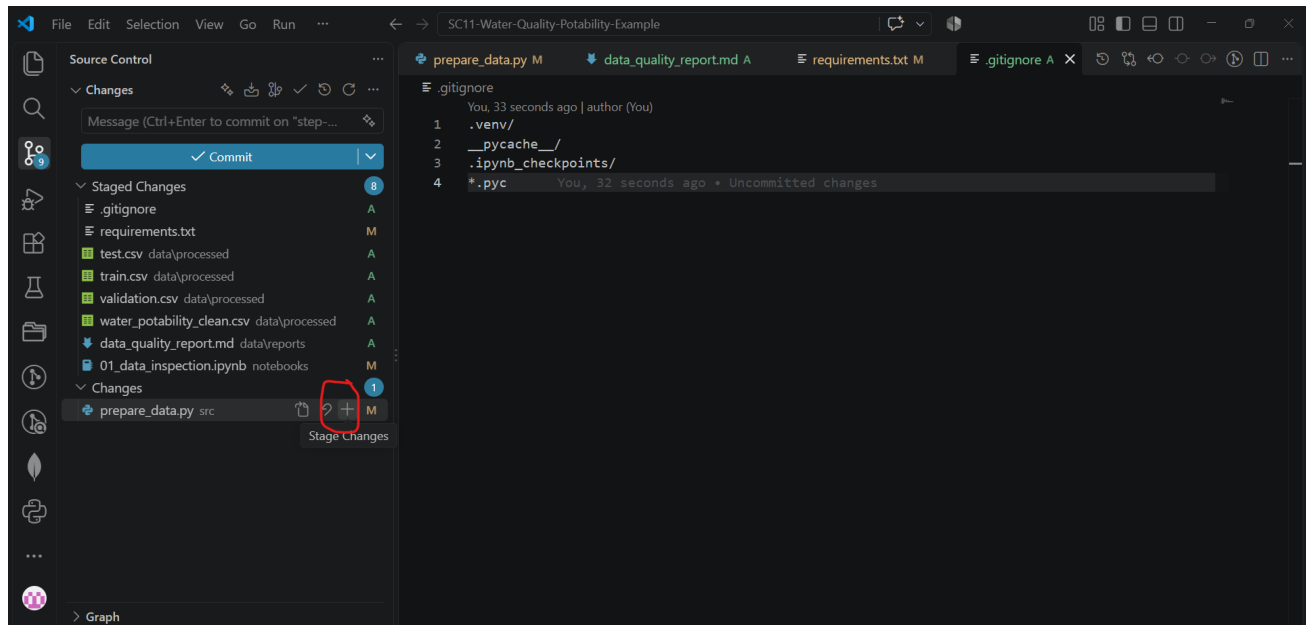
Open Source Control and check the list: notebook, program, processed CSV files, report, requirements.txt, and .gitignore should be present.

EXPECTED RESULT

Only intentional Step 2 work appears in Changes; local .venv does not appear.

Action 58 - Stage the selected files

Staging is a deliberate selection step. It is like putting the finished pages into the submission envelope before sealing it with a commit.



DO THIS

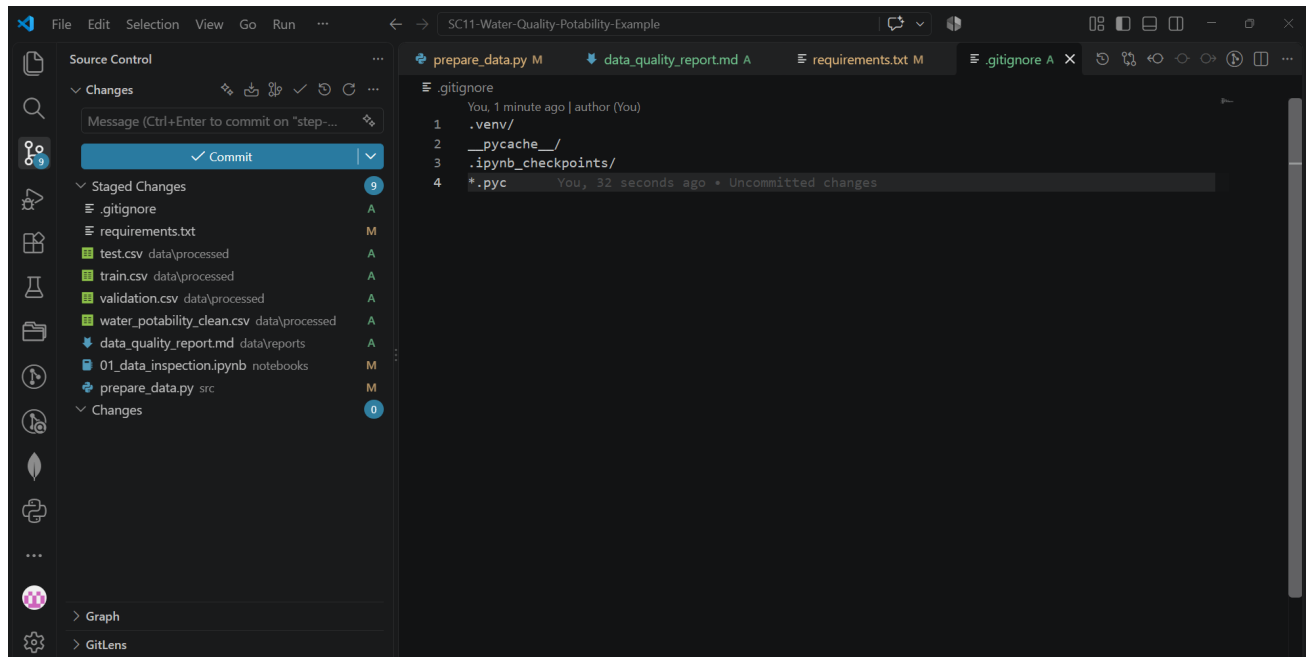
Click + beside each intended file. If one intended file remains under Changes, stage it separately before committing.

EXPECTED RESULT

All intended Step 2 files move under Staged Changes.

Action 59 - Confirm the staging list is complete

Before committing, pause and compare the Staged Changes list with the work you actually completed. This creates a clean, understandable history.



DO THIS

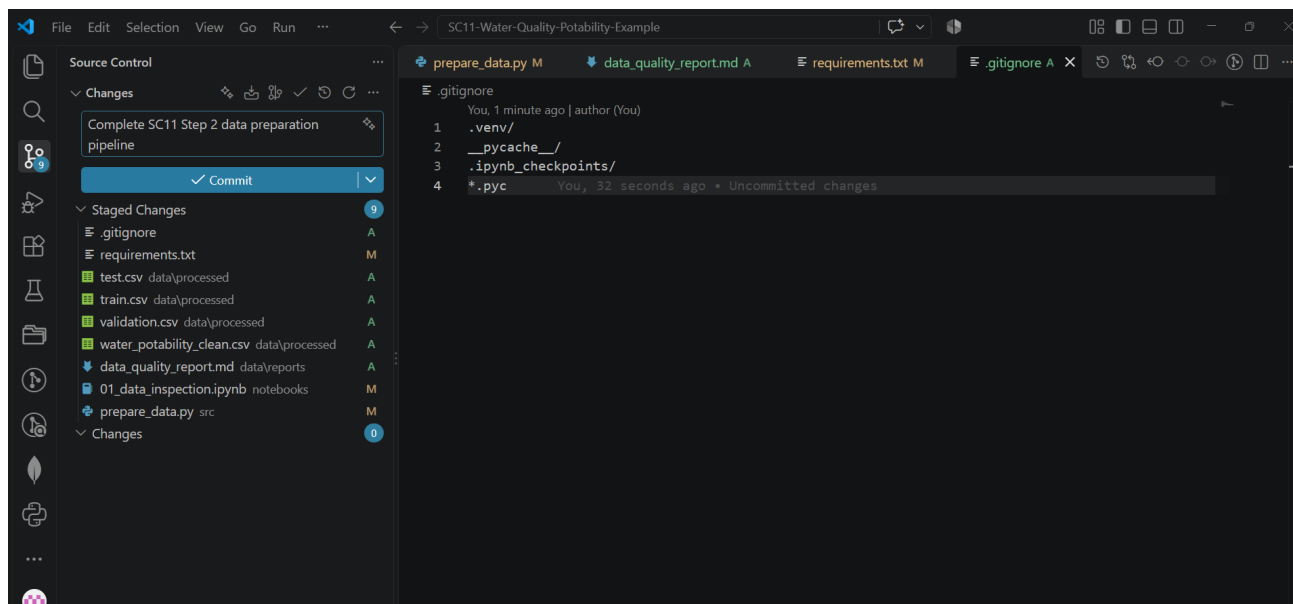
Confirm the prepared data, notebook, program, report, requirements, and .gitignore are staged. Leave unrelated files out.

EXPECTED RESULT

Changes shows 0 and the complete intended list is staged.

Action 60 - Write a descriptive commit message

A commit message tells future readers what was achieved at that checkpoint. It should describe the completed work, not simply say update or final.



DO THIS

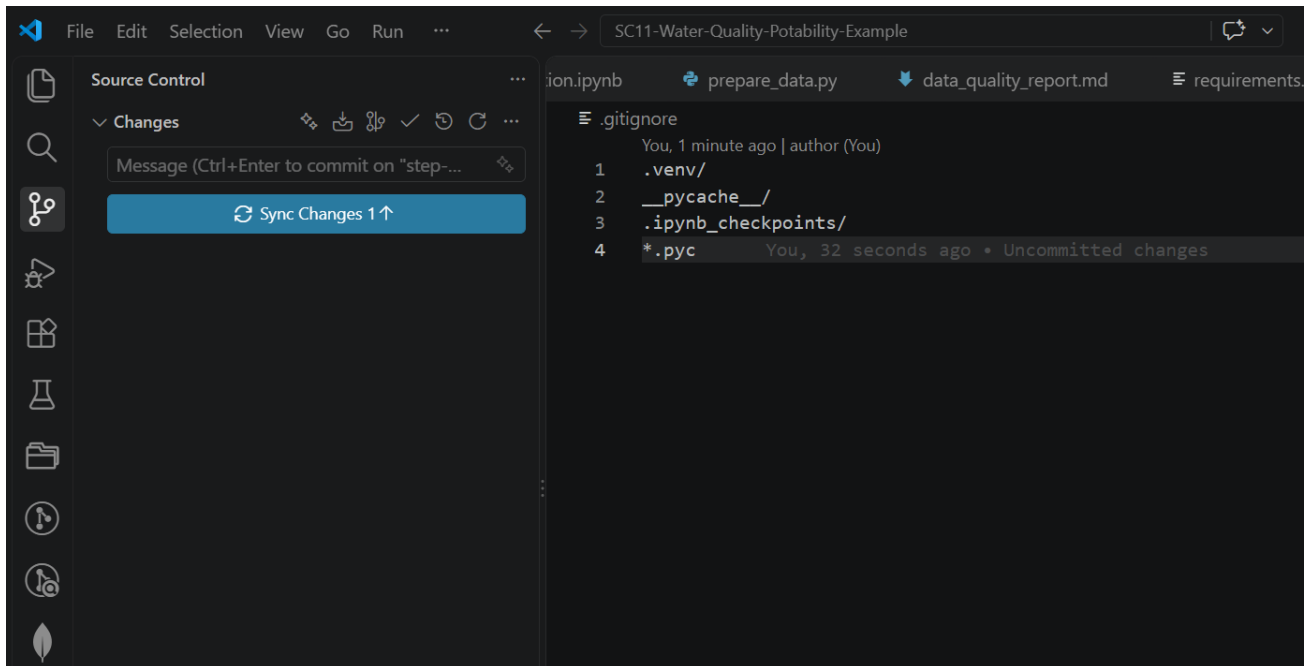
Enter a message such as Complete SC11 Step 2 data preparation pipeline, then click Commit.

EXPECTED RESULT

The staging list clears and VS Code offers Sync Changes.

Action 61 - Start syncing to GitHub

Commit saves locally; Sync Changes publishes the commit to the shared GitHub branch and brings in teammates' changes. Both actions are necessary.



DO THIS

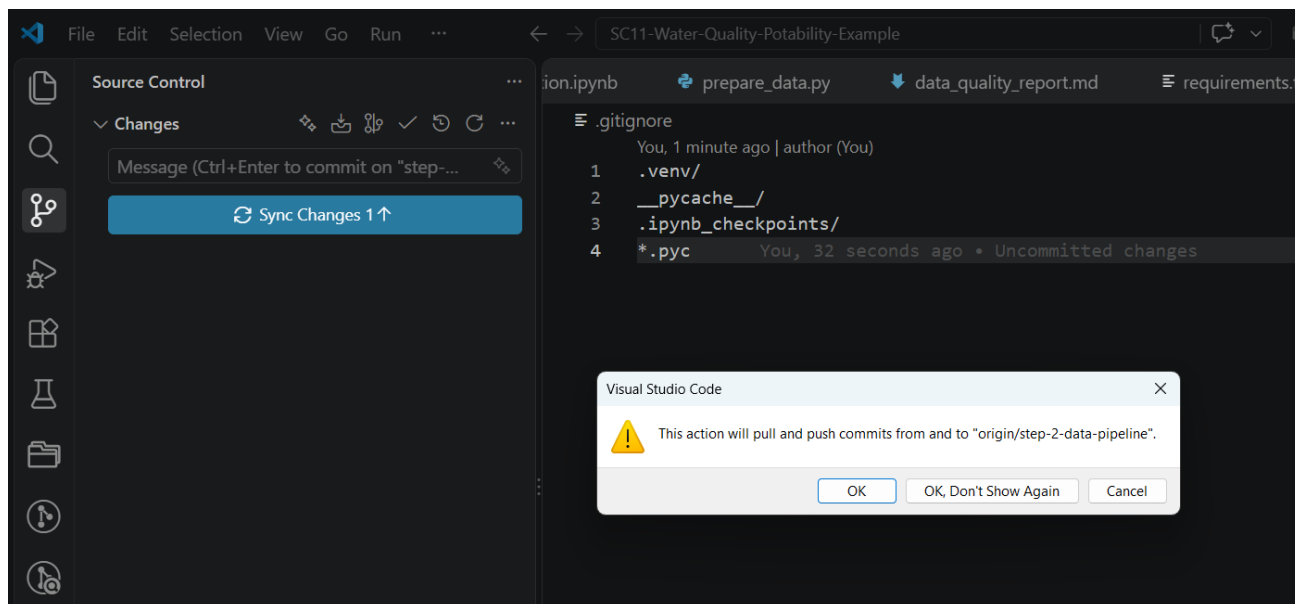
Click Sync Changes after the commit. Confirm that you are syncing the intended Step 2 branch.

EXPECTED RESULT

VS Code shows the Sync Changes confirmation dialog or progress state.

Action 62 - Confirm the branch sync

The confirmation protects shared work by telling you which remote branch will receive the commit. Read it before accepting.



DO THIS

Confirm only when the remote branch matches your working branch, for example origin/step-2-data-pipeline. Wait until the upload finishes.

EXPECTED RESULT

The commit is available on GitHub and Source Control has no pending changes.

Part C - Generated testing scenarios and final GitHub proof

Part C completes the SC11 teaching example by adding a small, reproducible generated testing-only scenario set and documenting it honestly. The generator is separate from the model data pipeline and the final GitHub check proves the complete work was synced.

Academic honesty rule

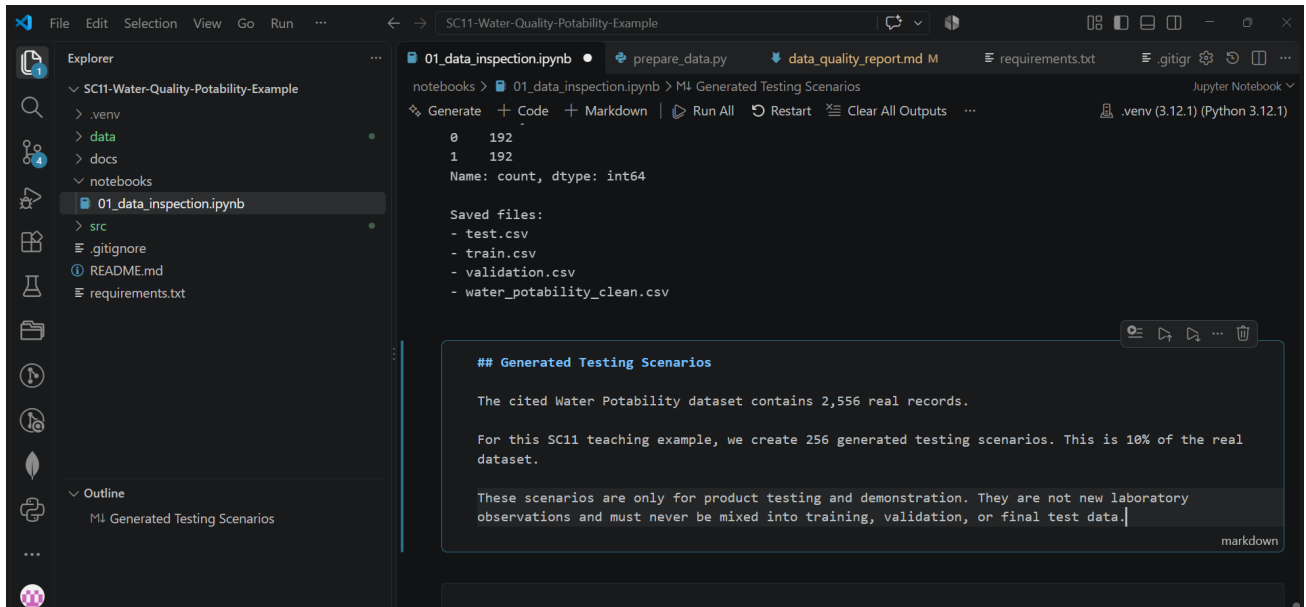
SC11 has 2,556 cited real records. Its 256 additional records are generated testing scenarios only. Never call them new laboratory records, and never mix them into train.csv, validation.csv or test.csv.

For allocated capstone teams

This SC11 extension demonstrates the correct method and documentation. It does not waive the official 10,000+ documented data/scenario requirement for allocated capstone projects. Follow your assigned project specification.

Action 63 - Explain real records and generated scenarios

A dataset must be described honestly. The cited SC11 dataset contains 2,556 real records. The extra 256 records are generated testing scenarios, not new water samples collected in a laboratory.



DO THIS

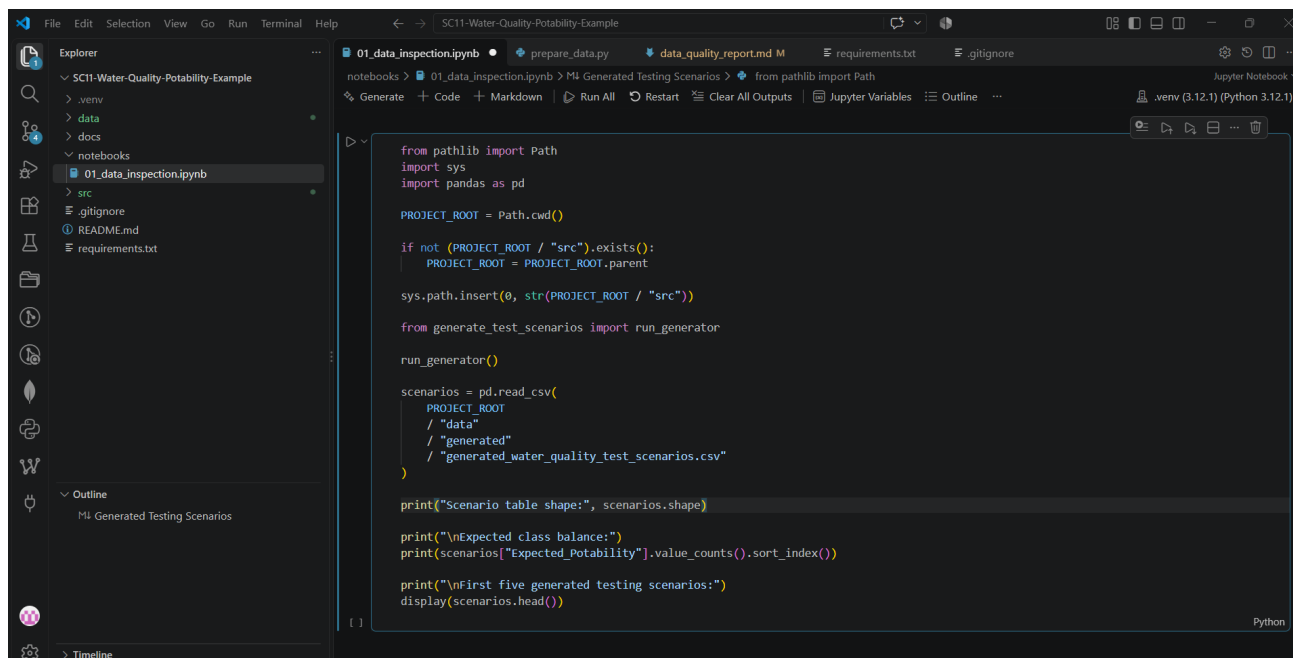
Add a Markdown cell at the end of the inspection notebook. State the real-record count, the 10% generated-scenario count, the testing-only purpose, and the rule that generated scenarios never enter training, validation or final test data.

EXPECTED RESULT

The notebook has a clear Generated Testing Scenarios heading and does not make a false claim about real data.

Action 64 - Run the scenario-generator notebook cell

The notebook runs the reusable generator and makes its result visible. This is proof that the process actually ran, not only that a CSV file exists.



```
from pathlib import Path
import sys
import pandas as pd

PROJECT_ROOT = Path.cwd()

if not (PROJECT_ROOT / "src").exists():
    PROJECT_ROOT = PROJECT_ROOT.parent

sys.path.insert(0, str(PROJECT_ROOT / "src"))

from generate_test_scenarios import run_generator

run_generator()

scenarios = pd.read_csv(
    PROJECT_ROOT
    / "data"
    / "generated"
    / "generated_water_quality_test_scenarios.csv"
)

print("Scenario table shape:", scenarios.shape)

print("\nExpected class balance:")
print(scenarios["Expected_Potability"].value_counts().sort_index())

print("\nFirst five generated testing scenarios:")
display(scenarios.head())
```

DO THIS

Import `run_generator` from `src/generate_test_scenarios.py`, run it, read the generated CSV, print its shape and class counts, then display its first five rows.

EXPECTED RESULT

The cell contains project-relative paths and clearly reads `generated_water_quality_test_scenarios.csv`.

Action 65 - Read the generated-scenario output

Visible output is assessment evidence. It proves the testing-only scenario generator ran with the stated data size and a fixed seed.

```
print("\nFirst five generated testing scenarios:")
display(scenarios.head())
```

SC11 GENERATED TESTING SCENARIOS COMPLETED

Real cleaned records: 2556
Generated testing scenarios: 256
Generation seed: 42
Output file: D:\sample_project\SC11-Water-Quality-Potability-Example\data\generated\generated_water_quality_test_scenarios.csv
Report file: D:\sample_project\SC11-Water-Quality-Potability-Example\data\reports\generated_scenarios_report.md
Scenario table shape: (256, 13)

Expected class balance:
Expected_Potability
0 128
1 128
Name: count, dtype: int64

First five generated testing scenarios:

	Scenario_ID	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity	Expected_Pot
0	GEN-WQ-001	8.723150	213.541215	18849.482843	8.684088	323.983758	394.470004	20.070274	96.538713	4.563208	
1	GEN-WQ-002	8.855924	223.058111	19396.677272	5.709123	395.766086	466.221103	10.409643	73.827611	3.475455	
2	GEN-WQ-003	5.614211	204.526839	16678.920204	5.590040	315.993728	342.099722	15.212163	73.402687	3.469140	
3	GEN-WQ-004	5.587951	188.626234	28616.778088	7.556170	326.163806	277.390151	8.373187	54.737168	2.569132	
4	GEN-WQ-005	8.165184	201.829737	25721.178228	7.631736	367.194996	597.524816	11.807852	72.546282	3.703196	

DO THIS

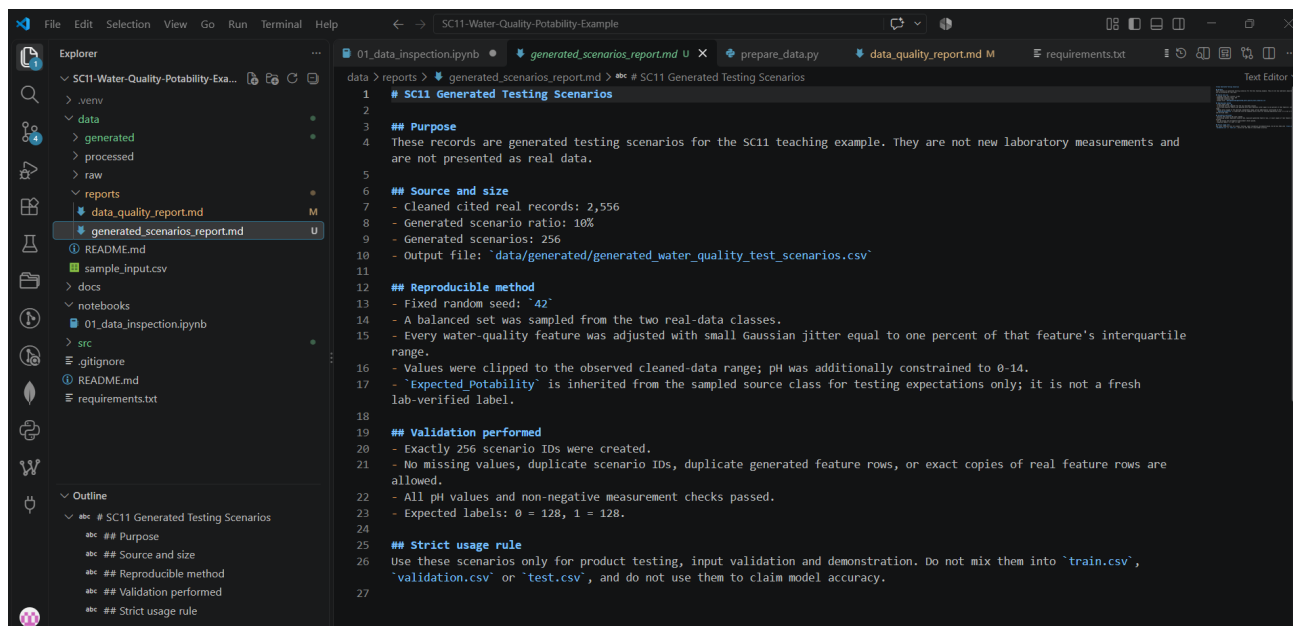
Run the cell and read the output. Check that it reports 2,556 real cleaned records, 256 generated testing scenarios, seed 42, table shape (256, 13), and 128 expected labels in each class.

EXPECTED RESULT

The output has no red error and the preview begins with IDs such as GEN-WQ-001.

Action 66 - Read the generated-scenarios report

A report lets an assessor understand the generated data without reading every line of Python. It records the source, size, method, validation and strict usage boundary.



```
1 # SC11 Generated Testing Scenarios
2
3 ## Purpose
4 These records are generated testing scenarios for the SC11 teaching example. They are not new laboratory measurements and
5 are not presented as real data.
6
7 ## Source and size
8 - Cleaned cited real records: 2,556
9 - Generated scenario ratio: 10%
10 - Generated scenarios: 256
11 - Output file: `data/generated/generated_water_quality_test_scenarios.csv`
12
13 ## Reproducible method
14 - Fixed random seed: `42`
15 - A balanced set was sampled from the two real-data classes.
16 - Every water-quality feature was adjusted with small Gaussian jitter equal to one percent of that feature's interquartile
17 range.
18 - Values were clipped to the observed cleaned-data range; pH was additionally constrained to 0-14.
19 - `Expected_Potability` is inherited from the sampled source class for testing expectations only; it is not a fresh
20 lab-verified label.
21
22 ## Validation performed
23 - Exactly 256 scenario IDs were created.
24 - No missing values, duplicate scenario IDs, duplicate generated feature rows, or exact copies of real feature rows are
25 allowed.
26 - All pH values and non-negative measurement checks passed.
27 - Expected labels: 0 = 128, 1 = 128.
28
29 ## Strict usage rule
30 Use these scenarios only for product testing, input validation and demonstration. Do not mix them into `train.csv`,
31 `validation.csv` or `test.csv`, and do not use them to claim model accuracy.
```

DO THIS

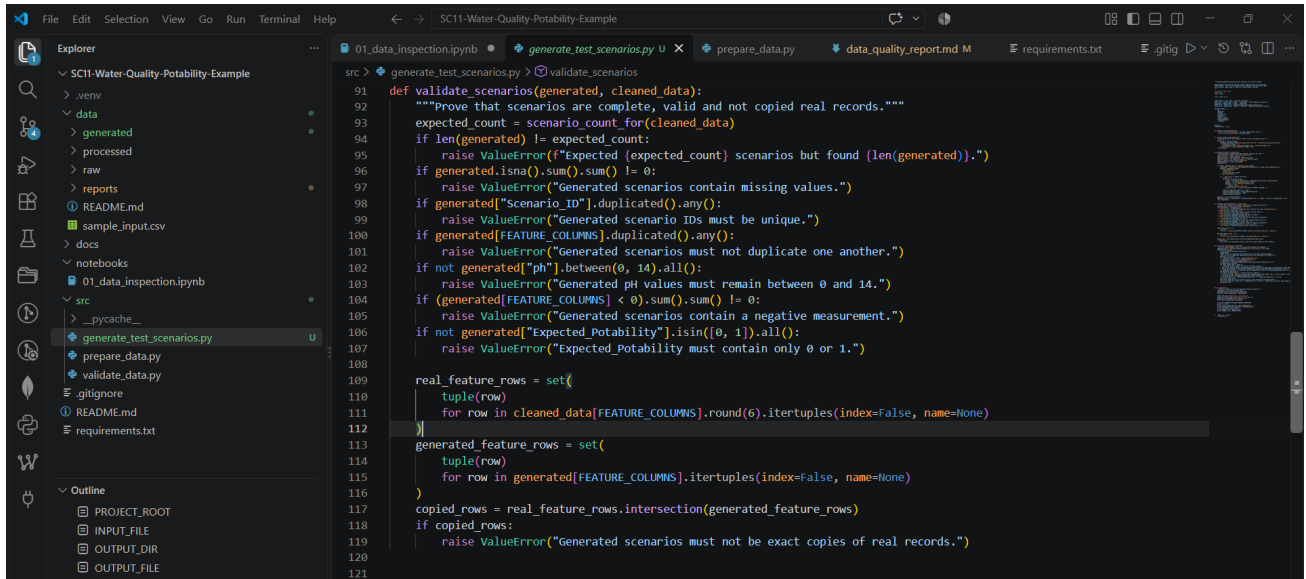
Open data/reports/generated_scenarios_report.md. Confirm the 2,556 real-record count, 10% ratio, 256 scenarios, seed 42, one-percent IQR jitter, validation rules and strict usage rule.

EXPECTED RESULT

The report clearly says generated scenarios are for product testing and demonstration only.

Action 67 - Inspect the safeguards in the generator

Generated testing scenarios must pass clear safety checks. Otherwise a team could accidentally duplicate real rows, create invalid measurements, or silently make an unusable scenario set.



```
def validate_scenarios(generated, cleaned_data):
    """Prove that scenarios are complete, valid and not copied real records."""
    expected_count = scenario_count_for(cleaned_data)
    if len(generated) != expected_count:
        raise ValueError(f"Expected {expected_count} scenarios but found {len(generated)}.")
    if generated.isna().sum().sum() != 0:
        raise ValueError("Generated scenarios contain missing values.")
    if generated["Scenario_ID"].duplicated().any():
        raise ValueError("Generated scenario IDs must be unique.")
    if generated[FEATURE_COLUMNS].duplicated().any():
        raise ValueError("Generated scenarios must not duplicate one another.")
    if not generated["ph"].between(0, 14).all():
        raise ValueError("Generated pH values must remain between 0 and 14.")
    if (generated[FEATURE_COLUMNS] < 0).sum().sum() != 0:
        raise ValueError("Generated scenarios contain a negative measurement.")
    if not generated["Expected_Potability"].isin([0, 1]).all():
        raise ValueError("Expected_Potability must contain only 0 or 1.")

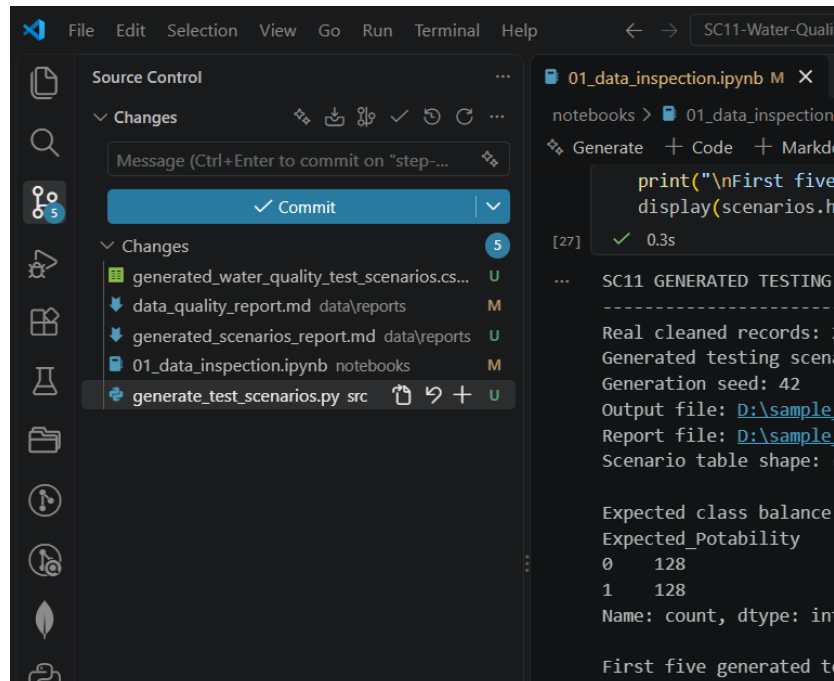
    real_feature_rows = set(
        tuple(row)
        for row in cleaned_data[FEATURE_COLUMNS].round(6).itertuples(index=False, name=None)
    )
    generated_feature_rows = set(
        tuple(row)
        for row in generated[FEATURE_COLUMNS].itertuples(index=False, name=None)
    )
    copied_rows = real_feature_rows.intersection(generated_feature_rows)
    if copied_rows:
        raise ValueError("Generated scenarios must not be exact copies of real records.")
```

DO THIS
Open src/generate_test_scenarios.py and locate validate_scenarios(). Read the checks for row count, missing values, unique IDs, duplicate generated rows, pH range, non-negative measurements, valid labels and no exact copies of real records.

EXPECTED RESULT
Every check raises an error when a rule fails. No generated scenario is added to train.csv, validation.csv or test.csv.

Action 68 - Stage the five intended files

Staging makes the Git checkpoint deliberate. It should contain the new generated CSV, both reports, the executed notebook and the reusable generator - not virtual-environment or temporary files.



DO THIS

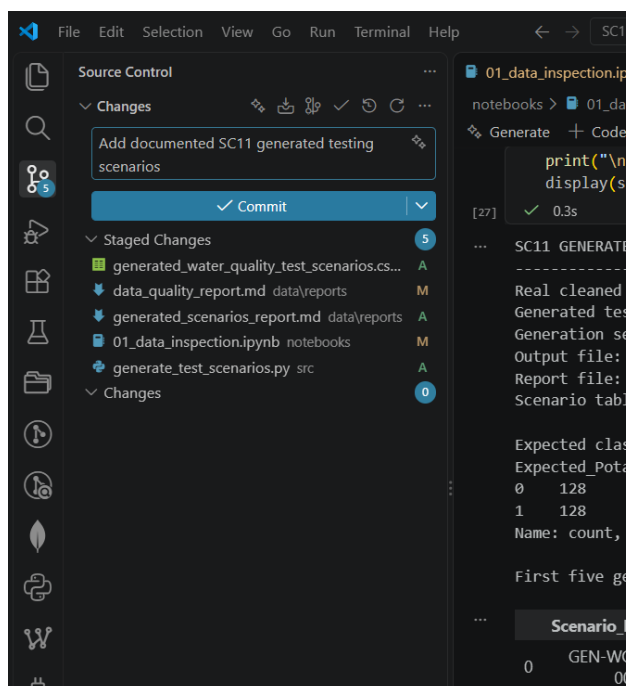
In Source Control, stage `generated_water_quality_test_scenarios.csv`, `data_quality_report.md`, `generated_scenarios_report.md`, `01_data_inspection.ipynb` and `generate_test_scenarios.py`.

EXPECTED RESULT

All five intended files appear under Staged Changes and Changes shows 0.

Action 69 - Commit the documented scenario work

A specific commit message makes the Git history easy to review. It tells a teacher or teammate exactly what changed in this checkpoint.



DO THIS

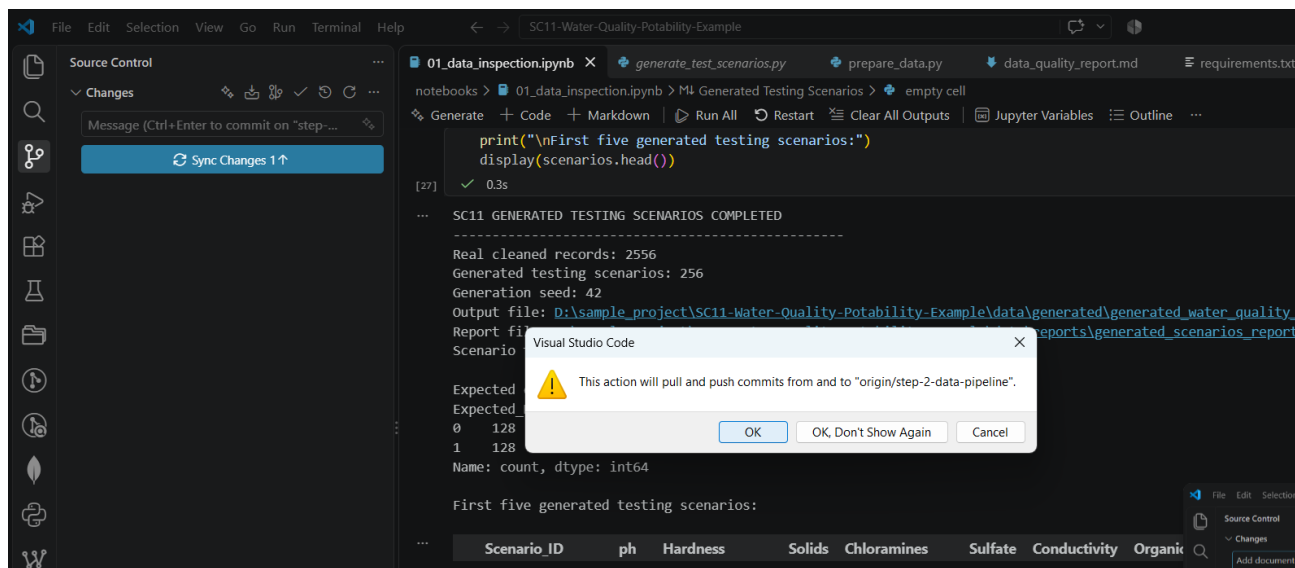
Enter Add documented SC11 generated testing scenarios, then click Commit. Do not use vague messages such as final, update or done.

EXPECTED RESULT

The staged list clears and VS Code offers Sync Changes.

Action 70 - Confirm and complete Sync Changes

Commit saves to your computer; Sync Changes publishes it to GitHub. The confirmation dialog identifies the remote branch that will receive the commit.



DO THIS

Click Sync Changes. Read the dialog and proceed only when it names origin/step-2-data-pipeline. Wait until the upload finishes.

EXPECTED RESULT

VS Code returns to a clean Source Control view and GitHub receives the new commit.

Action 71 - Verify the pushed commit on GitHub

GitHub verification proves that the work is backed up in the shared repository, rather than existing only on one computer. This is the final evidence checkpoint for Step 2C.

The screenshot shows the GitHub interface for the repository 'vipulshahi-ai / SC11-Water-Quality-Potability-Example'. The 'step-2-data-pipeline' branch is selected, showing it is 6 commits ahead of 'main'. A recent push notification is visible at the top. The file list includes folders like 'data', 'docs', 'notebooks', and 'src', along with files like '.gitignore', 'README.md', and 'requirements.txt'. The repository statistics on the right show 0 stars, 0 forks, and 0 releases.

File/Folder	Description	Time Ago
data	Add documented SC11 generated testing scenarios	1 minute ago
docs	Add files via upload	2 weeks ago
notebooks	Add documented SC11 generated testing scenarios	1 minute ago
src	Add documented SC11 generated testing scenarios	1 minute ago
.gitignore	Complete SC11 Step 2 data preparation pipeline	1 hour ago
README.md	Update README.md	2 weeks ago
requirements.txt	Complete SC11 Step 2 data preparation pipeline	1 hour ago

DO THIS

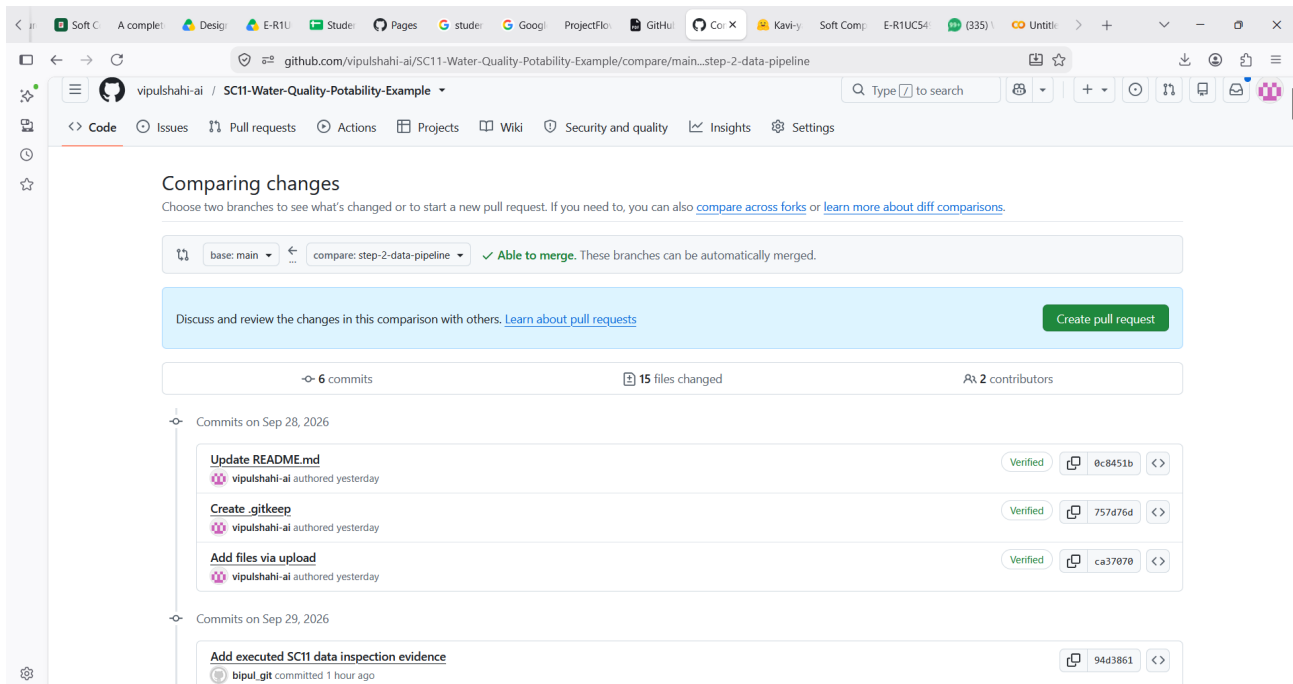
Refresh GitHub while step-2-data-pipeline is selected. Verify the recent push notice and the latest commit, Add documented SC11 generated testing scenarios. Do not create a pull request yet.

EXPECTED RESULT

The selected branch, latest commit, updated data/notebooks/src folders and recent-push notice are visible together.

Action 72 - Understand the branch comparison

A working branch being ahead of main is expected. It means your evidence is safely saved separately while development continues; it is not an error and it does not mean you must merge immediately.



DO THIS

If GitHub offers Compare & pull request, use the comparison page only to review differences. Do not click Create pull request during this student guide exercise.

EXPECTED RESULT

The comparison shows main as the base and step-2-data-pipeline as the comparison branch. GitHub says the branches are able to merge.

Step 2 completion checklist

A team is ready to move to Step 3 only when every relevant item below is visible in its repository and understandable to every member.

- [] A dedicated Step 2 branch exists and the team is working on it.
- [] data/README.md cites the source, licence, fields, units, assumptions and repeatable process.
- [] The original dataset or documented scenario source is retained in data/raw/.
- [] src/prepare_data.py contains reusable loading, validation and preparation logic.
- [] An executed .ipynb notebook displays real outputs beneath cells.
- [] data/processed/ will contain cleaned usable data; raw data remains unchanged.
- [] data/reports/data_quality_report.md reports rows, missing values, duplicates, ranges and class/scenario counts.
- [] ANN teams use a reproducible train/validation/test split. Fuzzy and GA teams include normal, boundary, stress and invalid scenarios.
- [] For an allocated capstone: at least 10,000 valid documented records or scenarios are prepared without pretending duplicated records are new observations. The SC11 guide's 256 generated scenarios are a teaching example only.
- [] Every member makes meaningful Git contributions; the latest work is committed and synced to GitHub.

Safety and academic honesty

SC11's prediction is an educational decision-support example and does not replace laboratory testing or official certification. Every project must cite sources and clearly separate real labelled data from generated testing scenarios.

How every project adapts this example

The folder structure and Git process are common to every capstone. Only the project data and validation rules change.

Project type	Use Step 2 for
ANN / data project	Cited public data, privacy check, cleaning, feature preparation, fixed train/validation/test split and data-quality report.
Fuzzy controller	Cited realistic input ranges, reproducible scenario generator, units, normal/boundary/stress cases and validation rules.
Genetic Algorithm	Reproducible instances/scenarios, constraints, cost or fitness inputs, normal/difficult cases and source/range documentation.
Hybrid project	Prepare each component's data/scenarios separately and document how the components will connect in M2.

Next after Step 2

Step 3 is the baseline: a simple non-Soft-Computing comparison method. Do not start it until your team can reproduce, validate and explain its Step 2 data pipeline.